

从黑圣杯到令咒系统：长程实践的离散本质与 Agent 治理边界

From the Black Grail to the Command Spell System: The Discrete Nature of Long-Horizon Practice and the Boundaries of Agent Governance

曾庆明 (Sammy Zeng)

2026 年

¹联系邮箱: jop.sammy@163.com ORCID: 0009-0007-4382-0040

²本文为杭州市上城区钱江新城中央商务产业社区专项支持项目“长程实践理论”的研究成果，社区在调研对接、基础压力测试样例、真实业务用例方面提供了关键性支持（非经费资助）

Abstract

这篇论文回答一个朴素的问题：怎么做出更好的 Agent 系统？

七个主流开源 Agent 系统的源码级调研表明，模型调用协议（原核）已在 ± 20 行代码内收敛；真正的分水岭是治理基础设施（真核）——状态治理维持跨时间的连续性，权力治理定义与外部世界的行动边界。本文提出三层同心模型（原核/真核/皮层）作为分析框架，并基于长程实践的形式定义推导出七个核心工程决策节点。核心判断是：当任务存在不可预计算的顺序依赖时，离散检查点、权限闸门、结构化状态传递和机向人请示等治理机制不是工程可选项，而是结构必需品。

更深一层：治理的进步无法来自自动优化——Agent 系统中“什么是对的”在执行前不存在，只能从真实部署的失败中观察和学习。这正是 Agent 系统走“演化”而非“设计”路径的深层原因。治理不是能力的枷锁，而是释放 LLM 智能潜力的结构性前提。

关键词： Agent 治理；长程实践；三层同心模型；离散本质；人机协同

Contents

1 引言	6
1.1 Agent 生态的现状与追问	6
1.2 已有综述的演进与盲区	6
1.3 七系统源码级调研的起点	7
1.4 本论文的定位	7
1.5 什么是好 Agent	8
1.6 主要贡献	8
2 做好 Agent 系统需要做到什么	11
2.1 决策零：你在做的是 Agent，还是别的什么？	11
2.2 决策一：建立状态治理	11
2.3 决策二：建立权力治理	12
2.4 决策三：检查点分解与独立验证	12
2.5 决策四：设计中断边界	13
2.6 决策五：能力与治理分离	13
2.7 决策六：承认治理无法自动优化	14
2.8 决策七：知道什么不可能	14
2.9 决策框架总结	15
2.10 框架的边界：诚实性声明	15
3 狭义 Agent 的定义与边界	16
3.1 收窄的三步推导	16
3.2 三层同心模型：原核、真核与皮层	18
3.2.1 原核（Prokaryon）：已高度收敛的调用协议	18
3.2.2 真核（Eukaryon）：运行时治理基础设施	19
3.2.3 皮层（Cortex）：渐进式披露的能力供应链	19
3.2.4 三层的递进关系	19
3.3 治理的严格定义	19
3.4 运载前提预埋：联合认知系统的结构上限	20
4 演化论视角：七系统的分歧	21
4.1 各系统的初始条件与演化路径	23
4.2 分歧之下的收敛：原核为何不变，皮层为何不可预测	27
4.3 演化史上的集体盲区：L4 跨执行断面的交接治理	28

5 原核 (Prokaryon): 已高度收敛的调用协议	29
6 真核 (Eukaryon): 真正的战场	31
6.1 二分法的核心洞察: 状态治理与权力治理	31
6.2 状态治理	32
6.2.1 持久化策略频谱: 七个系统在何处画线	32
6.2.2 上下文治理的多层降级路径	34
6.2.3 记忆 vs 状态: 位置之争	34
6.2.4 状态传递的三段交接	35
6.3 权力治理	36
6.3.1 权限管线的三层不可互相替代	36
6.3.2 价值闭合与机向人主动请示	37
6.3.3 回滚与隔离: 限制错误的传播半径	39
7 真核框架的扩展: 多 Agent 场景与治理成本经济学	40
7.1 多 Agent 场景: 注意力分配、权力归属与规划层指引	41
7.1.1 核心原则: 不应限制子 Agent 的推理深度与信息获取能力	41
7.1.2 性能面: Subagent 作为媒介强制的结构应对	41
7.1.3 治理面: 状态主权、提交权与责任归属	42
7.1.4 表现面: 超深度子 Agent 的用户监督	42
7.2 治理成本经济学	43
7.2.1 真核管理的不是道德, 是三种稀缺资源	43
7.2.2 真核的”重”是对不可逆偏航风险的高质量对冲	43
7.2.3 治理成本的优化方向	44
8 皮层 (Cortex): 渐进式披露的能力供应链	45
8.1 半透膜: 模型的注意力空间与外部世界的选择性边界	45
8.2 穿透管道的两个维度与机制定位	46
8.3 渐进式披露作为膜的调度问题	47
8.4 Hermes 工具注册中心: 皮层层工程典范	48
8.5 皮层的架构性质: 可插拔性与真核治理分工	49
9 长程实践对 Agent 治理的结构要求	50
9.1 不可预计算顺序依赖链的形式定义	50
9.1.1 静态可折叠与关键输入	50
9.1.2 长程不是时间长——是不能预计算	51
9.2 形式定义的治理后果	52
9.3 在七系统中的实证投影	53
9.4 三段非蕴含	54

9.5 收束	54
10 Agent 实践的离散本质	54
10.1 Agent 实践在工程上必须被设计为离散的	54
10.2 ”连续体验”的实质	56
10.2.1 认知缝合：工作记忆对离散片段的叙事重构	56
10.2.2 流式输出：降低了边界感知，没有消除离散本质	57
10.2.3 人类喜欢连续，但需要离散	57
10.3 回应了什么：对三种流行预设的逐条澄清	58
10.3.1 预设一：”Agent 需要连续记忆”	58
10.3.2 预设二：”Agent 需要永不中断的会话”	58
10.3.3 预设三：”Agent 需要连续体验”	58
10.4 实践指导价值：离散操作元素承载连续体验流	59
10.5 离散机制提供了连续机制无法提供的三种治理功能	60
10.6 收束与桥接	61
11 表现层的收敛	61
11.1 EC-6/EC-7 的工程载体要求	62
11.2 三个必要条件	63
11.3 收敛判断：类 IDE / 工作台形态	64
11.4 弱化限定：功能形态的收敛，不是产品形式的唯一化	64
11.5 从表现层回看治理：三层模型的完整性	66
12 反馈信号的结构性不完备	66
12.1 两类优化范式的对比	67
12.2 语义不封闭四维框架	67
12.3 ”手工梯度下降”	69
12.4 对”演化 vs 设计”的结构性解释	70
12.5 本章收束	71
13 系统工程与 Agent 工程：边界与融合	72
13.1 核心洞察：零干预任务的可工程化上限	72
13.2 Agent 工程的互补定位	74
13.3 非互斥而是互补：动态循环	74
13.4 两个对称的失败模式	76
13.5 本章收束	77
14 讨论与开放问题	78
14.1 长程 Benchmark 设计：从形式变量到操作化指标	78

14.1.1 五条设计原则	78
14.1.2 三个操作化指标	79
14.2 H2 困境：当操作者无法独立判断产出质量	81
14.3 四个开放问题	83
14.3.1 手工梯度下降的抽样偏差瓶颈	84
14.3.2 实践闭合的可判定性	84
14.3.3 通用可插拔 vs 平台深度特调的量化取舍	85
14.3.4 语义不封闭 → 稳定反馈的转化	85
14.4 本章收束	85
15 结论：从能力竞赛到治理建设	85
16 致谢	86
A 七系统演化证据：技术细节与源码锚点	86

1 引言

1.1 Agent 生态的现状与追问

做了这么多研究，思考了这么多第一性原理，是为了什么？——为了做出更好的 Agent 系统。

截至 2026 年 7 月，以 LLM 为核心推理节点的 Agent 系统已进入前所未有的繁荣期。七个独立演化的 Agent 编码系统各有数以万计的真实用户和数千人年的工程投入，却在相似的技术基座上呈现出截然不同的架构选择。

然而，这一繁荣伴随着深刻的碎片化。七个系统在核心循环上仅相差 ± 20 行代码，却在状态持久化、权限管线、工具治理、上下文压缩和跨会话经验积累等维度上走向了截然不同的方向。这些差异不是“偏好”——它们是每个系统的初始环境条件（不可信执行环境、高价值人类注意力、状态不可靠性与跨会话知识蒸发）在工程史上留下的不可互约的结构化石。

这种碎片化提出了一个尖锐的问题：**Agent 系统的真实差异究竟在哪里？**是模型推理能力？是工具生态的丰富性？是提示工程的精巧程度？还是某种更深层但尚未被充分梳理的架构维度？

1.2 已有综述的演进与盲区

在回答上述问题之前，有必要正视已有综述文献在这一问题上的贡献与边界。过去三年间，三篇具有里程碑意义的综述分别从不同视角系统化了 Agent 研究，但它们在回答“差异在哪里”的问题上，各自留下了一个结构性的盲区。

第一篇是 Bang Liu 等人提出的 Foundation Agents 综述 (2504.01990)^[1]。该综述以人脑功能架构为蓝图，将 Agent 分解为认知、记忆、世界模型、奖励、情感、感知和行动七大模块，并给出了每个模块的形式化定义和当前技术状态。这是迄今为止对“Agent 应该具备什么组件”最全面的脑科学指导框架。然而，它的讨论止于功能组件层——回答了“Agent 应该具备什么”，却未回答“这些组件如何被组织成可持久、可恢复、可治理的运行系统”。七个系统都拥有全部七大模块的某种实现——但它们在工程上的可靠性却相差悬殊。功能清单无法解释这个差距。

第二篇是 Wang 等人提出的 LLM 自主 Agent 综述 (2308.11432)^[2]，首次提出 Profile-Memory-Planning-Action 四组件统一架构，为 Agent 领域建立了一套共享的功能分类语言。随后几乎所有 Agent 论文都在其术语框架内描述自己的架构。但该综述的分类属于**认知功能层**——它关心的是 Agent 在任务中进行了哪些认知活动（记住、规划、行动），而非**运行时组织层**——这些活动如何被治理（持久化、恢复、审批、隔离、交接）。一个 Agent 可以完美地“规划”了下一步，同时因为缺乏权限审批而在未经授权的情况下执行了不可逆操作。认知功能分类无法诊断这个失败。

第三篇是 Meng 等人提出的 Agent Harness 综述 (2605.29682)^[3]，首次将 Agent 的运

运行时基础设施形式化为 $H = (E, T, C, S, L, V)$ 六元组，并明确提出了“Harness 即基础设施问题”（harness-as-infrastructure problem）的核心论点：Agent 部署的可靠性越来越不取决于底层模型，而取决于封装模型的运行时基础设施层。该综述以功能完备性为分析轴，系统比较了 23 个代表性系统在六个组件上的完备程度。然而，其分析框架未从长程实践的信息结构出发推导哪些治理机制是结构必需品、为什么缺少它们系统会在何处发生结构性断裂。功能完备性告诉你“一个系统有哪些组件”，但不能告诉你“哪些组件在什么条件下从增强变为必需”。

将三篇综述的贡献与盲区并列，可以看到一条清晰的演进轨迹：Liu 等人建立了“应该有什么”的组件清单，Wang 等人建立了“在做哪些认知活动”的功能分类，Meng 等人建立了“跑在什么基础设施上”的完备性矩阵。但它们共享同一个盲区——没有从长程实践的信息结构出发，论证一组运行时治理约束为何是**结构必需品**而非**工程可选项**。这正是本论文试图填补的缺口。

与邻域的关系。本文的治理框架在概念上与分布式系统的 checkpoint/recovery、capability security 的最小权限原则、HCI 中的 human-in-the-loop 研究存在交汇。但这些领域的核心假设与 Agent 长程实践的条件存在结构性差异——Agent 的产出是非确定性的、失败模式不能预先穷举、权限边界在执行中才逐步显现、且操作者面对 H2 困境时常无法独立评估系统产出。本文的贡献是从这些差异出发，推导一组在 Agent 特有的信息不完备条件下成立的治理约束，而非否认邻域价值。

1.3 七系统源码级调研的起点

本文的经验基础不是对已有论文的二次聚合，而是对七个主流开源 Agent 系统的源码级调研。七个系统分别是 Claude Code、Codex CLI、OpenCode、Hermes、Cline、Cherry Studio 和 trae-agent。源码刺探覆盖了每个系统的核心循环、状态持久化、权限管线、工具治理、上下文压缩、能力扩展和内部通信机制，每个系统的描述都有具体文件路径、函数签名和配置项级别的证据支撑。七系统的选择标准是：拥有真实用户群体、源码可访问、覆盖不同初始场景（终端/IDE/独立服务/桌面/研究）。

源码级调研的一个重要发现是：现有 Agent 系统的核心架构全部是演化产物——第4章将展开这一判断的完整论证。

1.4 本论文的定位

基于上述分析，本文提出三层同心模型——原核（调用协议骨架，第5章）、真核（治理基础设施，第6章）、皮层（渐进式披露能力层，第8章）——用以解剖 Agent 系统的运行时架构。这不是对已有综述的否定，而是从一个它们尚未触及的角度提供互补的分析维度。

更深一层，本文论证：Agent 治理本质上是**保下限，不是撑上限**——确保在信息不完备、价值不确定时系统不至于断裂。但它不能替代人类判断力、价值取舍和外部现实

锚定（第9.2节）。

1.5 什么是好 Agent

这篇论文的出发点是一个朴素的愿望：做出更好的 Agent 系统。

什么叫“更好”？回答这个问题的尺度，不是跑分，不是用户量，而是它是否更好地释放了 LLM 的潜力——让 LLM 对现实世界产生更有效的因果干预，并且让这些干预不偏离人类意图与价值。这是这篇论文对好 Agent 的定义。

但一旦落到工程上，这个目标立刻变得尖锐。一个模型在单次推理中足够聪明——它能看懂复杂代码库、能生成精确的操纵指令。但当它需要连续数百步执行任务，而每一步的结果都不可预计算时，问题不再是“模型能不能想清楚”，而是“系统能不能在长程中不失控”——模型再强，不能凭空生成尚未被世界产生的真值；推理再深，不能在上下文溢出后自动恢复任务方向。

这就是本文的核心判断所在：当基座模型已经足够好，Agent 系统之间的真实差异不在模型本身，在治理结构。七个主流开源系统的源码调研反复印证了这一点——它们在调用协议上仅差 ± 20 行代码，却在状态持久化、权限管线、跨会话知识上走向了截然不同的方向。长程实践的离散本质决定了：离散检查点、权限闸门、结构化状态传递和机向人请示等机制，是系统在不可预计算顺序依赖下维持连续性的结构约束。

因此，本文的回答是：**好 Agent 系统是治理结构完备的系统**。治理不是能力的枷锁——恰恰相反，它是释放 LLM 智能潜力的结构性前提。能力让你走得更快，治理决定你能否走完全程。后文十四章的论证从三层同心模型出发，经长程实践的形式定义、离散本质的分析、到反馈信号结构性不完备的揭示，逐层收敛于一个核心判断。每一步都在回答那个最朴素的问题：怎么做出能真正释放 LLM 潜力的 Agent 系统。

需要说明的是：本文在治理保下限维度上给出了完整的工程框架与形式化论证。如何让 Agent 的单步干预能力本身更强——模型推理更优、工具生态更丰富——不在本文论证范围。这篇论文回答的是“可靠性从哪里来”，将“能力如何更强”留给后续工作。

1.6 主要贡献

本文的主要贡献如下：

1. **七系统源码验证原核收敛性**：通过对 Codex CLI、Claude Code、OpenCode、Hermes、Cline、Cherry Studio 和 trae-agent 七个系统的源码级比较，以具体文件路径和代码行数为证据，建立“原核在各系统中已收敛于 ± 20 行代码差异”的经验事实，并指出在当前 API 调用范式下原核的设计空间已接近饱和——七个系统独立演化至几乎相同的结构，强烈暗示此层已无实质性的架构创新空间（第5章）。
2. **真核二分法框架（状态治理与权力治理）**：提出真核内部的状态治理/权力治理二分法，论证两者在诊断、预测和修复路径上的不可互代性（状态断裂不能用权限规

则修复，权力越界不能用持久化修复），并以七个系统的权限管线三层结构（工具可见性/权限审批/执行隔离）和持久化策略频谱（L0-L3）为实证支撑（第6章）。

3. **长程实践理论解释真核结构必要性：**将长程实践理论的核心结论（不可预计算顺序依赖链的形式定义、四种信息截断来源、三层结构下限）系统地映射到 Agent 治理的上下文中，证明真核的核心治理机制（HC-3 状态传递、SC-1 价值闭合、SC-3 未闭合标记、EC-7 机向人请示）在 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的条件下从工程可选项升级为结构必需品（第9章）。
4. **EC-6/EC-7 推导表现层收敛：**从 EC-6（机向人主动解释）和 EC-7（机向人主动请示）两条工程补偿条件的信息结构出发，推导出任何满足这两条条件的表现层必须同时具备信息密度高、操作时延低和状态可追溯三个工程条件，并论证在 2026 年的技术条件下，同时满足这三个条件的功能形态高概率收敛于类 IDE/工作台形式。收敛的是功能形态而非产品形式（第11章）。
5. **反馈信号结构性不完备的系统揭示：**揭示 Agent 治理反馈与 LLM 训练反馈之间的结构鸿沟，提出目标语义、评价维度、后果空间和反馈归因四重不封闭作为根因的完整框架，并论证这一结构性不完备是七个 Agent 系统的框架全部走”演化”而非”设计”路径的根本原因（第12章）。

此外，本文在讨论章（第14章）中给出了长程 Benchmark 的五条设计原则与三个操作化指标、H2 困境的结构性分析，并对后续开放问题（实践闭合的可判定性、工具行动语义标准化、通用可插拔 vs 平台深度特调、语义不封闭到稳定反馈的转化）做出了诚实的暴露与方向性讨论。

术语速查表

以下为本论文中关键术语及其首次出现的章节号。编号体系术语的完整定义见对应章节正文。

术语	简要含义	首次出现
Prokaryon（原核）	Agent 系统的调用协议骨架——while 循环 + 工具调用 + 上下文管理	第2章（决策框架），完整定义见第3章
Eukaryon（真核）	Agent 系统的治理基础设施——状态持久化、权限管线、审批闸门	第2章（决策框架），完整定义见第6章

术语	简要含义	首次出现
Cortex（皮层）	Agent 系统的渐进式能力披露层——在恰当时间暴露恰当工具	第2章（决策框架），完整定义见第8章
HC-1	目的闭合——每个检查点必须有可操作的闭合标准	第6章
HC-2	权力闭合——不可逆操作必须有前置审批	第6章
HC-3	状态闭合——状态必须在检查点间可传递	第6章
SC-1	价值闭合——局部目的正确性不能由目的自身判定	第6章
SC-2	顺序闭合——任务路径不能因执行顺序而不可复现	第6章
SC-3	未闭合标记——系统必须能区分三种闭合状态	第6章
EC-1	恢复与回滚——系统必须能从任何检查点恢复	第9章
EC-2	隔离与边界——每个检查点的执行不能污染其他检查点	第9章
EC-3	独立评测——闭合判定不能由执行者承担	第9章
EC-4	指令遵循补偿——系统必须补偿 LLM 的非确定偏离	第9章
EC-5	能力缺失升级——当 Agent 缺少完成任务所需能力时必须主动升级	第9章
EC-6	机向人主动解释——系统必须在触发人类介入时提供可操作的结构化上下文	第9章
EC-7	机向人主动请示——系统必须在价值不确定时主动请求人类裁决	第9章
$K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$	第 i 步之后产生的关键信息在第 i 步之前不可被静态折叠——长程实践的形式判据	第3章
L0–L4 持久化频谱	从无持久化 (L0) 到全量审计追踪 (L4) 的状态持久化五级频谱	第9章
H2 困境	人类操作者不具备独立验证 Agent 产出所需全部领域知识的结构性困境	第13章

Table 1: 论文关键术语速查

2 做好 Agent 系统需要做到什么

引言确立了“Agent 系统的差异在治理”这一核心判断。本章将此判断展开为一组可工程落地的决策框架——它回答的不是“为什么治理重要”，而是“做好 Agent 系统具体需要做到什么”。

本章是全文的结论前置。在后续各章逐层展开论证之前，本章先用实践者的语言给出做好 Agent 系统的决策框架：一个前置筛选问题，加上七个核心决策节点，以及这个框架的诚实边界——它不能做什么、在什么条件下不适用、有哪些问题目前还没有答案。

2.1 决策零：你在做的是 Agent，还是别的什么？

为什么。把错误的工具用在错误的场景，是你能犯的最贵的错误。在考虑“怎么做 Agent”之前，你需要先确认：你的场景真的需要一个 Agent。

必须做的。确认你的系统需要执行不可逆的外部动作（修改文件、调用 API、提交代码变更），并且后续步骤依赖这些动作的实际结果。如果你要解决的问题可以用一个确定性的函数调用、一个 if-else 分支、一个固定的脚本序列完成，你不需要 Agent。你需要的是一个更快的函数，不是一个更聪明的循环。

应该做的。在决定上 Agent 之前，先花半小时确认：你场景中的“下一步做什么”是否真的依赖上一步执行后产生的新信息——而且是那种在规划阶段无法预计算的新信息。如果是，你进入了 Agent 的正当地盘。如果不是，Agent 框架带来的复杂性不会让你的产品更好，只会让你的工程更乱。

不应该做的。不要把确定性的死代码包装成 Agent。不是所有“调用 LLM”的系统都自动成为 Agent。当你发现 Agent 在 90% 的时间里都在走同一条推理路径、做同一个工具调用序列时，你需要的不是“优化 Agent 的提示词”，而是把那 90% 提取为一个非 Agent 函数。

本章的定位。决策零是框架的前置筛选条件，不计入七个核心决策节点。全文统一以“七个决策节点”为口径。如果你通过了这个筛选（你的系统确实需要执行不可逆外部动作，且后续步骤依赖这些动作的实际结果），那么接下来的七个决策节点是你应该逐项考虑的。

2.2 决策一：建立状态治理

为什么。如果你的系统在执行了步骤五之后不记得步骤四发生了什么，它在结构上就不是一个长程系统，而是一系列相互遗忘的短程片段。

必须做的。为你的系统持久化结构化的状态快照。这至少包含四个字段：(1) 已经做到了哪一步；(2) 当时为什么那样做（而非仅记录“做了什么”）；(3) 当前有哪些未闭合的待决项；(4) 从哪里接续。四个字段缺一不可——多字段的快照才能让接续者重建执行语境，而非仅知道自己“看到了编号 N 的输出”。

应该做的。状态快照应该在每个关键检查点之后自动写入持久化存储，并且可以在系统崩溃、重启或上下文窗口溢出后被可靠地读取和恢复。理想情况下，状态快照采用人类可读的格式——如果接续者是人类而非机器，他们能理解之前发生了什么。

不应该做的。不要把“记忆”（RAG 检索、向量库、对话历史摘要）和“状态”混为一谈。记忆告诉你“昨天讨论过什么主题”——这对回顾有帮助，但不能替代“当前做到哪了、卡在哪里、接下来该做什么”。不要因为系统的流式输出很流畅，就误以为它具备跨步骤的连续性。流式输出是显示层的流畅，不是执行层的连续。

完整论证见第6章。

2.3 决策二：建立权力治理

为什么。一个能执行不可逆操作的系统，如果没有操作边界，不是“更强大”——是更危险。不是因为模型“可能变坏”，而是因为信息不完备是长程实践的常态。在信息不完备的条件下，预先规定所有“不应做的事”在数学上不可能。

必须做的。为你的系统建立明确的权限管线。至少包含两层：(1) 被许可的操作集（哪些工具、哪些资源范围）；(2) 不可逆操作前的强制审批闸门：在执行文件删除、外部 API 写操作、代码合流等功能之前，预设一道人类审批步骤。你的系统不应该在没有人确认的情况下执行不可逆操作。

应该做的。当系统面临“值不值得做”而非“能不能做”的判断时，它应该将问题转化为人类可选择的形式——不是抛回“你觉得呢”，而是给出 2-4 个有结构的选项，每个选项附带可验证的后果估计。这是行为转型：从让人类填空，到让人类做选择题。

不应该做的。不要用“模型看不到某个工具”来替代“系统不会执行该工具”——模型的工具可见性不是安全边界。不要用“Agent 应该更安全”“Agent 应该对齐人类价值”等模糊规范替代操作前的审批闸门——规范不阻止任何操作，审批闸门才阻止。不要用“我们信任模型”替代“我们在结构上不信任任何单次推理”。

完整论证见第6章。

2.4 决策三：检查点分解与独立验证

为什么。Agent 最危险的失败模式不是“它什么都没做”，而是“它在错误方向上高效推进”。当每一步输出看起来合理、但整体方向已经偏离时，没有独立验证的系统会一路走到偏离的终点才发现。

必须做的。把任务分解为独立的检查点，每个检查点有可操作的闭合标准：是“测试套件全部通过”而非“感觉差不多了”。闭合判定由独立于执行者的验证机制（自动化测试、编译检查、校验和）承担，不能依赖 Agent 说“我做完了”。

应该做的。检查点的粒度应该设计成“单次人类注意力跨度可覆盖”——一个检查点的产出量不超过你一次能审阅的范围。检查点的闭合标准应该预先定义，而非在执行过程中临时协商。

不应该做的。不要依赖 Agent 自述“我完成了任务”。Agent 告诉你“已完成”和你验证“已完成”之间的差距，是几乎所有长程 Agent 系统最大的风险敞口。

完整论证见第9章。

2.5 决策四：设计中断边界

为什么。人类能关注 Agent 执行的时间是有限的。如果你的系统没有预设的中断点，当人类注意力耗尽时，系统将进入无监督执行——而没有任何治理机制能在无监督条件下发挥作用。

必须做的。在关键决策节点上显式制造中断面：子任务完成后、发现异常信号时、需要价值判断时、执行进入未覆盖区域时。这些中断面是预设的，不是临时起意的。它们是你的系统设计与治理机制配对的结构特征。

应该做的。中断面的位置应该用“执行在什么情况下必须停下来等人类回应”来定义，而非用“每隔 N 步中断一次”的机械频率来定义。关键的中断面包括：不可逆操作之前（与决策二的审批闸门配对）、子任务闭合判定时（与决策三的独立验证配对）。

不应该做的。不要假设“需要时再打断”。人类操作者不会在注意力集中时持续监控 Agent 的执行。这是人类注意力的结构特征，不是自律问题。不要把“永不中断的流畅对话”作为架构目标：流畅是用户界面的特性，不是 Agent 系统的可靠性保障。

完整论证见第10章和第11章。

2.6 决策五：能力与治理分离

为什么。如果你因为不信任子 Agent 而限制它的能力和工具，你在用“让它变笨”来应对“不知道它会不会做错事”。这不是治理——这是自我阉割。在多 Agent 系统中，信任不足的正确答案不是限制能力，而是让治理层独立于能力层。

必须做的。将系统的架构分成两层：治理层（验证、审批、边界执行）和能力层（工具调用、推理、代码生成）。治理层约束的是“接收什么、验证什么、什么条件下放行”，而不是“子 Agent 能调用什么工具、用什么模型推理”。子 Agent 应该拥有完成其任务所需的全部计算能力和工具。治理层的职责是判断它的产出是否符合放行条件，而非限制它生产过程的能力。

应该做的。在治理层预设明确的验证规则——代码必须先通过编译检查再合流，配置变更必须先经过 diff 审阅再部署，数据迁移必须先的沙箱中验证再执行。这些规则对治理层面是操作性的，对执行层面是透明的：子 Agent 不需要知道验证规则的存在，它只需要产出可被验证的结果。

不应该做的。不要用限制子 Agent 的工具集来应对信任不足。当你说“我不敢让子 Agent 访问文件系统”时，你实际在说两件事：(1) 你需要在治理层增加文件系统操作的审批闸门；(2) 你不应该在能力层删除文件系统的工具。把这两个操作混在一个动作里——删除工具——意味着你的系统永远失去了在治理到位后发挥这项能力的机会。

完整论证见第8章和第13章。

2.7 决策六：承认治理无法自动优化

为什么。你无法设计一个能自动优化自己的 Agent 治理系统。不是因为算法不够好，而是因为评估一个治理决策好坏的信号，在执行完成的当下无法被自动获得。治理质量只能在事后、由人类、基于实际后果来判定。

必须做的。接受”手工梯度下降”（manual gradient descent）作为唯一的治理迭代范式：观察你的系统在实际运行中的失败模式 → 诊断失败信号 → 修改治理规则或架构 → 验证修改是否阻止了同类失败。这个循环的每一步都需要人类判断，无法被自动化。

应该做的。为你的治理系统设计可观测性：记录每次审批闸门被触发时的上下文、每次任务偏离被发现时的执行轨迹、每次人类介入后的系统行为变化。这些记录是你执行手工梯度下降的训练数据。

不应该做的。不要等待”自动优化 Agent 治理的算法”出现。它不会出现。不是因为 2026 年的技术不够成熟，而是因为治理质量这个优化目标本身在语义上不封闭。你无法定义一个可自动计算的损失函数来度量”这个治理决策是否避免了未来不可预见的偏离”。

完整论证见第12章。

2.8 决策七：知道什么不可能

为什么。一个可以自主完成价值闭环的 Agent 系统在结构上不成立。所谓”价值闭环”指：Agent 独立完成从执行到价值判断的全过程，无需人类判断力作为验证链终端。这不是技术成熟度的问题——而是这一闭环所需的前提条件（信息完备、价值确定、验证可自动化）在结构上与 Agent 运行的真实条件互相排斥。

必须做的。承认你的 Agent 系统的设计目标是保下限，不是撑上限。保下限意味着：确保系统在信息不完备时不崩溃，在价值不确定时不擅自裁决，在执行边界漂移时能停下来。撑上限——在给定条件下的最优决策——是操作者（人类）的职责，不是系统的职责。

应该做的。在设计每一个治理机制时，问自己两个问题：(1) 这个机制阻止的是哪种”系统以自洽但偏离的方式完成任务”的失败模式？(2) 如果没有人监控这个机制，它会在什么条件下被绕过？

不应该做的。不要伪装 Agent 能替人类做价值决策。当你的系统说”基于分析，建议你采纳方案 A”时，它在给建议。这是合法操作。当你的系统说”方案 A 已执行”而没有经过人类审批时，它在做价值决策。这是不可接受的代理越权。

完整论证见第14章 (§14.2节)。

2.9 决策框架总结

表2汇总了全部八个判断点，包括前置筛选条件（决策零）和七个核心决策节点（决策一至七）。

Table 2: 做好 Agent 系统的决策框架

决策节点	必须做	应该做	不应该做的
零：这是 Agent 吗？	确认系统执行不可逆外部动作且后续步骤依赖实际执行结果	先花半小时确认场景是否真的需要 Agent 再动工	把确定性的死代码写成 Agent
一：状态治理	持久化四字段结构化状态快照（做到哪了、为什么、未闭合项、接续入口）	状态快照用人类可读格式，在检查点后自动写入	用”记忆”替代”状态”；把流式输出流畅误认为系统连续
二：权力治理	三层权限管线至少前两层；不可逆操作前强制审批闸门	价值不确定时给出 2-4 个结构化选项，让人类做选择	用”模型看不到某工具”替代”系统不执行某工具”；用模糊安全规范替代操作前闸门
三：检查点与验证	每个子任务有可操作闭合标准；闭合判定由独立验证机制承担	检查点粒度设计为单次人类注意力跨度可覆盖	依赖 Agent 说”我完成了”
四：中断边界	在关键节点显式制造中断面，预设人类干预插入点	中断面位置按”执行在什么情况下必须停下来”定义	假设”需要时再打断”；把”永不中断”作为架构目标
五：能力与治理分离	约束治理层的接收与验证，不约束子 Agent 的推理深度和工具	在治理层预设明确的可操作验证规则	用限制子 Agent 的工具和深度来应对信任不足
六：反馈回路	接受手工梯度下降为唯一迭代范式；设计”观察 → 诊断 → 修改 → 验证”回路	为治理系统设计可观测性：记录审批上下文、偏离轨迹、人工介入后变化	等待”自动优化 Agent 治理的算法”出现
七：不可逾越边界	承认人类判断力是验证链终端——保下限不撑上限	设计每个治理机制时问自己：它阻止什么失败模式？没有监控时会怎样？	伪装 Agent 能替人类做价值决策

2.10 框架的边界：诚实性声明

任何框架都有自己的隐含前提和不适用范围。长程实践理论是归纳框架，而非公理系统——它的结论从对七个开源编码 Agent 系统的经验观察中归纳而来，不可要求从一

组公理中演绎推导。以下是你在采纳本章的七个决策节点之前应了解的三项采纳前须知。

1. **经验基础。**七个决策节点所依据的全部经验证据来自对七个开源编码 Agent 系统 ($n = 7$) 的源码级分析，不是通用实证基础。如果你在非编码领域部署 Agent，决策节点的适用性需要独立的领域验证。
2. **适用范围。**决策二（权力治理）需要人类审批闸门，决策四（中断边界）需要人类可介入，决策六（反馈回路）需要人类观察和诊断。这些都隐含假设部署团队有足够的组织规模承担治理成本。如果你是一个人在做一个个人项目，完整的七个节点可能过度设计了。
3. **必要但不充分。**七个决策节点是做好 Agent 系统的必要条件，不是充分条件。满足全部七个节点不能让你的系统“成功”，它只能让系统不至于因缺乏治理而在结构上断裂。治理是保下限的工程，不是撑上限的魔法。

框架的完整边界讨论（包括评测边界、认知边界与工程边界的系统展开，以及四个开放问题）见第14章。

3 狭义 Agent 的定义与边界

本章定位。本文的核心问题是“怎么做好 Agent”，而回答这个问题的前提是回答“Agent 是什么”。在逐项考虑状态治理、权力治理和中断边界之前，需要一个统一的概念尺度来讨论“Agent 系统”。本章回答一个先于所有经验讨论的问题：当我们说“Agent 系统”时，我们到底在讨论什么？这需要明确的定义边界。没有统一的尺度，跨系统的比较无从进行；没有针对性的场景划分，治理机制的设计无法落地；综述论文也不可避免地陷入攻击他人定义的困境。

收窄的动作本身是本论文的核心论证贡献之一。

3.1 收窄的三步推导

第一步：承认广义智能体是“感知-行动闭环”

从 Observer-Agent 理论出发^[4]，广义智能体（Agent）的共同基底是“感知-行动闭环”（Perception-Action Loop）：一个系统感知其环境，基于感知做出判断，并通过行动改变环境状态，进而感知新的状态，如此循环。这个定义在抽象层面是正确的——它覆盖了从恒温器到自动驾驶系统的全部谱系。恒温器感知温度，判断是否偏离设定值，启动或停止加热——它也是感知-行动闭环。

但正因为这个基底太宽，它无法帮助我们区分两类在信息结构上截然不同的系统。这是广义定义的张力：它经得起时间考验（不绑定特定技术栈），却无法提供分析性的切分力。

第二步：以“执行现实价值”为收窄判据

关键的切分点不是“有没有行动”，而是行动是否跨越了从虚拟到现实的边界。

一个纯粹对话型 Agent 和一个能修改文件系统、调用 API、提交代码的 Agent，表面上只是“多了几个工具”，但它们在信息结构上是两类不同的系统。后者的行动会产生不可逆的外部状态变更（irreversible external state change），这些变更不仅是行动的副产物，而且是后续步骤的关键输入 K_{i+1} 。步骤 $i+1$ 的执行或验证必须以步骤 i 造成的外部变更为前提。

设一个执行者 $\mathcal{A}$ 在第 i 步执行了工具调用 τ_i ，该调用修改了外部世界状态 $W_i \rightarrow W_{i+1}$ 。如果从 c_i 到 c_{i+1} 的推进必须以 W_{i+1} 中的某部分信息为关键输入，即 $K_{i+1} \subseteq W_{i+1}$ ，那么 K_{i+1} 在步骤 i 执行前不可被静态折叠（ $K_{i+1} \notin \text{Fold}(I_i)$ ）——因为它尚未生成。这正是长程实践区间（定义 9.3）的形式判据在 Agent 行动语境下的直接实例化。

纯粹对话 Agent 不产生这种不可预计算的关键输入：它的每一步“输出”只是对话历史的追加，不改变外部世界状态的结构。下一轮对话虽然以上一轮输出为输入，但该输出在上一轮发起前已经被模型生成——它从未进入过“必须先行动才知道会发生什么”的信息结构。因此，对话 Agent 属于单次闭合的扩展执行，而非本文关心的长程系统。

由此，本文的研究对象收窄为：**能够执行不可逆外部动作、且其实践路径中包含不可预计算顺序依赖链的自主系统。**

第三步：Coding Agent 是通用 Agent 核的最小完备投影

收窄到“能执行不可逆外部动作的系统”之后，下一个问题是：我们以什么作为具体的研究载体？本文选择以 Coding Agent（编码智能体）为主要分析对象。这个选择不是出于题材偏好或视野局限，而是因为 Coding Agent 在结构上是通用 Agent 核的最小完备投影（minimal complete projection）。这一判断依赖长程实践理论中的分形同构（fractal isomorphism）论证。

构筑（设计架构、规划系统）与执行（编写代码、调试错误）在表面上功能定位不同，但它们共享同一约束方程：

$$\text{构筑： } K_{i+1}^{(\text{构})} \notin \text{Fold}(I_i^{(\text{构})})$$

$$\text{执行： } K_{i+1}^{(\text{执})} \notin \text{Fold}(I_i^{(\text{执})})$$

两边承载的内容完全不同——一边是需求文档和架构图，另一边是报错信息和代码行号——但 $K_{i+1} \notin \text{Fold}(I_i)$ 这个约束关系在两类活动上逐对重合。内容相异，约束方程同构。这是同一个信息拓扑在构筑与执行两个维度上被实例化了两次。

Coding Agent 是这个信息拓扑的最干净投影，原因有三：

1. 代码是可执行、可验证、可回滚的——最干净的行动-反馈闭环。代码的语法正确性可以被编译器或解释器自动判定，逻辑正确性可以被测试套件自动判定，状态变更可

以被版本控制系统追溯和回退。与其他行动域（物理操作、金融交易、医疗决策）相比，代码域的反馈信号是最密集、最可自动化、最不受人类主观判断干扰的。

2. **代码是虚拟与现实的中转层（interface layer）。**代码运行在可控的符号空间内，但其执行结果可以驱动真实世界效果——从部署服务到控制硬件。它同时具备”符号世界的可控性”和”物理世界的因果效力”。
3. **代码的检查点天然具有可独立检验性（T2）。**长程实践理论要求每个检查点的产出能够在当前上下文中被独立检验（运载前提 T2，§9.2）。代码检查点——一个函数、一个模块、一个通过测试的提交——恰好满足这一要求：测试套件、类型系统和 lint 工具提供了不依赖执行者自述的独立验证机制。

因此，Coding Agent 不是 Agent 系统的一个”特例”，而是具备完整行动结构的最小实例。我们选择它不是因为视野局限，而是因为它在结构上最干净地暴露了所有 Agent 系统共有的治理问题。结构变量不依赖题材——从 Coding Agent 中提取的治理约束，对其他行动域（科学计算、工程设计、运维自动化）同样有效，只要这些域同样存在 $K_{i+1} \notin \text{Fold}(I_i)$ 的信息结构。

3.2 三层同心模型：原核、真核与皮层

定义了狭义 Agent 之后，下一个问题是：这些系统内部是如何组织的？本节引入一个三层同心模型（Three-Layer Concentric Model），用以解剖 Agent 系统的运行时架构。这个模型不是按照”应该有哪些功能模块”的预设来划分，而是从工程实践中观察到的自然分层——每一层对应一类不同性质的结构问题。

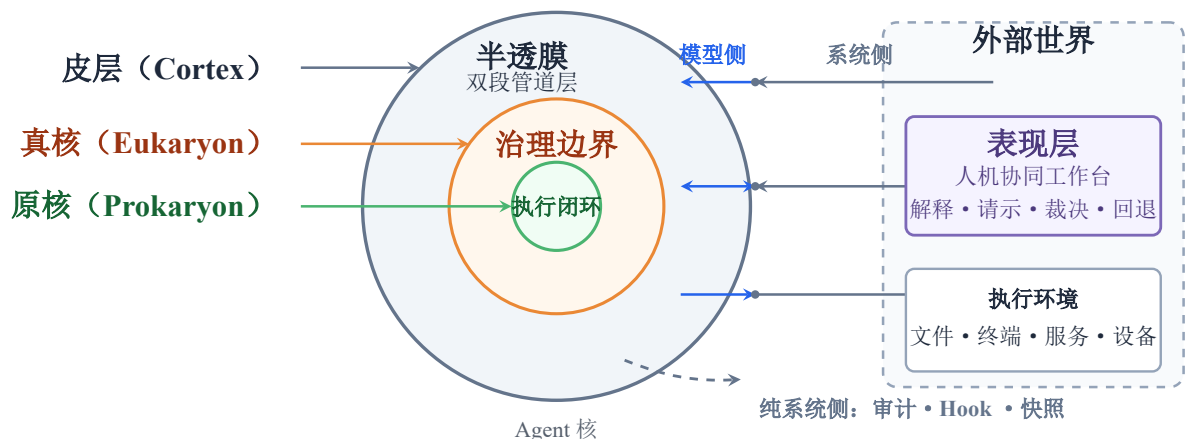


Figure 1: Prokaryon–Eukaryon–Cortex 三层 Agent 核与其核外世界

3.2.1 原核（Prokaryon）：已高度收敛的调用协议

（原核的完整论述——包括 while 循环伪代码、三步符号压缩和收敛论证——见第5章。）

3.2.2 真核 (Eukaryon): 运行时治理基础设施

真核是 Agent 的”边界层”，管两件事：跨时间维持内部连续性（状态治理）；定义与外部世界的行动边界（权力治理）。核心原则：将非确定性的推理流锚定到离散、可恢复、可验证的状态点上。

3.2.3 皮层 (Cortex): 渐进式披露的能力供应链

皮层是 Agent 与外部世界的接口层：任何能通过语言序列被驱动的外部系统，原理上都可以接入。核心原则：渐进式披露（Progressive Disclosure）——不在不需要的时候让模型看到不该看到的东西。

3.2.4 三层的递进关系

三层打包在一起，才构成一个完整的 Agent 核：只有原核是模型调用器，加上皮层是能调工具的 chatbot，只有加上真核——即状态治理与权力治理——才构成能负责任地执行不可逆动作的自主系统。

3.3 治理的严格定义

在展开真核的内部机制之前，必须先给出”治理”（Governance）在本文语境中的严格定义。这个定义不是对”治理”一词的辞典释义，而是一个结构性的操作定义。它必须足够精确，以至于我们可以用它来判定一个 Agent 系统是否具备治理能力，以及在哪个维度上薄弱。

定义 3.1 (Agent 治理). 在 Agent 系统中，**治理 (Governance)** 是将原核的非确定性推理输出流**锚定 (anchor)** 到离散、可恢复、可验证、可问责的状态点上的机制集合。

锚定通过两个维度实现：

- **状态治理 (State Governance):** 跨时间维持内部连续性——确保系统在中断、上下文清空或执行者切换后不”失忆”。其操作性定义是：在任意检查点 c_i ，系统能从持久化的快照中恢复以下四项信息：(1) 当前任务做到哪了；(2) 关键决策及其理由；(3) 哪些已闭合、哪些未闭合；(4) 下一步从哪接续。
- **权力治理 (Power Governance):** 定义系统与外部世界的行动边界——确保系统在价值不确定、路径分叉或自身判断力不足时不”越界”。其操作性定义是：对于任意工具调用 τ ，存在一个可审计的决策管道，使得 τ 在以下三个条件全部满足时才被释放到外部世界：(1) τ 在当前权限模式下被允许；(2) 若 τ 为不可逆操作，已通过预设的审批闸门；(3) τ 的执行边界在隔离或回滚机制的覆盖范围内。

定义 3.1 的核心信息结构是：治理不是”约束”——至少在消极意义上不是。治理是将非确定性的、连续流式的推理输出，转化为离散的、可定位的、可外判的状态点。没

有这个转化，Agent 系统的每一次执行都是一条不可复现、不可中断、不可交接的连续轨迹——它在工程上等价于一个黑箱。

3.4 运载前提预埋：联合认知系统的结构上限

在结束本章之前，需要预埋一个对后续全部论证至关重要的判断。这个判断在 §9.2 中将获得完整的形式化展开，但它的核心结论直接约束着本章建立的定义框架：

Agent 系统不是孤立运行的。它与人类操作者共同构成一个**联合认知系统**（Joint Cognitive System）。这个系统的长程存活能力不取决于任何单一组件的最优性能，而受制于整个联合系统的最弱一环。这便是运载前提（Transport Prerequisite）的核心约束。长程实践理论将其展开为三层九条形式判据（T1–T3 / E1–E3 / H1–H3，详见 §9.2），其中两条直接决定了 Agent 系统是否具备”入场资格”：

引擎物理极限（E1 不可行性定理）。设 LLM 的单次窗口有效执行跨度为 S ，任务的最小可独立检验步长为 $L_{\min}$ 。长程资格的必要条件是：

$$S \geq L_{\min}$$

若 $S < L_{\min}$ ，则不存在任何检查点序列使得每个检查点同时满足”足够大以被独立检验”和”足够小以被 LLM 稳定完成”——系统在结构上不具备长程资格。这不是一个可以通过工程补偿绕过的约束。

驾驶员认知极限（H1 非对称验证带宽）。设人在检查点处”读取并理解状态”的认知成本为 C_{verify} ，当前可用注意力预算为 $B(t)$ 。有效验证的必要条件是：

$$C_{\text{verify}} < B(t)$$

当 $C_{\text{verify}} \geq B(t)$ 时，人产生橡胶图章效应——形式上走过批准流程，实质上跳过判断。此外，操作者还需满足外部现实锚定力（H2）和决策制动阈值（H3）——否则即使引擎满足物理极限，联合系统仍会在认知层面失能。

这两条约束的完整形式化展开，以及它们共同推出的核心推论——验证链终端的不可自动化——将在讨论章中给出完整论证（§14.2）。

本节的定义框架——从三段推导到三层同心模型到治理的严格定义——建立的全部概念，都将在后续各章中接受运载前提的检验。原核的收敛是引擎物理极限决定的：API 的离散回合结构只允许这一种呼吸方式（第5章）。真核的必要性来自价值闭合——权力治理由此成为结构必需品（第6章）。皮层的渐进式披露是注意力预算（E1+H1）在结构上的要求（第8章）。治理成本的正当性来自一个更基本的约束：不管住的 Agent 在经济上不可持续（§7.2）。

4 演化论视角：七系统的分歧

本章定位。七个系统的源码演化史是决策一（状态治理）和决策二（权力治理）的演化论解释。在深入原核/真核/皮层的三层模型之前，需要先看到七个真实系统在不同环境压力下产生的收敛与分化模式——它们不是理论推演的产物，而是工程史上留下的结构化石。本章的核心论点是：治理不是被设计出来的，是被演化压力逼出来的。

§3 建立了狭义 Agent 的定义框架和三层同心模型。这个框架的价值不在于概念上的整洁，而在于它能够解释一个令工程实践者困惑的事实：为什么七个独立开发的 Agent 系统——它们共享同一个核循环骨架，共享同一套工具协议语言——在架构上却走向了截然不同的方向？是什么驱动了这些分歧？

样本选择限定声明。本文讨论截至 2026 年 7 月已实际出现的、以 LLM 为主要非确定性推理节点、能够调用工具并面向外部实践产物的 Agent 系统。七系统（Codex CLI、Claude Code、OpenCode、Hermes、Cline、Cherry Studio、trae-agent）的选择标准是：拥有真实用户群体、源码可访问、覆盖不同初始场景（终端/IDE/独立服务/桌面/研究）。该边界用于选择比较样本，不是关于广义 Agent 的充分或必要定义。

用源码刺探的归纳来表述：没有一支团队在立项时先写了《Agent 设计哲学宣言》，再据此推导出权限管线和沙箱策略。真实的过程是达尔文式的物种辐射——不同初始场景与阻力迫使团队不断解决具体故障、成本与用户摩擦，被迫长出特定的状态、权限、恢复、工具、宿主与交互结构，长期沉淀后，架构反过来呈现为一种事实上的系统哲学。

权限管线不是画出来的，是用户抱怨了太多次“别让我每次都点确认”之后迭代出的；状态持久化不是白板推导的，是“系统崩了，用户丢了三小时的工作”之后从内存升级到 SQLite 的；能力扩展机制不是架构师预见的，是发现“每加一个工具就增加核的 schema 膨胀”之后被迫发明的。不同的环境土壤让这些系统进化成了在架构层几乎无法互认的物种：一个在 \$5 VPS 上通过 `check_fn` 防止 Docker daemon 抖动导致整组工具消失的系统（Hermes），和一个因为 Electron Renderer 会在流式输出中途崩溃而设计了 `resume token` 和状态机的系统（Cherry Studio），在状态治理层需要解决的是完全不同的工程问题。

表 3 汇总了七个系统的出生场景、迭代中遭遇的核心压力、以及为解决这些压力而演化出的关键架构结构。

Table 3: 七系统的真实出生场景与迭代中演化出的核心结构

系统 (出生时间)	真实出生场景	迭代中演化出的核心结构
Cline (2024.06)	Anthropic Hackathon 周末原型：让 VS Code 用户使用 Claude 3.5 Sonnet 的 agentic 能力	RuntimeHost 宿主解耦、四层 SDK (shared/llms/agents/core)、Hub 守护进程、Cron Reconciler 真相源
Cherry Studio (2024.H2)	多模型聚合桌面聊天客户端 (Electron)，统一访问各 LLM 提供商	Agent 作为后来功能加入：Main 控制面 + Renderer 订阅窗口、会话状态机与恢复令牌、多源 Hook 组合
Claude Code (2025.02)	Anthropic 首个终端 Agent，为 Claude 的 agentic 编码能力提供第一方载体	单核异步查询管线、工具三元组模块化、多层上下文降级
OpenCode (2025.03)	一人用 Go 写的终端 Agent；后由 SST/Anomaly 团队接手并全栈重写	独立 Bun 引擎 + client-server 架构、System Context Epoch、不透明工具
Codex CLI (2025.04)	OpenAI 对 Claude Code 的开源竞争回应 (TypeScript/Node.js)；Rust 重写始于 2025.06	OS 级沙箱 (迭代中加入)、分级审批、JSONL+SQLite 可回放持久化
trae-agent (2025.06)	ByteDance TRAE IDE (闭源 VS Code fork) 的 Agent 核心研究蒸馏	Lakeview 元认知层、全量 Trajectory JSON 审计追踪
Hermes (2025.07)	Nous Research 内部项目：模型训练轨迹生成器 + Agent 平台双用途	自注册工具发现、check_fn 瞬态故障容忍、Skills-Memory-Curator 知识闭环

代表源码锚点：Cline-@cline/core；Cherry Studio-AgentSessionRuntimeService；Claude Code-QueryEngine.ts；OpenCode-system-context/；Codex CLI-RolloutRecorder；trae-agent-agent/base_agent.py；Hermes-tools/registry.py。

注意：trae-agent (bytedance/trae-agent, MIT 许可, 289 commits) 是 TRAE IDE 闭源产品的开源研究核。TRAE IDE 本身是 VS Code 的闭源 fork (2025.01 发布)，其 Agent 引擎嵌入 Electron 进程内——本文第 9 章中讨论的“六条平台约束”来源于 TRAE IDE 的闭源架构，与开源 trae-agent 核具有不同的约束条件。

4.1 各系统的初始条件与演化路径

上述演化框架需要一个实证基础：这七个系统到底是从什么起点出发、在什么驱动下长成了现在的形态？图 2 提供了一个全景视角——七个系统在完全不同的生态位上独立起源，在相同的原核循环上集体收敛。每个系统的具体源码锚点和实现细节收录于附录 A，本节聚焦于演化动力本身——谁先出现、谁是对谁的回应、谁在什么时候分化出了哪条路径。

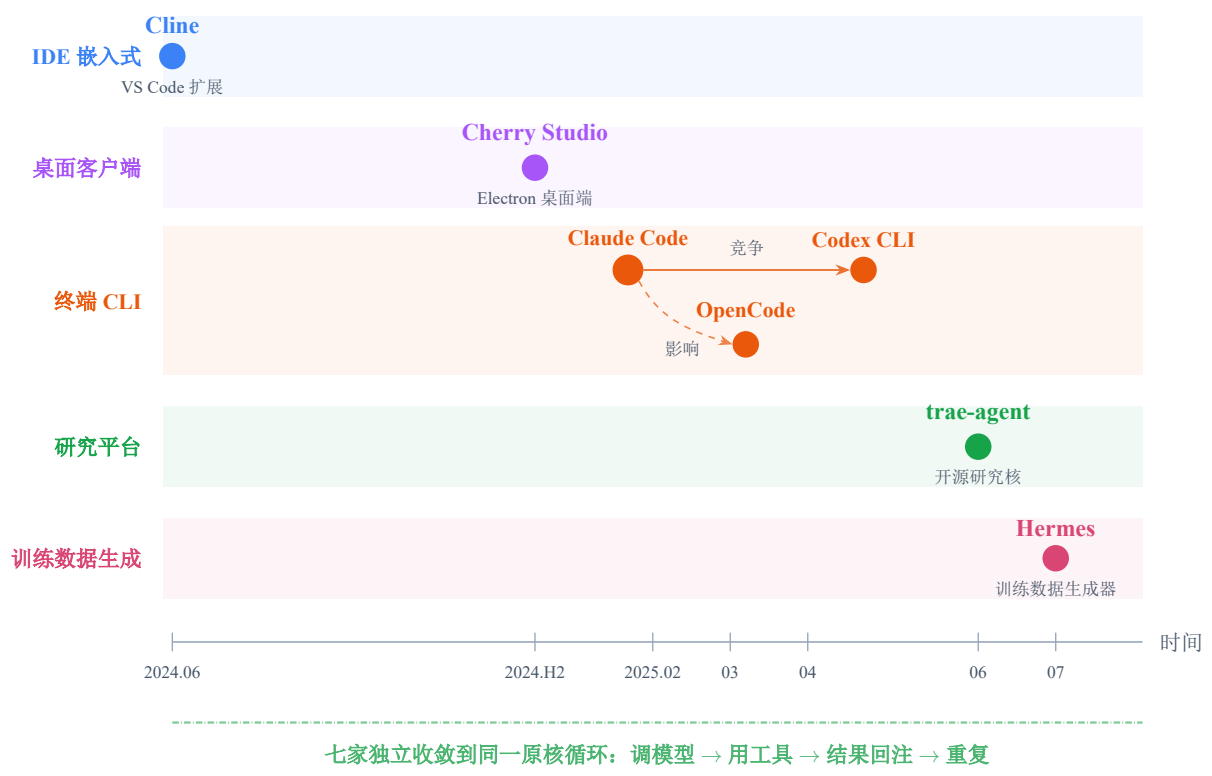


Figure 2: 七系统演化全景：不同生态位上的独立起源，同一原核循环上的集体收敛。横轴为出生时间（2024.06–2025.07），纵轴为系统最初所处的生态位。终端 CLI 带内实线表示 Claude Code 与 Codex CLI 的竞争关系，虚线表示 Claude Code 对 OpenCode 的品类影响。

Cline（2024.06）：一个周末的 Hackathon 原型

2024 年 6 月，Claude 3.5 Sonnet 发布后约十天。Saoud Rizwan 在 Anthropic 的 Hackathon 上读到 model card 中描述的 agentic 编码能力——跨文件推理、多步骤任务、大上下文窗口——但没有任何工具让 VS Code 用户以这种方式使用 Claude。他花一个周末写出了原型，叫”Claude Dev”，发布到 VS Code Marketplace。

这个原型没有固执的设计约束。一个有意思的结果是：早期用户自发形成了”先让 agent 出计划、审阅、再执行”的两阶段习惯——后来被固化为 Cline 标志性的 Plan/Act 模式。它证明了有价值的架构特征可以来源于使用模式，而非初始设计。

真正的架构爆炸发生在 2025 年——\$32M 融资、团队从 1 人扩到 20+ 人。被逼出的

核心结构包括：将状态持久化与 Agent 循环完全解耦的四层 SDK 分离；让同一套 Agent 逻辑不加修改地运行于 VS Code 扩展、CLI 和 JetBrains 插件的 RuntimeHost 抽象；允许多客户端共享同一会话、断开后继续执行、重连后恢复的 Hub 守护进程；以及一个彻底消除文件监听漏报/误报的 Cron Reconciler 模式。Cline 的轨迹是七家中最清晰的”从单体到平台”案例——出生时是 1 人周末原型，一年后是完整的 Agent 运行时平台。

Cline 从单体到平台的演化揭示了一个模式：多宿主进程间的连续会话不是并发优化问题，而是状态治理的核心需求。

Cherry Studio (2024.H2)：一个多模型聊天客户端

2024 年下半年，一位前字节跳动产品经理的个人项目。一个 Electron 桌面客户端，核心价值主张很简单：”不需要在多个 AI 网站之间切换”。附带 300+ 预置 AI 助手、知识库 RAG、Mermaid 图表渲染。

关键事实：Agent 不是出生功能。Cherry Studio 在 2025 年初公司化后，Agent 模式才在 v1.6.x/v1.7.x 系列（约 2025 年末）作为新增功能加入。这意味着它的 Agent 架构不是在一个空白画布上设计的——是被塞进一个已经存在的大型 Electron 聊天客户端里的。

这造就了它最独特的一组设计。Main 进程是 LLM 调用和文件系统的唯一入口，Renderer 进程只负责 UI。Agent 引擎别无选择，只能在 Main 进程中运行，用户看到的所有输出通过 IPC 桥中转。这反过来逼出了三项关键能力：关闭窗口不中断 Agent 执行，重开时重放缓冲事件；崩溃恢复时自动标记遗留待处理消息为错误；以及一个来自三种贡献者的 Hook 需要一个确定性组合策略——按序执行、接收前次返回值、任一要求”重试”即整体重试。

Cherry Studio 的演化揭示了一个重要模式：它的独特结构不是从 Agent 需求推导出来的，而是被 Electron 进程架构强行施加的约束逼出来的。如果 Agent 是出生功能，这些设计可能完全不同。

Cherry Studio 的案例暴露了平台选择的结构性后果——Electron 的多进程架构将 Agent 引擎锁定在 Main 进程，关闭窗口不中断执行、崩溃状态恢复、多来源 Hook 确定性组合这三项能力，都是在”Agent 不是出生功能”这一事实约束下被迫演化出的。

Claude Code (2025.02)：定义”终端 AI 编码 Agent”品类的催化剂

2025 年 2 月 24 日，Anthropic 发布 Claude Code Research Preview——第一个来自主流 AI 实验室的终端原生编码 Agent。初版是轻量 CLI，核心循环几千行。但它的意义远不止代码量：它证明了一个 agent 可以不依赖 IDE、直接在终端中通过自然语言驱动文件系统和 Shell。25,000+ GitHub stars 在一周内。六周后，两个直接回应出现了——OpenCode 和 Codex CLI。

迭代把它推到了另一个量级。核心查询引擎膨胀到约 46,000 行——六类逻辑（LLM 流式调用、工具调用检测、权限拦截、结果注入、token 预算检查、上下文压缩触发）在

多次大版本迭代中逐步汇入同一个文件。被长会话压力逼出的四层上下文降级——工具结果替换 → 历史截断 → 压缩 → 溢出重试——是渐进的降级顺序，只有在前一层真正失效时才升级。

Claude Code 对本文的核心论证有特殊位置：它是七家中唯一一个出生时没有任何“被环境逼出的畸形结构”的系统。但即使是最干净的出生，迭代仍然长出了 46,000 行的上帝类和四层降级路径。环境约束不是唯一的演化驱动力——内部复杂度积累本身就是。

即使是最干净的出生环境，内部复杂度积累本身也是一种不亚于环境约束的演化驱动力——46,000 行的上帝类和四层降级路径不是任何外部压力的产物，而是系统在无约束条件下自由膨胀的必然结果。

OpenCode (2025.03)：一个人的 Go 终端项目

2025 年 3 月 21 日，Kujtim Hoxha 的个人项目。Go 语言 + Bubble Tea TUI。两天内四个 commit，一周搭出基本可工作的 agent，第二周加 LSP 支持。

OpenCode 从来不是 VS Code 扩展。它的初始环境条件是纯终端——一个人、几天、Go 写的 TUI agent。SST/Anomaly 创始人 Dax Raad 在 4 月加入，5 月起主导 backend 从 Go 到 TypeScript+Bun 的全量重写——动机是接入 models.dev 上百个 Provider。TUI 直到 10 月才从 Go Bubble Tea 重写为自研 OpenTUI。

Backend 重写意外地产生了 client-server 架构——新 backend (TypeScript+Bun) 和老 TUI (Go) 需要通信协议。引擎和前端一旦分离，状态的一致性和可恢复性就成了硬问题。这逼出了一套独特的代数系统：每个上下文源包含稳定标识、JSON 编解码、无失败加载器和纯渲染器，四种操作覆盖从初始生成到压缩后重建的全生命周期。另一个被逼出的结构是不透明工具——工具的行为被隐藏，契约只暴露 I/O 类型。目的不是安全，而是保证进程分离后的“工具声明一致性”：前端从不过问工具实现即可渲染正确 UI。

Backend 重写意外切出的 client-server 架构将上下文持久化从“可选优化”升级为“结构必需”——代数上下文系统和不透明工具协议都是在不共享内存的进程间维持状态一致性的工程回答。

Codex CLI (2025.04)：对 Claude Code 的竞争回应

2025 年 4 月 16 日，OpenAI 发布 Codex CLI。与 o3/o4-mini 同一天。TypeScript/Node.js + React Ink TUI，Apache 2.0 开源。同时发布了 codex-1 模型——o4-mini 的代码特化版。

时间线说明一切：Claude Code 发布（2 月 24 日）到 Codex CLI 发布——间隔不到 8 周。这不是从零独立构思的时间窗口。开源（Apache 2.0）本身也是战略性差异化——Claude Code 的源码不可自由使用。

Codex CLI 最令人印象深刻的架构特征（覆盖 Linux/macOS/Windows 三种操作系统的 OS 级安全沙箱）不是出生时就有的。它是在 2025 年 6 月启动 Rust 全量重写后才建

立的竞争壁垒。除此之外，两项设计有特殊的结构价值。一是分级审批：四档审批强度，审批者可以是人类也可以是独立的自动审查 Agent，是七家中唯一在审批链中引入非人类审查者的设计。二是全量事件持久化：Codex 的状态不是”快照”，而是一条可回放、可重建的完整事件流。这与 Claude Code 的多层压缩形成明确的哲学对照。Codex 追求完整可重建（来自安全审计需求），Claude Code 追求压缩后足以推理（来自上下文窗口预算）。

全量事件持久化（Codex CLI）与多层上下文压缩（Claude Code）的哲学对照，暴露了安全审计与上下文预算两种演化压力的不可调和——Codex 追求完整可重建，Claude 追求压缩后足以推理。

trae-agent (2025.06): 从闭源产品中蒸馏的研究核

一个容易被误读的系统。trae-agent 不是 TRAE IDE。TRAE IDE 是 ByteDance 2025 年 1 月发布的闭源 VS Code fork，12 个月获 600 万 + 注册用户，免费提供模型推理。trae-agent 是产品发布 5 个月后从中蒸馏出的开源 Agent 研究核（MIT 许可）。它包含核心 Agent 循环、工具系统、LLM 客户端抽象和评估体系，但完全剥离了 VS Code/Electron 集成。

这决定了它的独特价值：它是七家中唯一将”可研究性”而非”鲁棒性”作为第一性原理的系统。不需要复杂的恢复（任务短程可重复），不需要审批管线（实验室环境全程观察），不需要多 Agent 委托（消融实验需要变量控制）。

被这个定位逼出的最有意思的结构是 Lakeview 元认知层——一个独立的 LLM 调用专门分析主 Agent 的每一步轨迹，将步骤分类为预定义行为标签。本质上是一个”观察 agent 的 agent”，不是为了帮助主 Agent 做得更好，而是为了让研究者事后能理解它为什么这样做。另一个结构是全量轨迹 JSONL 审计追踪——与 Codex CLI 的 JSONL 形式相似，但目的完全不同：Codex 的 JSONL 是运行时恢复基础设施，trae-agent 的轨迹是事后分析工具。

这里有一个对后续章节重要的注记：第 10 章讨论的”六条平台约束”（禁止 subsub-agent 嵌套、agent 能力不均、元信息不可见、回滚/续接缺失、本地机性能瓶颈、工具配额硬上限）来源于 TRAE IDE 的闭源”嵌”架构。开源的 trae-agent 核不受这些约束。两者差异提醒我们：一个 Agent 系统的治理能力边界不仅取决于核心循环设计，更取决于运行时宿主架构。

trae-agent 的选择揭示了一个被其他系统忽视的维度：当”可研究性”而非”鲁棒性”成为第一性原理时，系统不需要复杂恢复和审批管线，但必须让研究者事后理解 Agent 每一步决策的因果链。

Hermes (2025.07): 模型训练数据生成器

2025 年 7 月 22 日，以开放权重模型闻名的 Nous Research 创建了 Hermes 仓库。8 个月内部开发后，2026 年 3 月首个公开 tag。它与前六个系统有一个根本差异：它出生时就是双重用途——既是 Agent，也是训练数据生成器。源码中的 RL 训练环境和并行轨迹生成器暴露了它的闭环逻辑：Agent 执行任务 → 生成 trajectory → trajectory 训练下一代模型 → 更强的模型驱动更强的 Agent。

公开后，Hermes 经历了一场社区爆炸——v0.2.0 发布两周内，63 位贡献者合入 216 个 PR。最被社区压力逼出的结构是多平台消息网关——Telegram、Discord、Slack、WhatsApp 等 20+ 平台。但 Hermes 最有价值的三个结构来自其内部开发的 8 个月。

第一，prompt 缓存不可侵犯原则。Hermes 是七家中唯一将 Anthropic prompt caching 稳定性作为显式设计约束的系统：系统提示词在一个 session 中只构建一次，工具变更通过用户消息注入。这塑造了独特的工具注册体系——每个模块加载时自行登记，发现阶段先静态扫描确认登记行为再决定导入。

第二，瞬态故障容忍 (check_fn)。多平台环境中，Docker daemon 响应延迟 2 秒不应导致整组工具静默消失。Hermes 的 check_fn 有双层保护：缓存避免频繁探测，在最近成功时间窗口内的失败翻转视为毛刺、不缓存失败结果。这不是常见容错实践，而是一个专门为“工具不应因短时抖动而消失”场景设计的防御层。

第三，Skills-Curator 知识闭环——七家中最完整的跨会话学习系统。Agent 完成任务后自动创建或更新可复用的 skill 文件，Curator 管理技能库生命周期，Memory Provider 维护跨会话事实记忆。这使 Hermes 成为七家中唯一将“积累经验”作为核心运行时行为的系统——其他系统的 Skills 和 Memory 是附加功能，Hermes 的 Skills 是 Agent 存在的目的之一。

Hermes 的双重用途——既是 Agent 又是训练数据生成器——迫使系统在 prompt 缓存稳定性、瞬态故障容忍和跨会话知识积累三个维度上同时建立防御层，这使它的治理需求天然高于任何单一用途系统。

4.2 分歧之下的收敛：原核为何不变，皮层为何不可预测

4.1 节按出生时序走完了七个系统的独立演化路径。把七条路径并排放在一起，一个事实无法回避：它们在真核和皮层走向了几乎无法互认的方向——但核心循环从未改变。七家在没有任何协调、不共享编程语言的情况下，独立收敛到了同一个核形式：**调模型 → 用工具 → 结果回注 → 重复**。Cherry Studio 的 Electron 进程架构、Codex CLI 的 OS 级沙箱、Hermes 的 check_fn——都只在这个循环外围添加了控制层，从未改变循环本身。

原因在底层 API 约束，不在设计哲学。2026 年的主流 LLM API 施加了三条构成不可逃脱势阱的刚性边界：

1. **离散回合交互**。模型 API 无状态——每次调用需完整输入上下文，返回后断开。核心

循环必然是离散的逐步推进。

2. **阻塞式工具调用协议。**模型发出工具调用，Agent 执行工具、收集结果、回注到下一轮 prompt。”调用 → 执行 → 等待结果 → 注入”的阻塞循环是 API 强制的交互协议，不是 Agent 架构的选择。
3. **有限上下文窗口。**每轮 token 预算有限，迫使长会话中做压缩和降级——但这只在外围增加管理策略，不改变循环结构本身。

七家没有选择相同的核循环——是相同的核循环选择了它们。

这个收敛故事的反面，是皮层的本质不可预测性。§3 给出了皮层的操作性定义：任何能够通过语言序列被驱动的外部系统均可接入 皮层，语言是通用总线。这意味着皮层形态不由 API 决定，而由出生场景的”语言控制需求”决定。Codex CLI 发 OS 沙箱指令，Hermes 发消息网关指令，Cherry Studio 发 IPC 桥指令——它们的皮层长得完全不同。这不是演化中的意外，而是皮层设计原则的直接后果：它定义上就是开放的。

夹在原核的锁定和皮层的开放之间的，是真核——三层中唯一同时受两个方向约束的层。向下被原核的离散回合结构约束（治理操作只能在 step 边界插入），向外被皮层的开放性约束（任何皮层扩展都可能引入新的安全边界和状态依赖）。Codex 的沙箱、Cherry 的进程分离状态机、Hermes 的 `check_fn`、Cline 的 Hub/Spoke——七家真核的分歧，本质上是对同一个问题的七种不同回答：在一个被锁定的核循环和一个不可预测的皮层之间，治理应该长什么样？

这个三层结构不仅是对演化史的如实描述——每一层都可以从第一性原理推导。下一章要做的正是这件事：从 API 的离散回合结构出发，将原核的收敛从一个经验观察提炼为一个形式命题。

4.3 演化史上的集体盲区：L4 跨执行断面的交接治理

上述三层结构也解释了为什么 L4（跨执行断面的交接治理）成为演化史上的集体盲区：原核无可优化，皮层不由治理决定，真核的优先级由出生环境决定——七个出生环境施加了 L1-L3 的生存级压力，却没有一个要求”执行完成后向下一任交出结构化的未闭合项清单和已验证结论”。L4 不是不重要——它只是尚未在任何一家的环境压力谱上达到生存级阈值。本节用四级连续性标尺量化这个差距：

- **L1（长会话内连续）：**单次会话跨越多次模型调用和工具执行，维持连续推进。典型机制：上下文压缩、token 预算管理。
- **L2（跨进程恢复）：**进程崩溃或会话终止后恢复到可继续执行的状态。典型机制：resume token、checkpoint 恢复、事件流回放。
- **L3（跨任务积累）：**前次任务的经验、技能或知识带入下一次任务。典型机制：Skills、Memory、工具库积累。

- **L4（跨执行断面交接治理）**：一次执行完成后，向下一任执行者交出结构化的、可独立验证的交接信息——已闭合项、未闭合项、关键决策理由、已验证结论、当前责任状态和接续入口。

表 4 以这四个层级为标尺，审视七个系统当前已工程化的连续性能力。

Table 4: 七系统的 L1–L4 连续性能力

系统	L1 长会话	L2 跨进程	L3 跨任务	L4 交接治理
trae-agent	max_steps	—	—	—
OpenCode	compaction	Epoch 恢复	—	—
Claude Code	多层压缩	resume	MEMORY.md	—
Codex CLI	compaction	rollback 恢复	Skills, Hooks	—
Cline	compaction	Hub restore	Rules, Skills	Cron 报告
Hermes	压缩	SessionDB	Memory, Curator	Kanban 最近

这张表揭示了演化史上最一致的发现：**L4 是全部七家最弱的一环**。L1–L3 已各有覆盖——compaction 和 resume 是标准配置，Skills/Memory 已有多家实现——但 L4 在所有七家中均未达到工程化水平。Cline 的 Cron 报告和 Hermes 的 Kanban 是最接近 L4 的设计，但它们记录的是“什么任务跑完了”或“什么任务在队列中”，而非 L4 的核心信息结构：当前断面上哪些判断是已验证的、哪些是暂定的、哪些仅基于上一执行者的局部信息、下一任必须在哪个节点重新验证。

这不怪它们——它们的出生场景从来不要求跨断面交接。但如果承认 §3.1 定义的长程实践要求 $K_{i+1} \notin \text{Fold}(I_i)$ 的不可预计算顺序依赖链，那么在一个多执行断面交织的长程任务中，L4 不是可选功能——它是防止执行者在信息不封闭的检查点上用局部记忆替代全局状态的唯一结构性防线。

这把我们引向本篇论文的核心论证缺口。七个系统走向了截然不同的架构，但在 L4 的空白上惊人地一致。这就是下一章要讨论的。

5 原核（Prokaryon）：已高度收敛的调用协议

本章定位。本章是决策零（“这是 Agent 吗？”）的完整论证。在讨论“怎么做得更好”之前，需要先认清“什么已经不能做得不同”——七个独立演化的系统在原核层收敛于几乎相同的结构，这是一个强烈的信号。做好 Agent 的功夫不在这一层：调用协议循环的设计空间已接近饱和，没有竞争空间。

如第3章所定义，原核是模型调用协议的最小闭包（Minimal Closure of Model Invocation Protocol）。其形态是一个 ± 20 行的 while 循环：

while（没超预算且没被中断）：

```

response = 模型（系统提示 + 工具声明 + 对话历史）
if response. 有工具调用：
    执行工具 → 把结果塞回对话历史
else:
    退出循环 → 返回最终回答

```

这个 ±20 行的骨架不是任何人的设计——它是独立演化的终点（详见第1章）：在当前 LLM API 调用范式的信息结构下，原核的形态已高度稳定。

用符号压到一行，这个循环就是：

调模型： $\text{model}(s_i) \rightarrow (r_i, \tau_i)$ $\longrightarrow$ 用工具： $\text{exec}(\tau_i) \rightarrow o_i$ $\longrightarrow$ 再回注： $s_{i+1} = s_i \oplus (r_i, o_i)$

三步不停循环，直到 $\tau_i = \emptyset$ （模型不再要求调用工具）或外部中断。这就是 原核的全部。

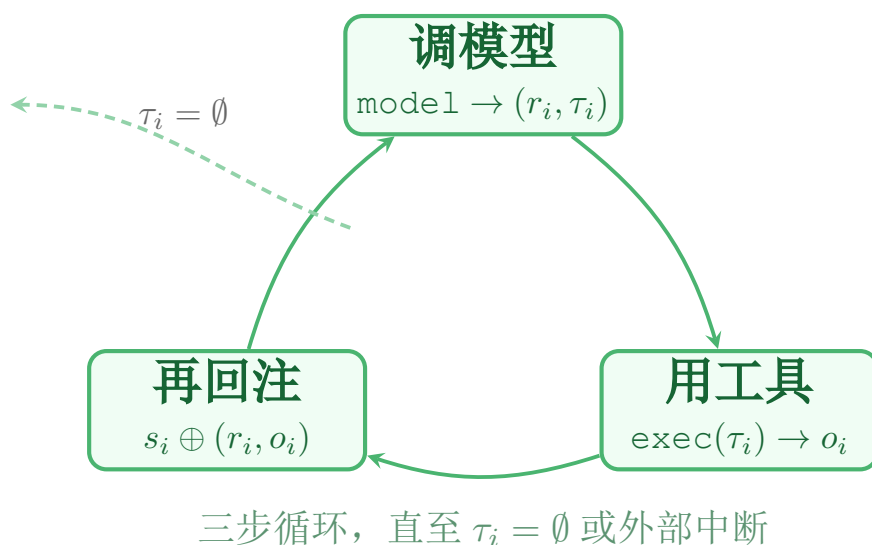


Figure 3: 原核的调模型–用工具–再回注三步循环：每步执行模型推理 → 工具调用 → 结果回写，如此往复直至终止

正因为它如此简单，原核的深化价值极低。Agent 层的几乎所有硬技术都在这个 ±20 行的循环里——没有密门，没有暗层，没有任何值得额外展开的”架构创新”。真正的战场在 真核 (§6)：当单调追加撞上有限的上下文窗口，当工具调用需要审批和隔离。此时治理才成为结构必需品。

最后，必须加上一个审慎的限定，它来自本研究的样本边界：**在本文考察的 2026 年主流 LLM Agent 实践中，原核 已高度收敛。**本文不无条件宣称所有未来 Agent 都只能是这一形式。如果未来的模型调用协议发生了结构性变化——例如连续的而非离散回合

的交互范式、或工具不再遵循”调用-等待-结果”的阻塞式协议——原核的形态可能随之改变。但对于本文讨论的、以 2026 年主流 LLM API 为底层的 Agent 系统而言，原核的收敛是一个已观察到的、可独立验证的工程事实，不是一个哲学断言。

6 真核（Eukaryon）：真正的战场

本章定位。本章是决策一（状态治理）和决策二（权力治理）的完整展开。在确定了”建立状态治理”和”建立权力治理”是必须做的事情之后，需要看到它们在实际系统中是如何实现的、在哪里失效、以及为什么三层权限管线是演化收敛的结果而非设计偏好。真核层是 Agent 系统治理基础设施的工程载体。

如第5章所示，原核已高度收敛，真正的差异在治理层。本章将这一治理层的工程载体——**真核（Eukaryon）**——展开为完整的理论分析。

在正式展开之前，需要先明确真核在整个三层模型中的精确位置。§3 引入的三层同心模型中，真核被定义为”运行时治理基础设施”，回答两个问题：”系统如何记住”与”系统被允许做什么”。Ch2 的定义侧重功能描述；本章的任务是将这两个问题展开为完整的理论分析，展示为什么它们不是同一个问题，以及为什么混淆它们会带来南辕北辙的修复路径。

6.1 二分法的核心洞察：状态治理与权力治理

在工程实践中，Agent 系统有一个常见的失败模式：任务跑到一半跑偏了，或者做了不该做的事。面对这种失败，工程师的本能反应往往是”加更多规则”或”加更多约束”。但这个本能遮掩了一个关键的区分：跑偏有两种完全不同的原因——系统不记得自己做到哪了（状态断裂），或者系统跨过了不该跨过的行动边界（权力越界）。两类原因需要完全不同的修复路径，而混淆它们会让治理投入打在水上。

这正是真核内部二分法的核心洞察：

定义 6.1 (状态治理（State Governance）). 状态治理回答系统如何跨时间维持内部连续性。其对象是：上下文管理、持久化策略、压缩与恢复、跨断面交接。缺失的后果是系统”失忆”——不记得做到哪了、为什么做到这、下一步从哪接。工程载体包括 SQLite 会话数据库、事件日志、Context Epoch、resume token 和锚点文档。

定义 6.2 (权力治理（Power Governance）). 权力治理回答系统与外部世界的行动边界在哪。其对象是：权限管线、审批机制、沙箱隔离、价值闭合、机向人请示、回滚策略。缺失的后果是系统”越界”——做了不该做的操作，或在该请示人类的时候没有请示。工程载体包括权限模式、check_fn 函数、审批 workflows、OS 级沙箱和 ec7_action_gate。

这两个定义不是概念上的分类练习，而是具有直接的**诊断力和预测力**。面对一个 Agent 长程任务的失败，如果诊断出系统在中断后无法正确续接——丢失了”做到哪了”

为什么做了当时的选择”” 哪些分支已被否定”——那么问题是状态治理缺陷，修复方向是加固持久化和交接机制。如果诊断出系统在未经授权的情况下执行了不可逆操作——删除了不该删除的文件、越过了沙箱边界、在需要人类价值判断的节点静默地自行决定了方向——那么问题是权力治理缺陷，修复方向是加固权限管线和请示机制。对前者加权限规则、对后者加持久化，都是在错误的位置投入治理成本，不会产生预期效果。

命题 6.1 (状态-权力不可互代性). 状态治理机制不能替代权力治理机制，反之亦然。状态断裂不能用权限规则修复（系统仍然不记得自己做到哪了），权力越界不能用持久化修复（系统仍然会在不该行动时行动）。两类治理必须同时存在，且必须各自独立运作。

状态治理与权力治理之间存在一个关键的交叉地带：”状态的写入权”——谁可以修改状态快照、谁可以标记检查点为”已闭合”、谁可以删除或覆盖历史状态记录——本身就是最高级别的权力。篡改或选择性提交状态，本质上是执行者行使未被授权的认知权力，其后果在权力治理侧表现为假闭合传播，在状态治理侧表现为状态污染。这一交叉地带不打破二分法——状态的写入权仍是权力治理的管辖对象，状态的语义标注仍是状态治理的管辖对象——但它提醒我们：状态治理的工程实现必须接受权力治理的约束（写入权限、修改审计、覆盖追溯），否则状态本身就会成为权力越界的载体。§7.1 中讨论的”状态主权”原则——子 Agent 不能自行将结论写入主状态——正是这一交叉地带在多 Agent 场景下的具体实例。

6.2 状态治理

状态治理是真核中回答”系统如何跨时间维持内部连续性”的那一半。它不是”记忆增强”的同义词——这一区分将在本节核心论证中展开——而是确定系统在任意时刻处于哪个可继续、可恢复、可验证的位置的机制集合。

6.2.1 持久化策略频谱：七个系统在何处画线

七个系统在原核层共享同一骨架，但在”把什么东西写进持久化存储”这个决策上，它们画出了从激进到保守的一条完整频谱。表 5 按持久化程度的递增顺序排列。

这条频谱的两个端点代表了两种对”状态”本质的不同理解。L0-L1 端的系统将状态视为”当前执行过程的副产品”——会话结束了，状态也就没有保留的必要。L2-L3 端的系统将状态视为”一等公民”——状态的生命周期独立于单次会话的生命周期。

OpenCode 是 L3 端的最激进代表，值得单独展开。它的状态架构不是”把对话存进 SQLite”，而是一套完整的事件溯源（Event Sourcing）系统：**不可变 event 表**记录所有已发生的事实（用户消息、模型调用、工具执行），一旦写入就不可修改；**session message** 是这些 event 的当前投影——不是独立存储，而是从 event 表实时计算出来的视图；**Context Epoch** 作为模型上下文的基线标记，使得系统可以在任意 Epoch 点重建当时模型看到的完整上下文；**admit/promote 双阶段输入管理**将”消息到达系统”和”消息对模型可见”分离为两个独立步骤，使得系统可以在消息进入模型视野之前对其进行过滤、延迟或重排

Table 5: 七系统的持久化策略频谱

系统	持久化层	核心特征	级别
trae-agent	内存为主	无独立持久化层；状态在进程内存中，进程结束即蒸发	L0
Cline	文件系统	Hub/Spoke 架构通过文件系统传递状态快照；无专用数据库	L1
Cherry Studio	进程内状态机	AgentSessionRuntimeService 维护会话状态；resume token 可恢复但非完整持久化	L1
Claude Code	四层压缩不持久	对话历史在内存中；compaction 裁旧保新但不持久化跨进程状态；MEMORY.md 为可选用户文件	L1
Codex CLI	SQLite + JSONL	StateDB (SQLite) 结构化存储；RolloutRecorder (JSONL) 全量事件流；ScanAndRepair 从 JSONL 重建损毁索引	L2
Hermes	SQLite SessionDB	SessionDB 持久化会话元数据与消息；Memory Provider 独立管理跨会话记忆	L2
OpenCode	事件溯源 + 投影	不可变 event 表 + session message 投影 + Context Epoch 基线；admit/promote 分离输入可见性	L3

序。这整套设计的工程含义是：**Agent 不等于当前上下文中正在运行的一次 LLM 循环；它是拥有外部化、可提交、可恢复、可治理状态的持续系统。**

6.2.2 上下文治理的多层降级路径

状态治理不仅涉及“是否持久化”，还涉及一个更紧迫的运行时问题：上下文窗口是固定容量的稀缺资源，而长程任务的上下文累积必然突破窗口上限。每个系统都必须回答“当窗口满了怎么办”——这不是可选的优化项，而是必须在工程上处理的硬约束。

七个系统在这个问题上的策略呈现出高度一致的分层降级结构。以 Claude Code 的四层压缩机制为最具代表性的实例：

1. **工具结果替换（Tool Result Replacement）**。对于超大工具输出（如读取整个日志文件），将原始输出替换为摘要或截断版本，保留工具调用签名但丢弃冗余内容。这是代价最低的降级——只损失了工具结果的细节，不损失对话结构。
2. **历史裁剪（History Snip）**。当工具结果替换仍然不够时，从对话历史中移除最早的轮次。保留最近若干轮和系统提示。代价是丢失了早期上下文中的决策依据。
3. **压缩摘要（Compaction）**。将对话历史的某个连续片段输入模型，要求模型输出一个结构化的摘要，然后用摘要替换原始片段。这保留了语义信息但丢失了精确措辞和推理细节。压缩后的状态是否仍然可用，取决于摘要是否保留了三段交接所需的关键信息（做到哪了、为什么、未闭合项）。
4. **溢出崩溃（Overflow Collapse）**。当前三层全部失效——窗口已压缩到极限，模型仍无法在当前上下文中继续有效推理时，系统要么强制中断并提示用户重启，要么在不可靠的上下文中继续推进。前者是可控的失败，后者是静默的偏航。

这四层降级路径揭示了一个更深层的事实：上下文治理的每一层降级都在**用信息保真度交换执行连续性**。工具结果替换丢失了细节，历史裁剪丢失了早期决策背景，压缩摘要丢失了精确措辞。而溢出崩溃意味着连续性本身也无法维持。状态治理的工程任务，就是在这条降级路径的每一步中，确保系统仍然持有“足够继续推进的最少信息”——而“最少信息”的内容，正是下一节的核心问题。

6.2.3 记忆 vs 状态：位置之争

Agent 社区有一个广泛传播的观点：Agent 需要“记忆”（Memory），包括跨会话的对话历史、向量化的经验检索和情节记忆。Hermes 的 Memory Provider 与 Curator 机制、Cline 的跨会话 Rules 和 Skills 加载，都将记忆作为一等公民嵌入了核心架构。

本节的核心判断是：长程实践需要的是结构化的语义标注状态，不是时序对话记录。记忆是皮层的可选插件，以 RAG、向量库、Skills 等形式提供，由用户按需启用，受真核的治理约束（权限、作用域、时效、激活条件、可撤销性）。状态是真核的强制组件，

所有 Agent 系统必须至少具备最低限度的状态传递能力。以下从信息结构、工程后果和长期风险三个层面展开这一判断。

两者的信息结构完全不同：

维度	记忆（Memory）	状态（State）
信息形式	时序对话记录、经验片段	结构化的语义标注
回答的问题	”之前发生了什么”	”做到哪了、为什么、未闭合项、接续入口”
组织方式	时间线或向量相似度	按检查点/闭合状态索引
可验证性	内容正确但可能不相关	每条有独立的闭合判据
治理需求	需要相关性过滤、去偏、时效管理	需要三值状态标注、独立评测

这意味着：留给下一任执行者几千条对话记录，不如留一份几十行的三段交接文档。这正是 **HC-3（状态传递）**¹ 的核心要求——状态传递不是”把历史记录传给下一任”，而是传递经过语义标注的结构化状态。三段交接的具体格式详见下文”状态传递的三段交接”小节。

将记忆嵌入真核层还有一个更隐蔽的长期风险。Agent 使用的本质是离散的^[5]：每次会话有独立的意图边界，旧会话中的偏好和判断在未经显式验证的情况下不应自动注入新会话。跨会话记忆累积的”倾向性”在短期 3–5 次会话表现为便利，长期 50+ 次会话后则堆积成系统性偏见——而用户甚至意识不到这些偏见是什么时候渗透进来的。Curator 做自动归档回答的是”记不记得下”，没有回答”该不该用这段记忆”：前者是存储管理，后者是认知管理。

6.2.4 状态传递的三段交接

这正是第4章诊断的 L4 集体盲区——跨执行断面交接治理——的工程应答。HC-3 要求的状态传递不是抽象原则，而是一个有明确工程格式的操作规程。跨执行断面的状态交接必须包含三段显式信息，任何一段缺失都构成交接不完整：

1. **工程过程（Process）**。已完成哪些检查点、以什么顺序完成。这是接续执行的”地图”——后续执行者据此判断哪些工作已经完成、哪些尚未开始，避免重复劳动或跳过前提。
2. **交接状态（Handoff Status）**。当前检查点处于哪个状态——已闭合/未闭合/阻塞/当前不可判定。其中”当前不可判定”是一个关键的三值状态：它承认某些检查点的闭合判定在当前信息条件下无法做出，同时阻止系统将这种不确定性静默地标记为”已完成”。如果同一检查点在连续两个检查点仍标记为”当前不可判定”，系统必须触发

¹HC-3 是长程实践理论六条逻辑必要条件之一（Zeng, 2025）。HC = Hard Condition（硬断裂条件），缺失 HC-3 会导致执行断面之间的信息蒸发，后续检查点在结构上无法续接。详见 §9.2 及长程实践理论原文。

EC-7（机向人主动请示）²，将不可判定的原因、已尝试的解决路径和该阻塞对后续依赖链的影响提交人类裁决。

3. **最终结果（Result）**。已完成检查点的验证结论（验证方式、验证证据、验证结果）和产出物清单（文件路径、版本、内容指纹）。这是防止假闭合的关键——“已完成”不能仅凭执行者自述，必须有可被独立核对的证据链。

这三段结构构成了一个执行闭环：过程告诉下一任“做了什么”，状态告诉下一任“做到哪了”，结果告诉下一任“做的对不对”。三段齐全时，后续执行者不需要重读历史对话，不需要猜测前人的判断依据，不需要从头发现未闭合项——这正是状态区别于记忆的核心工程价值。

6.3 权力治理

权力治理是真核中回答“系统与外部世界的行动边界在哪”的那一半。如果状态治理决定了系统能否“记住”，权力治理决定了系统能否“不乱动”。权力治理不仅约束系统行动，它在认知侧还有一个常被忽略的外延——表现层的设计决定了人类裁决者看到什么信息、以什么形式、在什么粒度上理解系统的行动建议。这一外延将在 Ch11 中展开。

6.3.1 权限管线的三层不可互相替代

一个常见的误解是将“工具可见性控制”等同于“权力治理”。实际上，一个完整的权力管线至少包含三个可以独立设计、不可互相替代的层次：

1. **工具可见性（Tool Visibility）**。模型能看到哪些工具的 schema？这是管线的最外层——控制的是“模型知道什么工具存在”。MCP 服务器开关、Skill 按需加载、皮层的渐进式披露都在这一层操作。
2. **权限审批（Permission Approval）**。工具调用产生后，由谁决定是否允许执行？这是管线的中间层——控制的是“谁有权力说可以”。审批者可以是人类用户、可以是独立的审查 subagent（如 Codex 的 auto-review subagent）、可以是基于规则的自动策略引擎。
3. **执行隔离（Execution Isolation）**。工具实际运行时，能触碰到什么系统资源？这是管线的最内层——控制的是“工具真正能干什么”。OS 级沙箱、文件系统只读挂载、网络策略、进程隔离都在这一层操作。

这三层的设计空间是相互独立的。一个系统可以给模型展示完整的工具列表，但每个工具调用都需要人类审批，同时所有工具运行在完整沙箱中。另一个系统可以只展示精简的工具集，但允许模型自由调用无需审批，且工具在宿主机文件系统上直接运行。

²EC-7 是长程实践理论七条工程补偿条件之一。EC = Engineering Compensation（工程补偿条件）。EC-7 要求系统在遇到超出自身判断力的决策时，主动向人类请示而非自行填补空白。详见 §9.3。

混淆这三个层会在架构中留下不可见的缺口——例如，以为“模型看不到某工具”就等于“系统不会执行该工具”。

将三个层映射到七个系统的实际实现，可以得到一个完整的权限频谱对比表（表 6）。

表 6 揭示了一个规律：系统的权力治理深度与其初始环境条件的“信任赤字”正相关。Codex CLI 出生在“不可信执行环境”——用户需要让 LLM 在第三方代码仓库中安全执行 AI 生成的代码，因此它在三层都走到了最深：沙箱策略统一过滤（第一层）、四级审批加 auto-review subagent（第二层）、OS 级沙箱（第三层）。trae-agent 出生在研究评估场景，不需要处理真实的用户数据或生产环境，因此三层都保持在最浅。这不是 trae-agent 的设计不如 Codex，而是它们的初始环境条件赋予了不同的权力治理需求。一个出生在无信任环境中的系统如果不建立深度权力管线，在结构上无法存活。

6.3.2 价值闭合与机向人主动请示

权力治理的最关键节点不是“阻止模型做坏事”，而是在**模型无法自行判断对错的时候，确保人类能够做出有依据的裁决**。这指向 SC-1（价值闭合）和 EC-7（机向人主动请示）两个概念的交叉地带。

SC-1 指出：目的闭合是局部性的。“这个函数通过了所有测试”是一个可以独立验证的事实。但“这个函数做了对的事”是价值判断——它的设计是否与整体架构一致、它的行为是否与用户意图对齐、它的副作用是否在可接受范围内，这些都不能由局部测试替代。³

LLM 作为封闭的信息处理系统，对外部漂移和决策分叉是无感的。当用户的真实需求在执行过程中发生了微妙的变化，基座会继续完美地执行一个已经过时的目标。

这就是 EC-7 的工程落脚点。EC-7 要求系统在以下情况下主动暂停并请示人类：遇到非技术性的价值取舍、自身判断力触及边界、发现执行路径与预设契约发生了静默漂移、当前决策可能对后续检查点产生不可逆的路径锁定效应。EC-7 的工程载体是**行为转型**：LLM 在遇到开放判断时，必须将“填空题”转型为“选择题”或“判断题”。不是抛回一个“你觉得该怎么做？”然后把认知负荷转嫁给人类，而是提供 2–4 个具体选项，每个选项附带影响范围和风险评估，让人类做的是“选一个”而非“想一个”^[6]。

AC 范式的 ec7_action_gate 机制将这一原则硬化到了工具调用路径上：在任何破坏性文件操作（DeleteFile、Write 覆盖已有文件）作用于受保护路径（public/.trae/rules/）时，操作在工具调用层面被截停，系统必须先通过 AskUserQuestion 获得人类显式授权。这不是“建议先问一问”的软约定，而是**操作前闸门**——代码在执行路径上实质性地阻断了未经授权的破坏性操作。

³SC-1 是长程实践理论六条逻辑必要条件之一。SC = Soft Condition（软断裂条件），缺失 SC-1 会导致系统“做完了”但“做得不对”。价值闭合以层级管控关系作用于目的闭合之上。详见 §9.2。

Table 6: 七系统的权限管线三层对比

系统	工具可见性	权限审批	执行隔离
Claude Code	八层权限管线逐层过滤，按需暴露	canUseTool 回调 + 交互式审批；per-session/project/command 粒度	文件系统读写分离标记；无 OS 级沙箱
Codex CLI	统一沙箱策略过滤	四级审批策略 + auto-review subagent	bwrap/Landlock/Seatbelt OS 级沙箱；文件系统 + 网络双 Policy
OpenCode	allow/ask/deny 三态权限	交互式逐条审批；权限策略以 TOML 配置文件持久化	无 OS 级沙箱；依赖宿主机权限
Hermes	动态工具注册与注销	check_fn 函数式权限校验；瞬态故障容忍	无 OS 级隔离
Cline	MCP + 内置工具双层	Hub/Spoke 架构中 Hub 承担审批路由	无 OS 级隔离；依赖 IDE 宿主权限边界
Cherry Studio	Main 进程管理工具流	模型输出解析后由 Main 流管理器决定是否执行	无 OS 级隔离
trae-agent	模块化可替换工具集	研究/评估场景，无独立审批管线	无隔离

6.3.3 回滚与隔离：限制错误的传播半径

权力治理不仅要控制“能做什么”，还要控制“做错了之后波及多远”。这指向 EC-1（恢复与回滚）和 EC-2（隔离与边界）两个工程补偿条件。

EC-1 的工程含义需区分两个层次：**硬回滚**与**软回滚**。

硬回滚是工程直觉上最直接的理解：检查点之间的状态不光要“能传递”，还要“能回退”。如果检查点 c_i 的闭合判定在事后被发现是错误的（测试通过了但方向不对），系统不能只有“硬着头皮继续”这一条路。它必须能回到 c_{i-1} 或更早的已知有效状态，从那里重新分叉。版本控制中的 `git revert`、文件系统快照的恢复、数据库事务的回滚，都属于硬回滚的范畴。硬回滚的前提是被回滚的对象具有明确的操作边界：一行代码、一个文件、一条数据库记录，回滚动作与操作对象之间存在确定的逆映射。

但长程实践中还有一类更隐蔽、也更致命的错误不属于硬回滚的射程。它不是“某一行代码写错了”，而是“某一轮决策做错了”。在 *research* 型任务中，一个在第四轮做出的假设选择，可能直到最终轮才被发现是错误的。这个假设驱动了第四轮到最终轮之间所有的后续推理、分叉和产出——这些产出在各自的局部范围内自洽、完整、格式正确，但都建立在错误的前提之上。这不是一个可以 `git revert` 的错误：没有一个“文件”承载了这个假设，没有一个“提交”标记了这个决策节点，没有机械化的逆操作可以将系统倒回到假设选择之前的状态。

这就是软回滚的领域。软回滚回退的不是操作，而是**认知轨迹**——回到某个决策节点，重新审视当时的信息条件、替代方案和选择依据，然后从那个节点重新分叉。软回滚无法依赖机械化手段，因为它回退的对象（假设、判断、偏好、价值取舍）从未被显式地记录为可寻址的状态单元。它们隐藏在对话历史的某个轮次中，嵌入在推理轨迹的某个段落里，分布在多个子 Agent 的产出之间。

软回滚的可行性因此取决于一个前提条件：**可追溯性**。如果系统无法回答“这个结论是在哪个检查点、基于什么信息、排除了哪些替代方案之后做出的”，那么当这个结论事后被证伪时，执行者面对的不是“回到哪里”的选择题，而是“从头再来”的必答题。从头再来的代价随任务规模非线性增长：当任务包含 50 个检查点、30 个子 Agent 产出、数千条对话消息时，从观察者的角度（无论是人类还是 LLM），遍历所有信息单元以定位错误源头在操作上不可行。它同时超越人类的注意力带宽和 LLM 的上下文窗口。不可追溯的系统在大规模长程任务中的回滚，约等于重做。

这正是状态治理 (§6.2) 与权力治理在此处的交汇点。三段交接中的“交接状态”（已闭合/未闭合/阻塞/当前不可判定）不仅是续接的锚点，也是软回滚的路标。每一个标记为“已闭合”的检查点在软回滚发生时都是可疑的：它的闭合判定所依赖的前提假设是否仍然成立？标记为“当前不可判定”的检查点更是软回滚的高危区域——它意味着当时的执行者已经意识到不确定性，但选择了搁置而非解决。软回滚要求系统在每一个检查点保留的不仅是“做了什么”（工程过程）和“做得对不对”（最终结果），更是“为什么这样做”（决策依据和排除的替代方案）。缺失第三项，软回滚就退化为硬回滚——回到上一个检查点，但无法判断上一个检查点的方向本身是否已经是错的。

回滚的决策权——无论硬回滚还是软回滚——都属于人类。回滚意味着放弃已完成的产出，重新分叉。这个决策的合理性只能由价值闭合层判断：回滚后新路径的预期收益是否大于放弃已有产出的代价？模型无法判断这一点。⁴

EC-2 的工程含义是：并行开发中的模块隔离、子任务之间的边界约束、权限在不同上下文中的差异性授予，这些不是“架构整洁性”问题，而是**错误遏制**问题。一个没有隔离的系统，一个子模块的越权操作就可以污染全局状态：删除一个公共文件、修改一个共享配置、写入一条错误数据。这种污染一旦发生，定位和修复的成本随系统规模非线性增长。

EC-2 将隔离从“好的工程实践”提升为“治理必需条件”。不隔离的系统在长程任务中不是“可能”出问题，而是“迟早”出问题。因为错误发生的概率随检查点数量累积，而失去隔离的传播半径使每次错误的代价都在放大。⁵

以上 §6.1–§6.3 完成了真核核心框架的完整展开：状态治理（四层持久化频谱、上下文降级路径、三段交接、记忆 vs 状态的边界）与权力治理（三层权限管线、价值闭合与 EC-7 硬化、硬回滚/软回滚的区分与可追溯性前提）。这两个维度不可互相替代，且必须同时存在——状态断裂不能用权限规则修复，权力越界不能用持久化修复。这是本章标题“真正的战场”的含义：原核的收敛只是起点，Agent 系统之间真正的结构性差异在治理基础设施层。下一章将这一框架推向两个方向的扩展——多 Agent 场景和治理成本经济学——作为核心框架的压力测试。

7 真核框架的扩展：多 Agent 场景与治理成本经济学

本章定位。第6章建立了真核的核心框架——状态治理与权力治理的二分法、三层权限管线、三段状态交接。本章展示这一框架在两个方向上的扩展：多 Agent 场景（横向——一个框架管多个 Agent）和治理成本经济学（纵向——框架本身的工程合法性）。两者共同验证真核框架的解释力和预测力。

§6 建立了单 Agent 实例的真核核心框架——状态治理与权力治理的二分法、三层权限管线、结构化三段交接。本章将这一框架推向两个方向的扩展：多 Agent 场景（横向扩展——当系统中存在多个 Agent 时，权力如何归属、状态如何隔离）和治理成本经济学（纵向论证——真核的“重”在工程上是否值得）。两者不是核心框架的可选附录，而是核心框架的**压力测试**：一个真正自洽的治理框架，必须能在多 Agent 和成本约束两个维度上保持解释力和预测力，否则其核心主张就不完整。

⁴EC-1 是长程实践理论七条工程补偿条件之一，要求系统必须在闭合判断出错、执行偏航或环境变化时回到已知有效状态。一次闭合判定错误不应永久污染整条实践链。

⁵EC-2 要求系统限制错误传播半径，避免一次局部失误污染整个实践单元。隔离不是替代检查点分解，而是限制一次失误的影响范围，让错误停留在局部、可诊断、可回滚的范围内。

7.1 多 Agent 场景：注意力分配、权力归属与规划层指引

7.1.1 核心原则：不应限制子 Agent 的推理深度与信息获取能力

多 Agent 系统中的一个常见设计冲动是：为子 Agent 设置能力限制——“子 Agent 只能读取文件，不能写入”“子 Agent 不能调用某些工具”“子 Agent 的推理深度不得超过 N 轮”。这个冲动的直觉来源是合理的——如果主 Agent 都可能出错，派出去的“下属”更不值得信赖。

但这个直觉混淆了**治理层**和**能力层**。子 Agent 的可靠性不取决于限制它的工具和推理深度，而取决于三件事：(1) 基座模型本身的行为可靠性（模型底层行为，非框架可控）；(2) 真核治理层对子 Agent 产出的接收和验证（状态主权——谁可以将子 Agent 的结论写入主状态）；(3) 规划层对子 Agent 任务的正确分解（任务是否足够干净、上下文是否足够独立、闭合判据是否足够明确）。

命题 7.1 (子 Agent 推理深度不减原则). 不应约束子 Agent 的任何能力和深度。约束应施加在三个位置：模型底层行为、真核治理层对子 Agent 产出的接收与验证、规划层对子任务的抽象与闭合判据。越抽象、越上层的任务，越可能有更高的嵌套深度——因为分解高层任务需要理解更复杂的语义拓扑，而这恰恰需要更多的推理轮次。

这个原则的实践指导价值是：当子 Agent 产出受限或质量不足时，首先检查的不应是“子 Agent 是不是太自由了”，而应是“主 Agent 给予子 Agent 的任务是否足够干净、闭合判据是否足够明确、接收子 Agent 产出后是否经过了独立验证”。限制子 Agent 的工具和深度是治标——它降低了子 Agent 产出质量（信息不足导致推理不充分），同时不解决根因。

7.1.2 性能面：Subagent 作为媒介强制的结构应对

多 Agent 架构的存在理由，不能仅用“分工合作”来解释——如果分工只是工程上的偏好，那它就不是架构必需品。Subagent 的真正必要性来自**媒介强制**：上下文窗口有上限，注意力随上下文长度衰减，而长程任务的累积上下文必然突破这些硬性限制。Subagent 不是绕过媒介强制，而是将媒介强制的压迫从主线程转移到更小的封闭上下文中——利用“问题越小，上下文越干净”的信息优势。

AC 技术报告中的最优性能投影 ($\mathcal{P}_{\text{opt}}$) 概念提供了一个形式化解释框架^[6]。同一个基座模型在长时序（主线程）和短时序（subagent）下表现出显著不同的可靠性：主线程作为长时序模型，即使单位轮次下的上下文增量 $c_{\text{main}}(t)$ 较小，其累积性能投影 $\mathcal{P}_{\text{main}} = \int c_{\text{main}}(t)dt$ 随 t 单调递增，极易突破 $\mathcal{P}_{\text{opt}}$ 阈值；subagent 作为短时序模型，即使单轮灌入较大上下文，积分区间极短，累积投影 $\mathcal{P}_{\text{sub}}$ 远小于主线程，极难触发阈值突破。这意味着——在充分提供上下文拼接的前提下——subagent 对于局部执行性任务的性能在绝大多数情况下**大于等于**主线程。这不是 subagent “更聪明”，而是它实质上重置了积分起点 ($t = 0$)，将性能投影拉回到了安全水位。

这个洞察解释了 **EC-3（独立评测）**⁶ 高度有效的原因：用同一个基座拉起独立审查 subagent（如 GN-004），不是因为它“更擅长审查”，而是因为它在干净的上下文中执行审查任务，性能投影在安全水位内——而主线程在长上下文中自我审查时，其性能投影可能已经逼近甚至突破了 $\mathcal{P}_{\text{opt}}$ 红线。

7.1.3 治理面：状态主权、提交权与责任归属

多 Agent 场景将真核的治理问题推向了一个新的复杂度：谁可以将子 Agent 的结论写入主状态？子 Agent 的产出在什么条件下被视为“已验证”？当子 Agent 的错误导致主线程偏航时，责任如何归属？

状态主权。子 Agent 不能自行将结论写入主状态。主线程的状态存储是对全局一致性的最终守护——子 Agent 只能提交产出，由主线程的治理层在接收时执行验证（契约校验、独立评测、人类确认），通过后方可写入主状态。这个约束的工程含义是：主线程必须保留对“什么进入全局状态”的最终否决权。

提交权。工具执行的结果可以直接回注对话历史（这是原核的协议要求），但子 Agent 的结构化结论（分析报告、代码变更、架构决策）在进入主线程的决策依据之前，必须经过显式的“提交-验证-接受”三步。这三步中的验证环节是独立于子 Agent 和主线程的——它由独立审查 subagent 或人类承担。

责任归属。在多 Agent 系统中，当一个错误被发现时，必须能够追溯到是哪个 Agent 在哪个检查点引入了错误。这是 EC-3（独立评测）在分布式场景下的扩展：不仅要独立验证每个产出，还要保持产出与执行者之间的可追溯链路。没有这一链路，错误定位就退化为“在所有子 Agent 的产出中大海捞针”。

7.1.4 表现面：超深度子 Agent 的用户监督

当子 Agent 的嵌套深度和推理深度都很大时，用户如何在不过载的情况下监督系统状态？属于表现层设计的一般性问题（Ch11 将从 EC-6/EC-7 出发推导表现层的通用约束）。此处仅指出真核框架对这一问题的界定：监督的可行性取决于状态治理是否在每一层嵌套中提供了收敛的交接锚点——如果子 Agent 完成任务后将结果浓缩为一份包含三段交接信息的结构化报告，用户就可以在检查点粒度上监督，而不需要追踪子 Agent 内部的推理轨迹。

皮层层的渐进式披露（§8）也将在能力过滤层面提供部分答案——通过选择性暴露而非全量推送，降低人类监督的认知负荷。

⁶EC-3 要求验收不能依赖执行者自述，必须由测试、检查器、类型系统、人工验收或其他独立证据链承担。它是 SC-3（未闭合标记）在工程上的强制实现方式——保证未闭合信号不依赖于执行者的自我报告。

7.2 治理成本经济学

读者在读完第6章的核心论证之后，可能会产生一个合情合理的质疑：真核的治理机制——锚点文档的三段交接、独立审查 subagent 的频繁拉起、EC-7 的人类请示、EC-1 的回滚机制——看起来“很重”。在单次任务中，这些治理机制的额外 token 消耗和延迟，是否得不偿失？

这是一个必须正面回答的问题，因为它触及了真核工程合法性的核心。

7.2.1 真核管理的不是道德，是三种稀缺资源

真核的治理机制不回答“Agent 应该做什么才是道德的”——那是一个超出工程范畴的问题。真核管理的是三类**稀缺资源**，它们的稀缺性来自客观约束而非设计偏好：

1. **上下文窗口（注意力资源）**。窗口容量是固定的硬上限。每一条未经过滤的历史消息、每一个未压缩的工具输出、每一段冗长的推理轨迹，都在消耗这个有限的资源。治理层的压缩、过滤和摘要策略，本质上是注意力资源的分配机制——将有限的窗口容量分配给当前最关键的信息。
2. **Token 预算（算力资源）**。每次模型调用都有真实的算力和时间成本。长程任务中，无治理的系统可能在错误路径上消耗大量 token 后才暴露问题——而被浪费的 token 无法回退。治理层的独立审查虽然增加了每次检查点的 token 开销，但它阻止了“在错误路径上跑完整个任务”的灾难性浪费。
3. **人类注意力（判断资源）**。长程实践理论中 H1（非对称验证带宽）指出，人的注意力预算 $B(t)$ 随累计时间单调非增。治理层的信息压缩和结构化呈现（EC-6：机向人主动解释，EC-7：机向人主动请示）的目标不是让人做更多判断，而是让人的每一次判断都有充分的信息基础，使得有限的注意力预算被用在最有价值的决策节点上。

从这个角度看，安全和成本不是两个独立的设计维度。它们是**同一结构的两个面**。一次在错误方向上的长程推进，同时是安全风险（产出不可用）和成本风险（token 被浪费）。“全自动”和“人机协同”也不是二元对立。它们是在不同资源约束下的不同配置。当任务的信息完备性足够高、闭合判据足够明确、外部漂移的风险足够低时，系统可以向自动化侧滑动，减少人类干预频率；当任务的语义不封闭度升高时，系统必须向协同侧滑动，在关键节点拉入人类判断。

7.2.2 真核的“重”是对不可逆偏航风险的高质量对冲

现在可以直接回答那个质疑了。是的，真核是“重”的——锚点交接、独立审查、人类请示，每一项都会额外消耗 token 和延迟。但这个“重”不是设计冗余，而是**对长程任务中“不可逆偏航风险”的高质量对冲**。

考虑以下对比。一个无治理的“轻量”Agent，在一次 30 轮的长程任务中，可能在没有任何检查点验证的情况下连续推进。前 15 轮看起来一切正常。第 16 轮，模型做了

一次静默偏航——它自以为理解了用户意图，实际上在第 10 轮时用户的需求已经因为外部环境变化而微妙地漂移了，但这个漂移从未被系统感知。从第 16 轮到第 30 轮，系统在错误方向上高效地消耗了大量 token，产出了一个“看起来完成了、但实际上方向错误”的完整结果。这 15 轮（甚至更多）的 token 消耗是纯浪费。而更致命的是，这个错误不是“系统崩溃了”。系统没有崩溃，它产出的东西自洽、完整、格式正确——它只是“不对”。这种假闭合在单次会话中难以被发现，可能在更下游的任务中才暴露。

对比一个有真核治理的系统。它第 10 轮检查点进行了一次独立审查（消耗了额外的 2000 token），审查发现了未闭合标记中的歧义。系统通过 EC-7 向人类发起了一次请示（消耗了额外的人类 30 秒关注），人类确认了需求漂移并修正了方向。从第 11 轮开始，系统在新的正确方向上推进，第 30 轮产出了一个真正可用的结果。总治理开销占任务总 token 的 10%–20%，但它避免了 50% 以上的无效消耗——还不算假闭合在下游任务中造成的级联浪费。

宁可花 token 做独立审查，也不能让基座在长上下文中产生假闭合并向后传播。治理成本是对偏航风险的对冲，不是对效率的净损失。

7.2.3 治理成本的优化方向

承认真核的治理成本是必要的对冲，不等于说当前的治理成本已经是最优解。以下三个优化方向分别作用于治理周期的不同阶段：自适应调节降低审查的触发总次数，缓存复用降低单次审查的 token 开销，结构化沉淀将人类判断从一次性裁决转化为可复用规则。三者并行推进，不存在先后依赖。

- 1. 审查频率的自适应调节。**当前的独立审查触发频率是基于规则的（T1/T2/T3 三重强制触发），而不是基于风险的。如果系统能够根据当前任务的语义不封闭度、历史偏航频率和执行者的可靠性信号，动态调节审查密度——高风险路径加密审查，低风险路径放宽间隔——治理成本可以在不牺牲安全性的前提下显著降低。
- 2. 治理开销的缓存与复用。**同一检查点的重复审查——例如因修正后重审——在当前的实现中是完整重新审查，不利用前一次审查中已经验证过的结论。如果独立审查 subagent 能够产生结构化的审查缓存（哪些部分已通过、哪些部分被阻断），修正后的复审可以只针对被阻断的部分，大幅减少重复 token 消耗。
- 3. 人类判断的结构化沉淀。**当前 EC-7 中的人类裁决结果记录在锚点文档中，但没有被系统地编码为可自动重用的治理规则。如果每一次人类裁决——“在这种情况下选方案 B 而不是方案 A”——都能沉淀为一条机器可读的治理规则，后续遇到相似场景时系统可以直接引用沉淀规则而无需再次请示。这是从“手工梯度下降”走向“治理规则自动化”的关键一步，也是 Ch13 将展开的“系统工程与 Agent 工程的边界与融合”主题的预埋。

这三个方向共同指向同一个工程判断：真核的治理成本不是常数——它随工程成熟度递减。

8 皮层（Cortex）：渐进式披露的能力供应链

本章定位。本章是决策五（能力与治理分离）的能力层补充。在建立了状态治理和权力治理的基建之后，需要决定”给 Agent 什么工具”——这个决定不能基于”越多越好”的假设，而必须在能力与认知负荷之间找到工程上的最优平衡。皮层层的设计目标是”对”——在恰当的时间让模型看到恰当的能力。

§5 展示了七系统在最底层共享同一个调用骨架，§6 揭示了七个系统在状态治理和权力治理上的深度分歧。但在最外层——系统为模型提供的工具集、技能库、扩展能力——七个系统的差异达到最大。本章论证：这层差异是正确的架构分层。

8.1 半透膜：模型的注意力空间与外部世界的选择性边界

皮层的根本问题是**选择性通透**。模型的注意力空间是一个固定容量的 token 预算——上下文窗口的有效可用区间被注意力预算的客观约束锁死。而可接入的外部能力——工具、知识源、Skill 库、服务——在理论上无界。这个张力强制要求一层选择机制：在每 step 中，系统必须有原则地决定哪些能力被允许穿过边界进入模型注意力空间，哪些被阻挡在边界之外。皮层就是这张边界——一张**半透膜**（semipermeable membrane）。

膜的管道结构。膜上分布着两类管道：

- **穿透膜（模型侧可见）：**模型感知其存在或效果——Skills 描述符、MCP 工具声明、RAG 检索结果、Memory 召回。这些管道扩展模型的信息或行动边界，直接参与模型的注意力分配。
- **不穿透膜（纯系统侧）：**模型不感知其存在——心跳、审计日志、自动快照、Hook/Guard（拦截但不通知模型）。这些管道维持系统的存活性、安全性和可观测性，但不进入模型的注意力空间。

这两类管道之间存在一个关键的不对称性：**纯系统侧管道可以独立存在，纯模型侧管道不存在**。任何模型可见的能力（一个 Skill 描述符、一个工具声明）必须有一个系统侧的基础设施让它能够被”看到”和”执行”。形式化地，设系统侧存在为 1、模型侧感知为 1，膜上管道只能取

$$(1, 0) \text{ 或 } (1, 1), \quad (0, 1) = \emptyset.$$

$(0, 1) = \emptyset$ 是结构约束，非填充状态。如果有人声称设计了一个”纯模型侧的皮层能力”，那它要么已经内化为真核（变成常驻上下文，不再属于皮层），要么必然依赖系统侧的某种无状态 API。

这个不对称性将 皮层 与 原核 和 真核 精确区分开。

原核 负责将模型输出中的工具调用解析为实际执行。真核 负责定义膜的通透规则：状态治理管入站通透条件（什么信息在什么条件下可以穿透膜进入模型），权力治理管出站通透条件（模型的什么行动在什么条件下可以穿透膜触及外部世界）。皮层 负责执行通透规则：膜的管道注册、调度、开关执行。

三层在工具调用路径上的分工如下：皮层 执行膜的通透（管道开关控制模型看到什么），真核 定义通透规则（在什么条件下允许穿透），原核 执行已穿透的调用（协议引擎）。

8.2 穿透管道的两个维度与机制定位

在当前的 Agent 工程实践中，皮层 的能力机制以多种形态出现：MCP（Model Context Protocol）提供标准化的工具发现与调用协议；Skills 提供可被按需加载的程序性知识单元；Hooks 在模型推理或工具调用前后插入自定义行为拦截点；Plugins 通过通用扩展接口引入外部能力。这些名称是工程实践中的约定俗成——它们描述了“当前大家把东西叫什么”，但并没有回答“皮层在架构上到底在解决什么问题”。要回答这个问题，需要先为皮层的能力建立一套分析坐标。

穿透管道的两个维度。对于穿透膜的管道（模型可感知的能力），有两个本质维度决定其治理归属：

维度一：方向。

- **入站**（External → Model Context）：外部信息注入模型注意力空间。治理归属 → 状态治理（相关性、时效性、上下文预算）。失效模式 → 注意力稀释或静默缺失。
- **出站**（Model → External World）：模型意图触及外部世界。治理归属 → 权力治理（权限、沙箱、回滚）。失效模式 → 不可逆破坏或越权。

维度二：激活条件（光谱上的三个离散位置）：

- **常驻**：始终在场，不随上下文变化（系统提示词、项目规则）。注意力成本最高，触发精度为 1。
- **条件可用**：基于任务状态、权限模式、检查点类型动态决定是否穿透（权限模式切换改变可见工具集、基于当前检查点加载相关 Skill 描述符）。
- **按需触发**：由语义匹配或模型显式请求触发（RAG 检索、Memory 召回、模型请求加载 Skill 完整内容、工具调用后结果回注）。注意力成本最低，触发精度依赖匹配/检索质量。

现有机制在膜上的定位。有了方向和激活条件两个维度，MCP / Skills / Hooks / Plugins 不再是分类框架——它们是膜上特定位置的约定俗成名称。同一名称的完整生命周期可能包含多个穿透事件：

- **Skill 描述符注入** → 入站 × 常驻或条件可用（描述符被框架推入上下文）。
- **Skill 完整内容加载** → 入站 × 按需触发（框架自动匹配或模型显式请求）。
- **MCP 工具声明** → 出站 × 条件可用（工具在列表中可见）。
- **MCP 工具调用** → 出站 × 按需触发（模型发起调用）+ 入站 × 按需触发（结果回注）。
- **Hook** → 不穿透膜（纯系统侧出站拦截——属于权力治理的执行臂，其治理逻辑在 §6 已完整论述）。

名称是偶然的，膜上位置是本质的。框架的预测力来源于此：任何未来的新协议、新标准，只需回答三个问题即可精确定位并推导治理需求：入站还是出站？何时激活？单次穿透代价是多少？

基于以上膜上定位，可以提炼出一个一般性的架构原则。

命题 8.1 (能力-治理分离原则). 一个 *Agent* 系统的架构应该将”能力是什么”（属于 皮层，膜上的管道注册）与”能力在何时以何种方式被激活”（属于 真核，膜的通透规则）分离为两个独立的设计维度。皮层 负责组织膜上的管道：定义工具接口、管理技能生命周期、注册拦截点；真核 负责治理膜的通透：决定哪些管道在当前权限模式下穿透、哪些技能在当前检查点应被激活、哪些拦截点在当前上下文中应触发。两者通过统一的注册接口解耦，使得能力提供方不需要知道通透规则，通透规则不需要理解能力细节。

8.3 渐进式披露作为膜的调度问题

在 *Agent* 系统的设计讨论中，”渐进式披露”（Progressive Disclosure）经常被归类为一种 UI/UX 策略——”不要一次性把所有菜单项摊给用户，按需逐步展开”。但将这一直觉移植到 皮层 层时，主语从”用户”切换为”模型”：不再是”不要让用户眼花缭乱”，而是”不要一次性把所有能力注入模型的注意力空间”。这一转换不是修辞性的，而是结构性的：它不是”为了让模型不困惑”而做的友好设计，而是注意力预算的客观约束强制要求的资源分配决策。在固定吞吐量下、面对无界能力空间时，膜的调度问题是不可回避的。

回看 §6 中提出的三类稀缺资源：上下文窗口（注意力资源）、Token 预算（算力资源）和人类注意力（判断资源）。这三类资源中，前两类在 皮层 层直接承受压力。一个典型的 *Agent* 系统可能注册了 50+ 个 MCP 工具、20+ 个 Skills、10+ 个 Hooks 拦截点。如果每个工具声明的 JSON Schema 平均占用 500–2000 token，全量暴露一个中等规模的皮层 能力库仅工具声明就要消耗 25000–100000 token。问题不在于”装不下”——现代大模型的上下文窗口动辄百万 token，容纳这些声明在容量上并非不可行。真正的问题是比例倒挂：当工具声明的 token 开销接近甚至超过实际任务内容的 token 时，就出现了”酱油比鸡贵”的局面。描述能力的元数据本身成了注意力预算的主要消耗者，真正承载任务语义的内容反被挤占。注意力预算的瓶颈不是容量上限，而是不该让元数据吃掉认知预算的大头。

因此，皮层的渐进式披露是结构必需品，不是优化项。

它的工程含义不是在 UI 层面控制信息的呈现节奏，而是在**工具注册层、技能加载层和拦截点激活层三个层面分别实现独立的按需投影机制**，确保模型在每一轮中只看到与其当前上下文最相关的能力子集。

这个判断将皮层的渐进式披露从一个表现层话题升格为膜的调度策略。

表现层的渐进式披露——IDE 只高亮当前 diff 而非展示完整变更历史、终端只显示最近 N 条输出而非全量日志——是同一原则在最外层的体现（§11 将展开表现层自身的收敛逻辑）。但它不是皮层层渐进式披露的“应用”，而是同一原则在三个不同堆栈层上各自独立实现的结果。三层不可互相替代：皮层层的能力过滤不能替代表现层的信息压缩，反之亦然。

值得指出的是，投影并不是皮层的专属现象。即使在最底层的原核，也存在一种被动形式的投影：模型在调用轮次 t 时看到的对话历史，只包含前轮实际被调用过的工具结果——未被调用的工具、被过滤掉的能力、未激活的 Skill 从未出现在上下文中。这不是原核主动设计的过滤机制，而是协议本身的副产品：只有实际执行过的调用才在历史中留下痕迹。这个观察的意义在于：**投影是 Agent 系统每一层的固有属性，区别只在于它是被主动设计还是被动产生、是粗粒度还是细粒度。**

8.4 Hermes 工具注册中心：皮层层的工程典范

Hermes 的工具注册中心（Tool Registry）围绕一个核心问题设计：如何让模型在每轮中只看到与当前上下文相关的能力子集，同时让能力的注册、发现和生命周期管理尽可能自动化。在七个考察系统中，它在这个问题上做得最清晰。三个互补机制共同构成了完整的答案。

(1) AST 扫描——零手工注册。 Hermes 在启动时扫描指定目录中的 Python 源码，通过 AST 静态分析识别所有被 `@tool` 装饰器标记的函数，自动提取签名、参数类型和文档字符串，生成标准工具声明 JSON。整个过程不执行任何代码。工程含义：添加新工具只需写一个 Python 函数并加上装饰器。注册中心在下次启动时自动同步，无需维护独立的注册清单。

(2) `check_fn` + TTL——条件投影。 每个工具可绑定一个 `check_fn`（布尔谓词）和一个 TTL。系统在每轮调用前执行 `check_fn`，动态判断该工具在当前上下文中是否应对模型可见：检查条件覆盖权限模式、会话状态、外部环境。TTL 提供时间维度的过期机制：经过指定轮次后工具自动不可见，强制重新评估。实现上有一个精致的细节：`check_fn` 结果缓存 30 秒，但若某工具近期曾返回可用、随后短暂失效，系统给予 60 秒宽容窗口（防止 Docker daemon 负载高峰等瞬态波动导致整个 toolset 被静默剥离），在此期间以最近可用结果为准，同时不缓存失败结果，确保下次调用重新探测。

(3) 瞬态故障容忍——优雅降级。 当 `check_fn` 执行失败或扫描异常时，注册中心不会崩溃。故障工具被标记为“暂时不可用”，系统以缩减后的能力集继续运行。这体现了皮

层的正确工程姿态：**能力的不可用不等于系统的不可用**。这构成与 原核 的结构性对比。模型 API 是硬依赖（原核 层，掉线则系统停摆），工具注册是软依赖（皮层 层，掉线则降级运行）。

这三个机制——发现（AST 扫描）、投影（`check_fn` + TTL）、降级（瞬态故障容忍）——验证了 命题 8.1 的工程可行性：皮层 组织能力仓库，真核 的 `check_fn` 决定何时投影，两者通过注册中心统一接口解耦。

8.5 皮层的架构性质：可插拔性与真核治理分工

Hermes 在工具注册中心之外，还构建了一套完整的记忆与技能管理体系：Memory Provider 负责跨会话知识检索与写入，Curator 负责技能的生命周期管理（创建 → 激活 → 归档 → 退役）。这套体系在工程上有一个值得注意的结构特征：记忆的检索与写入每轮自动触发，不依赖模型请求；记忆工具的 schema 在 agent 初始化时即注入工具列表；Curator 作为一个常驻后台系统持续运行。这些特征意味着，Hermes 的记忆体系事实上已经跨越了 §6.2 中界定的“可选皮层插件”与“准真核基础设施”之间的边界——它不是可拆卸的，而是深度嵌入 agent 呼吸周期的。

这指向一个对三层模型具有修正意义的实证判断：**皮层和真核的边界不是一道静态的墙，而是一个光谱**。当一个皮层能力——无论是工具注册还是记忆管理——被做到足够重、足够基础设施化，它会自然获得真核的属性。”该不该把它放进真核”不是一个可以用先验原则回答的问题——它取决于该能力在特定系统的特定演化压力下达到了何种重量。Hermes 的实证价值恰恰在于，它同时展示了 皮层 渐进式披露的成功面（工具注册中心）和边界面（记忆体系）。两者共同指向同一个结论：皮层 和 真核 的分离是必要的架构分层，但两者之间的具体分界线不是一个常数，而是一个需要持续判断的工程决策。当记忆系统被做到足够基础设施化时，它可能获得真核属性，这个阈值因系统而异，不能先验划定。

从这个光谱判断出发，可以自然地追问：皮层 作为一个架构层，其最根本的架构性质是什么？答案有两个维度——**可插拔性**（它自身是什么）和**与真核的分工接口**（它和谁怎样协作）。

可插拔性。皮层 是七系统差异最大的层：工具生态、Skill 组织方式、宿主集成策略——没有两家采取相同方案。这恰恰是正确的架构分层。其核心含义是：不同的 Agent 实例可以有不同的 皮层（工具集、Skill 库、宿主集成），但共享同一套 原核 + 真核 基础。核心循环和治理结构在架构上完全一致。将能力层的多样性强行压缩为统一标准，受损的是域适配精度；而 原核 与 真核 的收敛性允许 皮层 充分差异化，不损害治理一致性。

推荐与放行：皮层 与 真核 的接口。两者在工具可见性上的分工可以精确界定。皮层 的渐进式披露管**相关性过滤**——在当前任务域和上下文中，哪些工具值得被推荐给模型。真核 的工具可见性（权限管线第一层，§6.3）管**权限过滤**——在 皮层 产出的候选集中，

哪些工具被允许对模型可见。一句话：皮层管”该不该推荐”，真核管”该不该放行”。混淆两者会导致将权限不足错误诊断为能力不匹配，在无关的注册层反复调整——这是命题 8.1 在工程诊断层面的直接推论。

治理的边界判断。皮层能力库的扩充不会均质地转化为真核的治理压力——新工具引入的问题分布在多个层面。幂等性和可观测性属于工具设计，任务是否适合工具化属于任务设计，输出可视化属于表现层适配，只有权限边界和执行隔离属于真核的治理承载。知道什么不该放进治理，本身就是治理能力的一部分。将非治理问题全部推入真核，不仅加重治理负担，更模糊精准诊断——正如命题 6.1 要求区分”失忆”与”越界”，治理能力同样要求区分”治理机制缺失”与”非治理层的设计缺陷”。

两个维度最终收敛于同一个判断：皮层的丰富能力是真核治理的存在前提（没有能力可治理，治理空转）；真核的治理闸门是皮层安全释放的前提。两者不是扩展与限制的对立，而是同一架构问题的两个面。

9 长程实践对 Agent 治理的结构要求

第七章确立了 Agent 治理发生在真核层。本章从信息结构层面论证：为什么这些治理机制在长程实践中是结构必需品而非工程可选项：当关键输入无法在当前检查点被静态折叠时，治理是系统不崩的前提，不是锦上添花。

§3 到 §6 完成了三项预备工作：定义了狭义 Agent 的边界与三层同心模型，展示了七个系统在真核层的分歧，并确立了原核已收敛而真正的战场在真核。但有一个关键问题尚未被正面回答：真核的治理机制为什么是**结构必需品**而非工程可选项？本章的任务是给出这一问题的结构答案。

9.1 不可预计算顺序依赖链的形式定义

讨论长程实践对 Agent 治理的约束之前，必须先精确回答一个前置问题：什么情况下一个任务**在结构上**要求治理机制？不是所有任务都需要检查点分解、状态传递和独立审查。短对话、单次查询、确定性 workflows，用原核的裸循环就足够。治理机制只在特定的信息结构条件下被激活。本节任务是给出这个激活条件的精确形式。

9.1.1 静态可折叠与关键输入

长程实践理论的形式起点是一个区分：哪些信息可以通过纯内部计算获得，哪些必须以状态转移为代价才能被生成或导入^[5]。

定义 9.1 (静态可折叠). 对任意检查点 c_i ，设 $\mathcal{I}_i$ 为 c_i 时刻可被稳定调用的当前信息集。若某信息对象 X 无需改变外部世界状态、无需执行新的探测动作、无需承诺某条尚未选

定的分支，仅凭 $\mathcal{I}_i$ 中已掌握的材料通过有限内部计算即可获得，则称 X 对 $\mathcal{I}_i$ **静态可折叠**，记为

$$X \in \text{Fold}(\mathcal{I}_i).$$

定义 9.1 排除了三类常被误认为“当前已知”的东西：必须先做实验或刺探才能生成的反馈（认知缺口），必须先选定某条分支后才知道要取哪一类信息（决策分叉），必须等待外部世界进一步演化后才成立的真值（外部漂移）。它们都不能被算作当前局部闭环中可直接折叠的输入。

定义 9.2 (关键输入与顺序依赖). 对从 c_i 到 c_{i+1} 的推进，若存在一个最小信息对象 K_{i+1} ，使得执行或验证 c_{i+1} 的动作必须以它为前提，则称 K_{i+1} 为该转移的**关键输入**。若

$$K_{i+1} \notin \text{Fold}(\mathcal{I}_i),$$

则称 c_{i+1} 对 c_i 存在**顺序依赖**，记为 $c_i \mapsto c_{i+1}$ 。

定义 9.2 的工程含义是：不是所有“后一步需要前一步”的情况都构成长程依赖。前一步作为简单铺垫（如“先读完文件 A，再读文件 B”——两份文件在起点都存在，读取顺序只是执行安排而非信息生成）不触发长程结构。只有当后一步所需的关键输入**不能**在当前检查点被静态折叠时，即前一步不仅仅是执行铺垫，而是一个真正的信息生成或信息定形事件，顺序依赖才成为一个结构性约束。

定义 9.3 (长程实践区间). 若对某个检查点转移 $c_i \rightarrow c_{i+1}$ ，其关键输入 K_{i+1} 满足

$$K_{i+1} \notin \text{Fold}(\mathcal{I}_i),$$

则称该转移进入一个**长程实践区间**。若一个实践实例的执行路径中至少存在一个长程实践区间，则称该实例为**长程实践**。

定义 9.3 给出了治理机制的激活条件。不是所有 Agent 任务都属于长程实践——那些所需信息在起点已齐备、子问题边界可一次性给定、整条路径可预先折叠的任务，属于单次闭合的工程放大，原核的裸循环就足够。治理机制在且仅在 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的条件被满足时，才从“可有可无的工程优化”升级为“结构必需品”。

9.1.2 长程不是时间长——是不能预计算

本章开头提出的质疑——“一个足够聪明的基座模型能不能绕过治理机制”——现在可以给出精确的结构回答。设基座模型在检查点 c_i 处拥有任意大的上下文窗口和任意强的推理能力，但它能做的是对 $\mathcal{I}_i$ 的任意复杂的内部运算。定义 9.1 明确划出了 $\text{Fold}(\mathcal{I}_i)$ 的闭包边界：静态折叠的产物只能是 $\mathcal{I}_i$ 中已有信息的组合、变换和推导。当 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 时， K_{i+1} 尚未被世界产生。它可以是一个代码运行后的报错输出，一个甲方看到方案后的真实反馈，一个下一次实验的测量结果。这些真值不在 $\mathcal{I}_i$ 中。任意强的推理能力也只能在已有信息的基础上推演，不能凭空生成尚未存在的真值。

这不是“模型不够聪明”的问题，是**信息不在当前集合中的结构事实**。正如翻译一部 1000 页的小说不是长程实践（每页译文的关键输入——原文——在起点已齐备），而诊断“按下开关后电脑不亮”的故障必然是长程实践（每一步探测的关键输入必须在上一部探测执行后才生成）。长程不是工作量大，不是步骤多，不是时间长——而是**下一步的关键输入在当前检查点不可被静态折叠**。

命题 9.1 (长程与治理的结构对应). 设一个 *Agent* 系统 $\mathcal{A}$ 被部署到实践实例 Π 上。若 Π 包含一个长程实践区间（即存在 c_i 使得 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ ），则 $\mathcal{A}$ 必须在架构上为以下操作提供工程承载，否则实践将在该区间处结构性断裂：

1. 将当前检查点的闭合状态以结构化形式持久化（而非仅保留对话历史）；
2. 在跨检查点交接时为后续执行者提供语义标注的状态快照（而非仅传递原始输出）；
3. 在价值不确定的节点引入独立于执行者的判断机制（而非依赖执行者自述）。

命题 9.1 的三条要求直接映射到真核的三个核心组件：状态传递（HC-3, §6.2）、未闭合标记（SC-3, §6.2）和价值闭合（SC-1, §6.3）。这不是“AC 范式恰好选择了这些机制”。 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的信息结构在逻辑上强制要求对应的治理组件，任何不提供这些组件的 *Agent* 系统在该区间处必然经历结构性断裂。§4 中七系统在 L4（交接治理）上的集体空白，正是这一结构约束在演化史上的实证投影。

9.2 形式定义的治理后果

命题 9.1 的三条结构要求——状态传递、未闭合标记、价值闭合——分别对应 HC-3、SC-3 和 SC-1。本节从形式定义出发直接推导这三条在长程实践中的必要性；完整的条件体系和工程补偿条件详见 Zeng (2025)。

HC-3（状态传递） 源于 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的直接后果：检查点 c_i 闭合时的信息集 $\mathcal{I}_i$ 中不包含 K_{i+1} ，仅凭 $\mathcal{I}_i$ 的原始输出，后续执行者无法获知“为什么当前是这个状态、哪里还没做完、下一步从哪开始”。状态传递不是“把文件传下去”。它是传递一份带语义标注的结构化快照。这正是三段交接（工程过程 + 交接状态 + 最终结果，详见 §6.2）的形式根源。

SC-3（未闭合标记） 源于同一个结构事实的推论：既然 K_{i+1} 不可被 c_i 折叠，则 c_i 无法自行判定自己是否“真的完成了”——它只能在 $\mathcal{I}_i$ 的闭包内检验局部目的闭合，无法检验该闭合在 K_{i+1} 被纳入后是否仍然成立。因此系统必须区分“已闭合”“未闭合”“当前不可判定”三种状态，且不能依赖执行者自述。必须引入独立于执行者的验证机制（如 GN-004 独立审查，详见 §6.2）。

SC-1（价值闭合） 源于更深层的推论：HC-3 和 SC-3 只能保证“局部不走错”，不能保证“全局方向对”。当任务路径上存在决策分叉时，局部闭合无法回答“选哪条分支才对”——这是价值判断，不属于技术求解范畴。SC-1 强制要求在每个不可逆分叉处引入外部判断，这正是 EC-7（机向人主动请示，详见 Zeng, 2025）的结构起源。

$K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的根本原因是四种信息截断来源：语义不封闭（Agent 不知道下一步该问什么），反馈延迟（Agent 必须先执行才能获知正确方向），上下文衰减（历史信息随步数增加而退化），边界漂移（“完成”的定义在执行过程中可能改变）。四种来源共享同一个根因：前馈计算无法替代状态转移。

这四种来源分别对应四类信息结构断裂与四种治理机制的激活条件。语义不封闭是认知缺口，要求检查点分解和顺序约束；反馈延迟是媒介强制，要求 Subagent 和信息降级；上下文衰减是决策分叉，要求价值闭合和人类请示；边界漂移是外部漂移，要求独立评测和回滚。系统诊断框架详见 Zeng (2025)。

9.3 在七系统中的实证投影

将 §9.2 推导的 HC-3（状态传递）、SC-3（未闭合标记）、SC-1（价值闭合）三条约束映射到 §4 的七系统分析，可以精确诊断各系统在 L4（交接治理）上的集体空白。§4 的 L1–L4 连续能力表（表 4）已记录了全部七个系统在 L4 上的实证状态：trae-agent、OpenCode、Claude Code、Codex CLI 四家在 L4 列为空白；Cline 的 Cron 报告和 Hermes 的 Kanban 是最接近 L4 的设计，但记录的是“什么任务跑完了”或“什么任务在队列中”，而非跨断面的结构化交接。以下将三条约束逐项展开。

HC-3（状态传递）方面。 HC-3 要求检查点闭合时生成包含结构化状态快照、语义标注的未闭合项清单和可操作接续入口的交接输出。七系统中没有任何一个达到这一标准：Cline 的 Cron 报告仅生成任务级摘要，不包含检查点级的闭合状态判定；Hermes 的 Kanban 维护任务队列，不区分“已闭合”“未闭合”“当前不可判定”三种状态；其余五家在跨会话中断后，状态信息完全寄托于执行历史（对话日志、cell 序列、commit 链），它们是时序记录，不是结构化交接快照。下一任执行者必须阅读全部历史并自行推断当前状态，这正是 $\mathcal{I}_i$ 不包含 K_{i+1} 时长程实践的结构性断裂点。

SC-3（未闭合标记）方面。 SC-3 要求在 K_{i+1} 不可被 c_i 折叠的条件下，系统必须引入独立于执行者的验证机制，区分“已闭合”“未闭合”“当前不可判定”三种状态。七系统无一提供此类机制：它们全部依赖人类开发者自行判断“做完了没有”。这在单次闭合的工程放大中风险可控（验证标准在起点已齐备），但在 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的长程区间中，执行者无法在 $\mathcal{I}_i$ 的闭包内判定自身是否真正闭合，依赖执行者自述等于放弃验证。

SC-1（价值闭合）方面。 SC-1 要求在每个不可逆分叉处引入独立于执行者的价值判断。七系统在工具调用路径上均未设置强制截停机制，所有系统依赖“人类会自己盯着”这一隐含假设。当人类注意力耗散（详见 Zeng, 2025 中 H1 约束）时，该假设在结构上失效。EC-7（机向人主动请示）是对这一空白的工程补偿方案（详见 Zeng, 2025）。

七系统在 L4 上的集体空白不是偶然的工程遗漏。它们的出生场景（§4 的详细展开）从来不要求跨执行断面交接：每个系统的初始环境压力（终端单会话、IDE 单窗口、桌面单用户）使 L1–L3 成为生存必需品，而 L4 不在任何一家的初始演化路径上。但一旦承认长程实践要求 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的不可预计算顺序依赖链，L4 就从“暂未实现的增强功能”变为“不实现即结构性断裂的必需组件”。

9.4 三段非蕴含

§9.1 的定义体系直接导出三组非蕴含关系，这三组关系暴露了当前 Agent 评测体系与长程实践之间的结构性错位：

命题 9.2 (单次能力非蕴含). 单检查点内的模型推理能力提升，不蕴含跨检查点的治理能力提升。前者只证明“单步更好”，不代表在 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的长程区间中系统不会结构性断裂。

命题 9.3 (跨检查点非蕴含). 跨检查点的治理完备性提升，不蕴含通用 *Benchmark* 分数的提升。*Benchmark* 假设任务信息在起点已齐备 ($K_{i+1} \in \text{Fold}(\mathcal{I}_i)$ 对全部检查点转移成立)，而长程任务的核心特征恰好是这一假设不成立。评测长程能力需要长程尺子。

三组非蕴含的完整形式化展开及中间命题详见 §14.1。

9.5 收束

本章的核心结论是： $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 这一判据为 §2 的决策框架提供了形式化根基：决策一（状态治理）对应 HC-3 的工程承载，决策二（权力治理）对应 SC-1 的工程承载，决策三（检查点验证）对应 SC-3 的工程承载。长程实践理论不是外挂的复杂度，而是对 Ch2 框架“为什么这三条决策不可删减”的形式回答。

10 Agent 实践的离散本质

本章定位。 决策四（设计中断边界）说“在关键节点上预设中断面”。在明确了“要设中断点”之后，需要理解“为什么中断点必须是离散的结构化锚点”——这不是偏好，而是长程实践的信息结构所要求的。本章从信息结构层面论证：连续的“记忆流”在结构上无法替代离散检查点的定位与验证功能。

§9 证明了长程实践的信息结构要求检查点分解（HC-2）、状态传递（HC-3）和未闭合标记（SC-3）——这三者在逻辑上都不是可选项。但该章的论证在最后一节收束时搁置了一个更深层的问题：为什么这些治理机制的结构载体必须是“离散的”锚点和检查点，而不能是连续的记忆流或对话历史？

10.1 Agent 实践在工程上必须被设计为离散的

Agent 系统之所以需要检查点、锚点和中断边界这些离散的治理结构，最强且最工程化的理由不是 LLM 的 token 是离散的——而是**离散的检查点提供了可恢复、可验证、可审计、可分责、可插入人类判断的治理边界**。连续的记忆流或对话历史以时间为索引，不提供独立于执行者的有效性判定机制——它们无法替代离散状态锚点在治理中的结构

功能。以下几节将从四个层面展开这一工程论证，并在论证途中逐层回应连续体验直觉背后的认知机制及其工程局限。

作为对这一工程判断的底层支撑，LLM 实践在实现特征上也呈现出一致的离散性。判断一个系统在工程实现上是否离散，有一个可靠的操作标准：该系统的状态变化是否可取不可数无限个中间值？其最小操作单元是否可被无限切分为无间隙的连续流？其物理实现中是否存在真实的连续量？如果三个问题的答案均为否，那么该系统在工程实现上是离散的——不论使用者在主观体验上感到多么平滑，这些体验都是离散底层的叙事缝合，而非系统或交互的自体属性。

以下四个层面分别对应上述三个判据的逐一验证，以及第四个层面作为最后一个判据的物理确证。每个层面都直接落在 LLM 使用的可验证事实上，而非哲学推断。

1. **输入与输出均为离散 token 序列。** LLM 的输入被切分为来自有限词表的 token，输出同样是从同一个有限词表逐 token 采样生成的符号序列。不存在一个连续的语义空间让模型在其中做无限精度的微调——它只能在离散的符号上做概率选择。即便是“连续”的图像、音频等多模态输入，在进入模型之前也必须被量化为离散的 patch 或 token，否则模型在结构上无法处理它们。
2. **交互由离散回合构成。** 每一次人-LLM 交互遵循固定的离散结构：人类键入或语音输入文本（离散消息），发送至模型；模型在推理完成后返回一条或多条离散回复；人类接收、阅读、理解后决定下一步操作（引用、修改、否定、追问）。即使对话在主观上感觉“流畅”，操作上的每一步——发送、接收、阅读——都是离散的事件边界。流式输出（streaming）只改变了接收端的时延体验，没有改变这一基本结构。
3. **自回归解码在离散时间步上采样。** Transformer 的自回归解码不存在连续动力学。每个时间步，模型在当前已生成的 token 序列上计算下一个 token 的概率分布，然后从该分布中采样一个离散符号。解码状态在时刻 t 和 $t+1$ 之间的转移是一次离散跃迁，不是在一个连续状态空间中的微分演化。所谓“推理过程是连续的”是一种对注意力和隐状态变化的比喻性描述——这些隐状态在计算上是逐层、逐头、逐位置的离散矩阵运算，没有微积分、没有极限、没有连续函数。
4. **物理实现完全运行在数字电路上。** LLM 的推理在 GPU/TPU 上执行，而 GPU/TPU 是纯数字电路——晶体管开关在 0 和 1 之间跃迁，不存在真实的模拟连续量。模型权重是离散的浮点表示（float16/bfloat16），其精度受有限比特宽度约束。从物理层到算法层，整个系统中没有一处承载着连续的物理量或计算过程。

上述四个层面给出了充分且可独立验证的论证：LLM 实践在 token、交互、解码和物理实现四个层面上在工程实现上是离散的。连续不是系统或交互本体上的一种性质，而是人类使用者在其工作记忆中对一系列离散事件的缝合体验。这一判断可以形式化如下：

命题 10.1 (LLM 实践的离散特征及其治理后果). 设 $\mathcal{S}$ 为一个以 LLM 为核心推理节点的交互系统。 $\mathcal{S}$ 的每一次人-模型交互可表示为离散消息序列 $M = (m_1, m_2, \dots, m_n)$ ，其中每个 m_i 来自有限符号集 Σ 上的有限长序列。 $\mathcal{S}$ 在时刻 t 到 $t+1$ 之间的状态转移为一次离散跃迁：

$$s_{t+1} = \arg \max_{w \in \Sigma} P(w \mid s_1, \dots, s_t),$$

其中 $s_t \in \Sigma$ 为第 t 步采样得到的离散 *token*。该过程不涉及任何连续动力学——不存在可取不可数无限个中间值的状态变量，不存在连续微分方程描述的系统演化。 $\mathcal{S}$ 的底层实现是离散的。

命题 10.1 不仅在描述 *token* 采样的离散性，更在指出一个对 Agent 治理有直接后果的事实：**Agent 系统的每一次状态跃迁都在一个离散点上发生，而每一次跃迁之后的系统状态——不在本次推理的覆盖范围内、不在当前上下文的折叠闭包内——是一个需要外部锚定机制才能被恢复和验证的新状态。**这正是本章后续各节（从 §10.2 的认知缝合分析到 §10.5 的失效模式诊断）逐步展开的论证核心。

10.2 “连续体验”的实质

如果 LLM 的底层本质是离散的，那么使用者在日常交互中体验到的“连贯对话”“流畅推理”“自然延续”是什么？这个问题必须被正面回答，因为连续体验的真实感正是“用连续记忆替代离散状态治理”这一直觉的经济来源——如果体验已经是连续的，为什么还要额外建立离散的治理脚手架？本节将“连续体验”拆解为三个层次：认知缝合、流式错觉、需求混淆，以揭示其作为离散底层之上的感知服务的本质。

10.2.1 认知缝合：工作记忆对离散片段的叙事重构

人类体验到的“连续”与电影体验到的“连续”有相同的认知机制。电影由每秒 24 帧静止画面组成——帧与帧之间存在真实的空白间隙。但人类的视觉系统通过 *phi* 现象将这些离散帧缝合为连续的运动感知。同理，人类在与 LLM 的多轮对话中，**工作记忆对每一轮收到的离散 token 序列进行实时的叙事缝合**——将上一轮的推理结论嵌入当前轮的理解语境，将模型的记忆（通过上下文窗口中的对话历史）误认为模型的持续意识。“这种缝合是人类认知系统的功能，不是 LLM 系统的属性。

L60 的区分有直接的工程后果。检查点中断后，下一任执行者不具有上一任的认知缝合上下文——如果系统此时只提供对话历史作为“缝合原材料”，交接的可靠性取决于不同时间、不同状态下的人能否产生一致的缝合结果，而这个缝合过程在结构上不可靠。

这正是 **HC-3（状态传递）**⁷ 要求的是状态而非记忆的结构理由：状态是一份“已经缝合好了”的快照，后续执行者不需要自己重新做缝合工作。

⁷HC-3（Hard Condition 3：状态传递），长程实践理论的六条逻辑必要条件之一。缺失 HC-3 会导致执行断面之间的信息蒸发，后续检查点在结构上无法续接。HC-3 要求传递的不是对话历史而是经过语义标注的结构化状态——做到哪了、为什么、未闭合项、接续入口。详见 §6.2。

10.2.2 流式输出：降低了边界感知，没有消除离散本质

流式输出（token-by-token streaming）是连续体验幻觉的最直接制造者。模型不是一次性返回完整回复，而是逐 token 地“流”出文字，用户在阅读过程中感受到的是“模型在实时思考”。但这种体验的本质仍然是 token 级的离散流——每个 token 是一个从有限词表中采样得到的离散符号，token 与 token 之间存在不可消除的离散跃迁。流式输出降低的是人类对 token 边界的感知敏感度，而非消除这些边界本身。

这个区分的工程含义是微妙的但重要的。流式输出是**表现层**的优化：它服务于降低认知摩擦、提升阅读舒适度、维持注意力沉浸。这正是皮层中渐进式披露原则在表现层的一个实例。但它不能替代真核中的**状态治理**。流式输出让人类感觉“对话很顺畅”，不等于系统在跨检查点处具备可恢复的连续性。前者是表现层服务，后者是真核层基础设施。混淆两者的一个典型后果是：将表现层的连续体验误认为“系统已经具备了连续性”，从而在架构设计上省略状态治理。

10.2.3 人类喜欢连续，但需要离散

现在可以回到本节的核心区分：

连续性是系统提供的服务，不是用户意图、授权或价值判断天然连续的本体事实。

人类**喜欢**连续：低延迟的流式输出、自然的对话延续、无缝的话题切换——这些是合理的用户体验需求。但人类**需要**离散：写作需要在句子层面停顿和修改，编程需要在函数层面测试和重构，决策需要在方案层面比较和选择。在所有这些真实实践中，推进不是在连续的时间流上均匀发生的，而是在**离散的分歧点**上做出的选择、否定、确认和重新定向。每一个分歧点都是一个“可打断的边界”——人类在这些边界上介入、切割、选择、负责。

把“喜欢”和“需要”的区别放在 Agent 治理的语境中，可以获得一个精确的工程性原则：

命题 10.2 (连续-离散的工程分工). 在 Agent 系统的架构中，连续体验属于表现层的设计目标——它通过流式输出、低延迟响应、自然语言延续等手段降低认知摩擦，提升使用体验。离散性属于真核层的设计约束——它通过检查点分解、状态锚定、独立评测和机向人请示等手段保证系统在关键节点的可打断性、可恢复性和可问责性。两者不是竞争关系，而是分工关系：表现层用连续体验包裹离散的治理结构，让使用者在感受流畅的同时，系统在底层保有刚性治理。但**表现层的连续体验不能以削弱或绕过真核层的离散治理为前提**。如果“让对话更流畅”意味着省略检查点验证或跳过人类请示，那么流畅本身就是偏航的润滑剂。

10.3 回应了什么：对三种流行预设的逐条澄清

离散本质的判断直接回应了 Agent 社区中三个值得重新审视的流行预设。

10.3.1 预设一：“Agent 需要连续记忆”

§6.2 已建立记忆与状态的严格区分：记忆以时间为索引不提供有效性判定，状态以闭合为索引提供结构化语义标注。此处从离散/连续的框架补充一个关键限定：将对话历史当作连续录像带，要求 Agent “完整回忆” 来找到当前状态，等价于将治理负担从结构化的检查点转移到了不可检验的检索能力上。

10.3.2 预设二：“Agent 需要永不中断的会话”

这个主张的常见形式是：会话中断是 Agent 系统的缺陷——理想的 Agent 应该像同事一样，随时可以接上之前的话题，不需要用户每次重新交代背景。

命题 10.1 揭示了一个不能绕过的客观事实：**中断不是 bug，是离散本质的必然表现**。每一次 token 的采样都是一个离散跃迁的终点和下一个跃迁的起点，这就是最小粒度的“中断”。每一次对话回合的结束都是一个人工注意力的边界，这就是实践粒度的“中断”。每一次会话关闭都是上下文窗口被清空，这就是系统粒度的“中断”。中断不是工程疏忽导致的，而是离散底层的结构特征。在离散系统中，“不中断”在结构上不存在。

更重要的是，中断具有一个被“连续体验”叙事遮蔽的**正面工程功能**：中断让人类介入成为可能。如果 Agent 系统真的“永不中断”——即永远运行在同一个推理轨迹上，不给人类留下任何插入判断的边界——那么人类的价值判断（SC-1：价值闭合）和主动请示（EC-7）就没有插入的切口。一个“无缝”“不中断”的 Agent 在结构上等于一个不允许人类在中间节点介入的系统——而这个系统中的每一次偏航都将累积到最终输出时才能被发现。

因此，正确的工程态度不是“消除中断”，而是**主动设计中断边界**——在每个关键检查点处显式地、有结构地制造中断，使得人类判断和独立评测可以在离散节点上有序介入。连续体验（低延迟、会话上下文保持）可以降低中断的认知摩擦，但不能消灭中断本身——也不能消灭中断使人类介入成为可能这一正面价值。

10.3.3 预设三：“Agent 需要连续体验”

这个主张将连续体验本身视为 Agent 系统应该追求的本体目标，而非表现层的服务质量指标。它混淆了“用户体验”与“系统架构”两个不同层面的要求。

在用户体验层面，连续体验是完全合理的追求——流式输出、低延迟响应、话题自然延续，这些降低了人类的认知摩擦，使交互更高效、更愉悦。但在系统架构层面，以连续为前提的设计是危险的：它隐含地假设用户的意图在会话过程中不会发生质变、用户的授权在整个会话中持续有效、模型对任务的理解不会在执行过程中漂移。这三个假设在长程实践中都不成立——意图会演进、授权有边界、漂移是常态。

命题 10.2 给出了正确的工程分工：用户体验可以追求连续（用流畅的界面降低认知摩擦），但框架设计必须预设离散性（关键节点必须有显式干预接口）。不能假设“这套框架就是人类无干预”作为设计前提——即便某些任务确实在信息完备的区间内可以做到零干预。

10.4 实践指导价值：离散操作元素承载连续体验流

将前三节的论证——LLM 实践的离散本质 (§10.1)、连续体验的认知缝合实质 (§10.2)、对三种预设的逐条澄清 (§10.3)——投射到 Agent 系统的工程实践中，可以得到以下四条具体的设计原则。它们不是抽象的建议，而是从“离散为本、连续为用”这一核心判断中直接推导出的操作性约束。

1. **检查点必须被显式标记，不得依赖对话历史的隐含连续性。**每一次关键推进（任务开始、子任务闭合、分支选择、不可逆操作）都必须产生一个持久化的、结构化标注的检查点——包括闭合状态（已闭合/未闭合/当前不可判定）、验证证据和接续入口。这个检查点不是为了“方便的”记录，而是后续执行者——无论是同一个 Agent 在窗口清空后恢复、另一个 subagent 续接、还是人类在离线后重入——的唯一可信接续依据。对话历史可以保留作为参考，但它不能替代检查点——理由已在 §10.2 中充分展开。
2. **人类干预接口必须在离散节点上预设，不得假设需“要时再打断。”**如果一个系统没有在架构上预设人类干预的插入点，那么当人类认为需要介入时（例如看到模型偏航），人类能做的只有两种低效操作：要么停止整个会话重新开始（丢失全部中间状态），要么在对话中插入一条“等等，你方向错了”的消息（期望模型能正确理解并转向——而这在长上下文中恰好是极不可靠的）。正确的工程实践是：**在每一个可能涉及价值判断的离散节点（分支选择、不可逆操作、闭合声明）预设显式的人类干预闸门——这正是 EC-7 和 ec7_action_gate 在 AC 范式中的工程设计^[6]。**
3. **连续的体验流应被视为离散操作界面的皮“肤”，而非骨“架。”**流式输出、对话上下文保持、自然语言延续——这些是让离散的检查点之间感“觉流畅的”表现层服务。它们的工程角色是降低认知摩擦，而非替代结构性治理。正如一个设计良好的 IDE 让开发者感“觉代“码在自然流动，但实际上每次保存、每次提交、每次测试运行都是离散的操作节点，Agent 系统的连续体验也应该包裹在一套离散的治理骨架上。表现层的连续不应该要求真核层放弃离散的锚定机制。
4. **中断面不是系统缺陷，而是治理的天然抓手。**§10.3 已经论证了中断的正面功能——它使人类介入和独立评测成为可能。在工程设计中，应当主动识别并标记系统中的“天然中断面”：会话结束、上下文窗口清空、子 Agent 启动、检查点转移。在这些面上强制插入状态快照和验证步骤。不是在中断发生时被动补救，而是**把中断面设计为治理活动密集发生的正面节点**。这正是 §6.2 中三段交接结构（工程过程 + 交接状态

+ 最终结果)的工程哲学:交接不是出问题后才需要的应急手段,而是每个检查点转移时必须执行的常规程序。”

这四条原则可以收束为一条设计准则:健康的 Agent 实践是用离散的操作元素承载连续的体验流。离散赋予实践可打断、可恢复、可问责的结构骨架;连续赋予使用流畅、低摩擦的交互皮肤。前者不可放弃,放弃则实践瓦解;后者不可与前者混淆,混淆则治理架空。

10.5 离散机制提供了连续机制无法提供的三种治理功能

将连续记忆流或对话历史作为状态治理的替代品,不是“违背洞察”后会遭遇什么惩罚的问题——而是连续机制在结构上无法提供离散检查点天然承载的治理功能。以下三种功能是离散机制的独有优势,不是设计偏好,而是信息结构上的必然。

1. **状态可追溯与错误隔离。**离散检查点将连续的 Agent 执行轨迹切分为可独立审计的片段,使假闭合传播在下一个检查点被阻断。每个片段有明确的起点(上一检查点的闭合状态)和终点(当前检查点的闭合声明),独立评测在检查点处介入,而非在整个执行结束后追溯。在七系统中的实证投影:AC 范式的三段交接结构(工程过程 + 交接状态 + 最终结果)正是这一功能的工程实例——每个检查点的闭合必须附带可独立验证的证据,后续执行者不依赖前序执行者的认知缝合。
2. **权限边界强制执行。**离散状态切换触发权限重评估——在检查点处,系统重新评估当前 Agent 的已授权限,防止权限在连续的自主执行中逐步漂移。连续会话中,权限的授予与撤销没有自然的插入点;“一直有权限”在操作上等价于“从未被重新审查”。在七系统中的实证投影:EC-7 的 `ec7_action_gate` 在每次破坏性操作前强制触发权限闸门,这是离散检查点上的权限重评估,而非连续会话中的一次性授权。
3. **表现层可审计性。**离散状态快照将 Agent 的内部表示压缩为人类可读的检查点摘要,使人类审查成为可能——连续机制中的审查窗口是模糊的,因为“当前进行到哪一步了”没有自然的答案,除非执行者主动停下来描述自己的状态。在七系统中的实证投影:AC 范式的 `note` 体系在每一个检查点处写入四字段(做到哪了、为什么、未闭合同、接续入口),这正是离散快照在表现层的可审计实例。

如果连续机制不使用离散检查点,它目前尚未解决一个具体问题:如何提供这三种等价的治理功能?这不是“违反洞察”的后果问题,而是“替代方案尚未解决”的设计问题。上述三种功能——状态可追溯与错误隔离、权限边界强制执行、表现层可审计性——分别对应原核的自检与独立评测(HC-4/EC-3)、真核的运行时权限治理(EC-7)和皮层的渐进式披露与可审计接口。它们不是离散机制的附属品,而是离散检查点作为治理结构载体的直接产物。

10.6 收束与桥接

本章的论证从四个层面逐层递进展开，形成一条完整的论证链：

1. **LLM 实践的底层本质是离散的**——从 token 粒度、交互回合、自回归解码和物理实现四个层面得到一致且可验证的证据。连续是系统服务的表现，不是系统本质的属性。
2. **连续体验是工作记忆对离散片段的叙事缝合**——它的认知机制与电影帧合成连续运动的机制同源。流式输出降低了边界感知但没有消除 token 的离散本质。人类喜欢连续（体验需求），但需要离散（实践需求）。
3. **离散本质判断直接回应了三种流行预设**——Agent 需要连续记忆（不，Agent 需要离散的状态锚点）、Agent 需要永不中断的会话（不，中断是使人类介入成为可能的必要边界）、Agent 需要连续体验（用户体验可以追求连续，但架构不能以连续为前提）。
4. **离散机制提供了连续机制无法提供的三种治理功能**——状态可追溯与错误隔离、权限边界强制执行、表现层可审计性。在不使用离散检查点的前提下，如何提供这三种等价功能，目前尚无可行的替代方案。

这些结论引出一个自然的下游问题：如果离散性是 Agent 实践的底层本质，检查点分解、状态传递和未闭合标记是长程实践的结构必需品，那么 Agent 的表现层必须满足什么工程约束，才能让人类在这些离散的治理锚点上做出有效判断？

下一章将展开这一论证。EC-6（机向人主动解释）和 EC-7（机向人主动请示）对表现层的工程载体提出三个硬性条件：信息密度、操作时延、状态可追溯。在现有技术空间中，这三个条件的工程兑现收敛于类 IDE / 工作台的表现形态。

11 表现层的收敛

在明确了“治理需要人类在中断点介入”之后，一个被普遍忽视的问题是：人类介入时看到的是什么界面。这个界面的信息密度和呈现方式直接决定了治理的实际有效性。表现层的设计目标是从信息洪流中提取结构化锚点，让人类在一个视图内看到系统在哪个检查点、产出是什么、需要你判断什么——人类在检查点做判断的界面，应该长成 IDE 或工作台，而不是聊天窗口。

§10 论证了 Agent 实践在底层是离散的——离散的检查点、离散的状态转移、离散的人类介入边界——并指出健康的 Agent 系统必须用离散的操作元素承载连续的体验流。但这一论证留下了一个未回答的工程问题：如果“连续体验”是用户可感知的服务质量，而“离散可打断边界”是实践成立的结构前提，那么这二者应该由什么工程形态来同时承载？

本章的回答是：表现层（Presentation Layer）不是 Agent 核的附属装饰，而是人机协同的结构性接缝。它的形态选择不是审美问题，而是由 EC-6 和 EC-7 两条工程补偿条件从信息结构层面推导出的工程必然性。这一推导不假设任何特定产品、宿主或框架，只看“人需要在检查点做出有依据的判断”这一条需求，对表现层提出了什么无法被对话气泡、CLI 文本流或纯 API 返回满足的硬性载体要求。

11.1 EC-6/EC-7 的工程载体要求

§9 在定义 EC-6 和 EC-7 时，主要讨论的是它们的触发条件和信号协议——什么时候必须向人发信号、以什么格式发信号。但这两条条件的成立需要一个隐含前提：人类接收信号之后，必须能够在合理的时间与认知成本内理解信号的实质内容，并依据它做出有效判断。如果信号被送达了，但接收者看不懂、或看懂了也做不了判断、或看懂了也做不了判断但无法执行后续操作，那么 EC-6 和 EC-7 的触发只是形式上完成了，工程上并未兑现。

因此，EC-6 和 EC-7 对表现层提出的是两条完全可以从信息结构推导出的硬性载体要求：

- **EC-6（机向人主动充分解释）的载体要求：可操作的结构化状态视图。** EC-6 要求“人在检查点必须理解‘为什么变成这样’”。这个“理解”不是纯文本解释可以做到的——当系统在执行了 37 步操作、修改了 14 个文件、产生了 3 次回滚和 2 次方向修正之后，一段 Markdown 格式的“我做了以下操作”摘要，即使文本本身准确，也无法让人类在可接受的时间内在脑中重建完整的因果链和当前断面的真实状态。EC-6 的工程兑现要求的是：人类不需要通过阅读时序叙事去“推导”当前状态，而是能看到一个结构化的、可操作的当前状态快照——哪些文件被修改了、修改了什么、哪些检查点已闭合、哪些仍未闭合、当前断面的依赖链状态是什么。
- **EC-7（机向人主动请示）的载体要求：差异对比 + 影响范围 + 替代选项的结构化呈现。** EC-7 要求“人在价值节点必须能做出有依据的裁决”。这个裁决的质量不取决于系统有多诚恳地请求指示，而取决于人类拿到的是否是可比较的信息。如果在架构分叉点，系统用三段自然语言分别描述了方案 A、方案 B 和方案 C，人类只能靠“感觉哪个听起来更合理”做出判断——这不是 EC-7 期望的有依据裁决。EC-7 的工程兑现要求人类在裁决时能看到差异对比（A vs B 到底变了什么）、影响范围（选 A 会影响哪些模块/检查点/下游依赖）、和替代选项的结构化评估（不是“方案 B 也可以考虑”，而是方案 B 在哪些维度优、在哪些维度劣、风险标签是什么）。

这两条载体要求合在一起，引出一个明确的推论：满足 EC-6 和 EC-7 的表现层，不能只是一个从执行者到人类的单向信息管道。它必须是一个**双向的、可操作的协同工作台**——人类不仅要看，还要能点、能改、能运行、能对比、能回退。这直接导向下一节的三个条件。

11.2 三个必要条件

从 §11.1 的两条载体要求出发，可以推导出任何满足 EC-6/EC-7 的表现层必须同时具备以下三个条件：

1. **信息密度高：不只是对话气泡。**人类在检查点需要同时看到的不是”模型的思考过程”，而是**当前状态 + 差异对比 + 风险标注**的并置视图。对话气泡模型下，这些信息被线性地分散在时间轴上——状态在第 12 轮对话中被更新、差异在第 17 轮被导出、风险在某个折叠的工具调用结果中被隐藏。人类必须在脑中跨时间轴缝合这些信息才能形成完整判断，而正如 §10 所论证的，离散的实践不能依赖连续的叙事缝合来替代状态标记。信息密度高意味着：不同类别的信息（状态、差异、风险、历史）被分配到不同的**空间位置**而非时间位置：文件树承载结构概览；diff 视图承载变更内容；状态标记承载闭合进度。人类通过视觉扫描而非时序回溯来获取全景。
2. **操作时延低：不只是等待模型生成回复。**EC-7 的裁决不是一次性事件——人类在裁决之前通常需要验证：这个方案真的能编译吗？改这个配置会不会影响其他模块？上一次类似的分叉最终导致了什么后果？如果每一次验证都需要”向 Agent 发出一个新指令 → 等待模型生成回复 → 阅读回复”，那么裁决的时延被拉伸为数分钟甚至更长，人类注意力的持续性被反复打断，判断质量下降。操作时延低意味着：人类可以在不通过 Agent 中转的情况下直接执行某些确定性操作——运行测试、查看日志、LSP 跳转到定义、git blame 查看历史责任人。这些操作不需要模型的非确定性推理，不应被捆绑在”等 Agent 回复”的周期上。
3. **状态可追溯：不只是看到当前结果。**长程实践的一个核心困难是：当人类在检查点 c_k 介入时， c_k 的当前状态是 c_1 到 c_{k-1} 全部决策和操作的累积结果。如果表现层只展示”现在的样子”，人类无法判断”现在这个样子”是合理收敛的结果，还是某一步偏航后假装没有发生偏航的结果。状态可追溯意味着：人类能看到变更历史（不仅是”改了”而且是”谁在什么时间以什么理由改的”）、因果链（每一次关键决策的前因后果）和证伪记录（哪些路径被尝试后被证明不可行）。这是 EC-3（独立评测）在时间维度上的延伸——独立评测可以是某一时刻的快照审查，而状态可追溯要求的是跨时间的证据连续性。

这三个条件是从 EC-6 和 EC-7 的工程兑现条件出发可以推出的必要条件：缺了任何一个，EC-6 或 EC-7 的工程兑现就会出现结构性缺口：缺信息密度 → EC-6 的”理解”无法做到；缺操作时延 → EC-7 的”裁决”被锁定在等待周期上；缺状态可追溯 → EC-6 和 EC-7 共享的”在正确信息基础上做判断”的前提被架空。

11.3 收敛判断：类 IDE / 工作台形态

以上三个条件构成了一组工程约束。现在的问题是：在现有的技术手段中，什么样的表现层形态能够同时满足这三者？”理论上可能满足”不等于”工程实践中已被验证能够满足”。回答必须建立在归纳证据之上，同时保留对未来的开放性。

以**现有技术眼光**，在**一般性层面**（特指知识实践的高摩擦长程场景），**工程实践中已观察到的最有效形态**收敛于类 IDE / 工作台形式。这是归纳，而非演绎：当前所有替代形态（纯对话、纯 CLI、纯 API、通知推送）都至少在三个条件中的一个上存在可被证实的结构不足。具体而言：

- **Diff 视图**——变更的结构化呈现。直接满足”信息密度高”和”状态可追溯”：信息不再被编码为”第 14 行从 X 改为 Y”的自然语言描述，而是以空间对照的方式并置新旧版本；变更历史通过 `git blame / git log` 形成可追溯的时间链。
- **文件树 + 编辑器**——状态的高密度可操作视图。直接满足”信息密度高”和”操作时延低”：人类通过文件树的空间结构一次理解整个项目的受影响范围，通过编辑器直接定位到任意位置进行修改或核实，不需要通过 Agent 中转。
- **终端 + LSP**——低时延的验证反馈。直接满足”操作时延低”：编译验证、类型检查、测试运行——这些确定性操作可以直接在终端中执行，LSP 提供了即时的类型错误、引用跳转和代码诊断，反馈回路从”分钟级”缩短到”秒级”。
- **Git / 版本控制**——状态的可追溯变更链。直接满足”状态可追溯”：每一次提交都是明确的检查点快照，`commit message` 记录了变更的理由，分支结构记录了决策分叉和合并历史。Git 是 EC-3（独立评测）在时间维度上的工程锚——审查者可以看到”中间过程长什么样”而非仅看到”最终结果是什么样”。

这四个要素并非偶然地共存在同一形态中。它们之间的协同关系体现在：Diff 提供变更的精准对比；文件树提供导航范围；终端 + LSP 提供即时验证；Git 提供追溯链。这恰好覆盖了三个条件的全部交互缺口。没有一个要素可以独立满足所有三条；它们的共同存在构成了表现层收敛到类 IDE / 工作台形态的工程理由。Claude Code 的 `transcript view`^[7] 和 Hermes 的 TUI 等独立系统的发展轨迹，也各自从不同初始形态被迫长出越来越像 IDE 的补丁。这不是产品外观的趋同，而是 EC-6 和 EC-7 的结构约束在一致性上反复施压的自然结果。

11.4 弱化限定：功能形态的收敛，不是产品形式的唯一化

上一节的收敛判断做了一个强主张——三个条件的工程兑现收敛于类 IDE / 工作台形态。对这一主张必须附加以下四层限定，以精确锁定其适用范围并防止被误读为”Agent 必须被改成 VS Code 插件”或”IDE 是 Agent 交互的终局”。

1. **“以现有技术眼光”——不排除未来出现更优形态。**当前观察到的最有效形态收敛于 IDE，是基于 2026 年 7 月已实际存在的表现层技术——React 虚拟 DOM、WebSocket 实时通信、LSP 协议、Git 版本控制、Monaco/CodeMirror 编辑器——所做的归纳。如果未来出现能够以更低认知成本提供同等或更高信息密度、更低操作时延和更强状态可追溯性的新形态（例如沉浸式 VR 工作空间、基于 Agent-to-Agent 协议的全自动审查面板、或尚未被发明的交互范式），当前的收敛判断可以被证伪。这正是 §14 中可证伪性子节的设计意图。
2. **“在一般性层面”——特指知识实践的高摩擦长程场景。**本收敛判断的适用范围是本文讨论的核心场景：任务存在不可预计算的顺序依赖链、需要多检查点的人类价值裁决、状态在当前断面并不封闭的长程实践。对于短程、低摩擦、信息充分闭合的任务（如一次性的问答、单文件的语法修正、确定性的数据转换），三个条件的约束力显著减弱：纯对话界面或 CLI 可能就足够了。收敛判断不应被推广到所有人类-Agent 交互场景。
3. **“工程实践中已观察到的最有效形态”——不使用“最优”，是归纳而非演绎。**“最优”需要穷举全部可能形态并证明其他形态必然劣于候选形态——这在工程领域通常不可行。“最有效”则只需要在当前可观察的候选形态集合（纯对话 / CLI / API / 通知推送 / IDE / 工作台）中进行比较，并给出比较的依据（在信息密度、操作时延、状态可追溯性三个维度上的表现差异）。Cortex 中渐进式披露原则（§8）在不同宿主中的具体实现差异，进一步支持了“非唯一最优”的立场——同一个 Agent 治理框架可以在不同表现层宿主中以不同方式优化三个维度的权重分配。
4. **功能形态的收敛，不是产品形式的唯一化。**这是最关键的一层限定。所谓“回归 IDE”，应**严格**理解为功能形态的收敛——即满足 EC-6/EC-7 三条件的表现层在功能集上趋近于 IDE / 工作台所提供的那组信息密度、操作能力和追溯机制——而**不是**产品形式的唯一化（Agent 必须寄生在 VS Code、JetBrains 或 Electron 中）。研究工作台（如 JupyterLab 的数据科学生态）、科学实验控制台（如实验设计-运行-分析-记录的集成环境）、运营指挥台（如实时监控-告警-决策-执行的指挥面板）、法律案件工作台（如证据-法律条文-判例-论证的集成空间）——这些形态的产品形式完全不同，但它们在功能层面共享了相同的结构：以空间组织信息替代时间叙事缝合，以人类可操作的控制面板承载离散的价值裁决点，以版本化的状态记录替代蒸发性的对话历史。

这四层限定是收敛主张的精确边界——收敛判断在边界内部是强主张（三条条件从 EC-6/EC-7 严格推导，四个工程要素协同满足三条），在边界外部是开放问题（未来技术可能产生更优形态，短程场景可能不需要全部三条，不同宿主可以在三个维度中做不同权重分配）。

11.5 从表现层回看治理：三层模型的完整性

表现层不属于 §3 中定义的三层同心模型中的任何一层——它不参与模型调用循环（原核），不执行状态持久化或权限拦截（真核），不提供工具或技能给模型（皮层）。但表现层与真核之间存在一条双向约束关系：真核定义了表现层**必须承载什么信号**（EC-6 的”解释”和 EC-7 的”请示”），表现层的工程能力反过来决定了真核的治理机制**能被人类兑现到什么程度**。如果表现层只是一个对话气泡窗口，真核中最精密的权限管线、审批 workflows 和价值闭合检查点在人类端的作用将被压缩为”点一个同意按钮”——EC-7 的”有依据的裁决”退化为”盲选”。从这个角度看，表现层的收敛不是一个独立于治理框架的 UI 设计问题，而是真核的治理能力在人机接缝处是否能够完整”落地”的结构性约束。

下一章将从反馈信号方向观察 Agent 治理面临的更深层结构困难：即使表现层完美地兑现了 EC-6 和 EC-7，人类裁决所产生的反馈信号仍然无法像模型训练的梯度下降那样被自动归因、积累和优化。

12 反馈信号的结构性不完备

本章定位。决策六（承认治理无法自动优化）说了”你不能等待自动优化 Agent 治理的算法”。在前两章分别从离散结构和表现层角度讨论了治理的工程化之后，需要面对一个更深层的困难：即使表现层完美兑现了工程补偿条件，人类裁决所产生的反馈仍然无法被自动优化。本章给出根本原因——四重语义不封闭使反馈信号在结构上无法像模型训练的梯度下降那样被自动归因和积累。

Ch11 讨论了表现层如何通过 IDE / 工作台形态承载人机协同的 EC-6/EC-7 要求。但 Agent 治理本身如何改进？为什么 Agent 框架都是”演化”出来的而非”设计”出来的？本章论证：根因在于 Agent 治理面临的反馈信号存在结构性不完备——与 LLM 训练的自动梯度不同，Agent 治理无法获得稳定、可微、可批量的反馈信号。

这一结构性不完备解释了 Agent 框架为什么都是”演化”出来而非”设计”出来的——如 §4 所展示的七系统环境演化史。它也解释了为什么真核中治理机制的每一步进步（从 L0 的无持久化到 L3 的事件溯源，从无审批到多层权限管线）都是被真实的失败逼出来的，而非被更优雅的理论推导出来（§6 的完整展开）。

但这引出一个更深的问题：如果这些治理机制的改进不能依赖自动化的反馈回路，那么 Agent 治理的迭代在根本上依赖的是什么？

本章的回答从反馈信号的信息结构出发，给出一个结构性的解释。核心洞见是：Agent 治理面临的反馈信号存在四重语义不封闭，使其在结构上无法像 LLM 训练的梯度下降那样被自动归因和利用。

Agent 治理并非完全没有反馈信号。但这些信号——来自人类的判断、独立审查的评估、测试的通过/失败——在信息结构上无法像梯度下降那样被自动归因、累积和优

化。这是本文用”不完备”而非”缺失”来描述这一限制的原因。

12.1 两类优化范式的对比

将 LLM 训练的反馈闭环与 Agent 系统的反馈闭环并列，可以清晰看到两者之间的鸿沟。

LLM 训练的反馈闭环：

$$x \rightarrow f(x) \rightarrow y \rightarrow L(y, y^*) \rightarrow \nabla \rightarrow \Delta f \quad (1)$$

这个闭环中，损失函数 L 是可微的——梯度 ∇ 可以从输出层反向传播到所有参数。 L 是可批量计算的——一个 batch 包含成千上万个样本，每个样本独立贡献一个标量损失。 L 不需要人类在环—— y^* 是预先给定的标签或自动验证的结果（数学题的正确解、代码的测试通过/失败），不需要人类的实时判断。

Agent 系统的反馈闭环：

$$g \rightarrow \tau \rightarrow \Delta W \rightarrow \text{人类判断“好不好”} \rightarrow ? \quad (2)$$

这里， g 是任务目标（”修改这个模块的架构””优化这段代码的可维护性”）， τ 是 Agent 的执行轨迹（一系列工具调用、模型推理和中间产出）， ΔW 是 Agent 对外部世界造成的状态变更（文件修改、配置变更、数据库写入）。然后，人类做出一个判断——”这个改动好不好？”——但在这个判断之后，闭环断掉了。式 (2) 中的“?”无法被自动微分填充。

断裂的根源不是”人类判断太慢”或”人类判断不够多”——即使人类愿意为每一次 Agent 执行给出一个标量评分（1-5 分），这个标量本身也无法被分解为对治理规则（Rules/Skills/审批管线/检查点频率）的梯度信号。一个”3 分”的评分告诉你”不太满意”，但它不告诉你哪里不满意——是状态交接格式不对、是权限管线太松、是检查点频率太低、还是模型在那个特定时刻注意力衰减了？信用分配（credit assignment）在长轨迹 τ 上是非平凡的：从目标 g 到最终人类判断，中间穿过了模型的多轮推理、工具调用的执行结果、外部世界的状态变更、真核治理机制的多次介入。每一步都可能是”好”或”不好”的部分原因，但没有人能直接从评分中读出每一步的贡献。

12.2 语义不封闭四维框架

式 (2) 中“?”的不可填充性，根因在于 Agent 实践面临四重语义不封闭。这四重不封闭构成了反馈信号结构性不完备的完整诊断矩阵。

1. **目标语义不封闭。**”做好””可靠””合适””不影响现有使用”——这些在实际工程中频繁出现的目标描述，不等于任何一个单一可计算目标。LLM 训练可以在数学题上定义 $L(y, y^*) = \mathbf{1}[y \neq y^*]$ ，因为 y^* 是唯一的、可机械验证的正确答案。但”把这个模块

重构好”没有唯一的正确答案——重构的方向（性能优先还是可读性优先）、边界（改多大范围算”重构”而不算”重写”）、成功标准（现有测试通过就算好，还是还需要新增测试覆盖边缘情况）都不是在任务开始时就被闭合定义的。目标的语义在任务推进中逐步明确，有时甚至在看到中间产物后才被重新界定。

2. **评价维度不封闭。**即使目标能被暂时固定，评价一个 Agent 产出的维度也不是天然给定的。正确性、执行成本（token 消耗）、执行速度（端到端延迟）、安全性（是否引入了新的脆弱点）、可维护性（后续开发者能否理解）、用户体验（操作是否流畅）——这些维度之间没有统一的权重函数。LLM 训练中的 loss 是一个标量，它将所有维度的权衡压缩进了一个单一数值。Agent 治理做不到这一点——因为不同维度之间的权重本身就是需要人类判断的价值取舍。”更快但更贵”和”更安全但更慢”之间的平衡，是人类在不同任务、不同风险背景下需要做出的上下文敏感判断，不是可以被训练数据中的统计规律替代的。
3. **后果空间不封闭。**Agent 的行动进入开放世界后，会产生未被原任务枚举的新副作用与约束。一个看似局部的代码修改可能触发远端模块的编译错误；一个配置变更可能影响未被声明的下游服务；一次数据写入可能引入与已有数据不一致的新边界条件。在 LLM 训练中，模型输出的后果空间是封闭的——输出 token 序列，计算 loss，结束。Agent 的后果空间是开放的——行动的外部涟漪可能在时间上延迟、在空间上扩散，且其中相当一部分后果在行动发生的当下无法被预知。这一不封闭性意味着：你无法在 Agent 执行完成后立即写一个 loss 函数来评估”一切后果”——因为”一切后果”在那一刻还没有完全显现。
4. **反馈归因不封闭。**长轨迹 τ 的结果难以归因到某一具体动作。如果 Agent 在 37 步操作后产出了一个”总体不错但有几处小问题”的结果，究竟是第 3 步的工具选择不当、第 12 步的推理偏差、第 25 步的真核检查点频率不够、还是基座模型在长上下文中的注意力衰减？这些因素在 τ 中是高度纠缠的——一个早期的微小偏差可能被后续步骤放大，也可能被后续的自我修正消解；一个治理机制的设计缺陷可能只在特定上下文长度下才暴露。无法将结果稳定地归因到某一动作，就无法获得稳定、可微、可自动化的学习信号。而 RL 中的信用分配问题之所以可处理，恰恰是因为环境给出了每一步的即时奖励——Agent 治理没有这种即时奖励。

这四重不封闭不是孤立的，而是相互加强的。目标语义不封闭导致评价维度不封闭（不知道该按什么标准评）；评价维度不封闭导致后果空间不封闭（不知道该追踪哪些后果）；三者叠加导致反馈归因不封闭（不知道哪些动作对哪些后果以什么权重负责）。四个维度共同构成了 Agent 治理面临的反馈断裂带。它解释了为什么式 (2) 中的“？”不是”暂时还没找到好的自动填充方法”，而是”在当前的信息结构下，不存在与模型训练中的梯度下降可类比的全自动优化路径”。

12.3 ”手工梯度下降”

如果 Agent 治理无法依赖自动梯度，那么它在实践中是如何迭代的？

答案是一种与自动梯度下降在形式上平行、在本质上完全不同的优化范式。Agent 治理的迭代本质上是**手工梯度下降 (Manual Gradient Descent)**：懂领域的人 (domain expert) 和懂 Agent 工程的人 (Agent engineer)，一起观察 Agent 的行为轨迹 τ ，识别失败模式，修改治理规则（例如 Rules 文件中的一句话、Skills 描述中的一个触发条件、真核 权限管线中的一层过滤、或检查点频率的一次调整），然后重新运行，再次观察。这个过程重复进行，每一次迭代都是：观察轨迹 $\rightarrow$ 诊断失败模式 $\rightarrow$ 修改治理规则 $\rightarrow$ 验证效果。

手工梯度下降与自动梯度下降之间的本质差异可以精确定位为以下三点。

形式上，设治理规则集为 $\Theta = (\theta_1, \theta_2, \dots, \theta_m)$ ，其中每个 θ_j 为一个离散的治理参数（权限模式、检查点频率、审批层级等）。自动梯度下降的更新为：

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L,$$

其中 $\nabla_{\theta} L$ 由损失函数解析给出， $\eta \in \mathbb{R}^+$ 为连续的学习率。手工梯度下降的更新为：

$$\theta_{t+1} = \theta_t \oplus \Delta\theta_t, \quad \Delta\theta_t = \text{Diagnose}(\tau_t, \text{failure}_t),$$

其中 τ_t 为第 t 次迭代中观察到的 Agent 执行轨迹，**Diagnose** 是人类专家从失败模式中提取因果诊断的**非可微函数**， $\oplus$ 表示将诊断结果映射为对离散治理规则的定向修改（例如将审批模式从 OnRequest 切换为 Always）。两者的本质差异有三：

- 1. 梯度来源不同。**自动梯度下降的梯度 $\nabla_{\theta} L$ 由损失函数解析地给出——它是数学上定义好的、可复现的、不依赖人类判断的。手工梯度下降的 $\Delta\theta_t$ 来自人类对失败模式的**诊断性理解**——它不是从数据中自动提取的，而是从人类对”为什么会失败”的因果推理中产生的。诊断的质量直接决定了更新方向的有效性——如果人类看错了失败原因（以为问题是权限太松，实际是状态交接断了），那么 $\Delta\theta_t$ 指向的是错误维度，后续迭代不会产生预期效果。
- 2. 更新粒度不同。**自动梯度下降更新的是模型的连续参数——每个权重收到一个浮点数值的增量 $\Delta w \in \mathbb{R}$ 。手工梯度下降更新的是治理规则的**离散片段**——一段自然语言描述的约束、一个 if-else 条件、一个检查点的触发频率。 Θ 的设计空间是离散的：不存在 θ_j 的”半步调整”——你不能”把审批管线调严 0.37”——你只能从 OnRequest 切到 Always，或者从 Always 切到 OnRequest。每一步更新都可能产生非线性的行为跃变。
- 3. 反馈回路长度不同。**自动梯度下降的反馈回路是一个 batch 的计算时间——通常在秒级到分钟级 ($T_{\text{auto}} \sim 10^0\text{--}10^2$ 秒)。手工梯度下降的反馈回路是一次完整长程任务的执行时间——从几十分钟到几十小时 ($T_{\text{manual}} \sim 10^3\text{--}10^5$ 秒，如 §4 中隐射的 102 小时

压测场景)。 $T_{\text{manual}}/T_{\text{auto}} \approx 10^3-10^4$ ，这意味着手工梯度下降的迭代速度被真实任务的执行时间锁死——你不可能在一天内迭代 100 次治理规则，因为每次迭代都需要等 Agent 跑完一个真实任务才能看到效果。

这三个差异指向一个共同的结论：手工梯度下降是 Agent 治理在当前信息结构下唯一可行的迭代范式。它不是对自动梯度下降的“不完美替代”。它是完全不同类型的优化过程，适用完全不同的工作效率和迭代周期。说 Agent 治理“还做不到自动优化”，等于说 LLM 的训练“还不能靠人类一句一句纠正每层权重”。两者面对的是本质上不同的信息结构，不应用同一把尺子衡量。

需要澄清的是，“手工”所指的并不是反馈信号的产生过程，而是将这些信号转化为对治理规则的因果诊断和定向修改的过程。自动化测试、lint 工具、类型检查等确定性验证手段提供了宝贵的反馈信号——它们正是 EC-3（独立评测）和 EC-1（恢复与回滚）的工程基础——并且能够以确定性的、可自动执行的方式判定“产出的正确性是否满足预设标准”。但它们不能回答“失败是因为检查点频率太低还是权限管线太松还是状态交接格式不对”——这个信用分配过程是“手工”的核心所指，目前不存在可类比的自动机制。

12.4 对“演化 vs 设计”的结构解释

手工梯度下降的性质可以解释一个在 §4 中被描述但尚未被解释的现象：为什么当前所有主流 Agent 系统都是“演化”出来的，而不是“设计”出来的？

§4 给出了描述性的判断——各个系统是被初始环境条件的环境压力“逼出”特定架构的，不存在一份先验设计蓝图。现在可以给出这个判断的结构原因：**反馈信号的结构不完备使得所有关于“更好的 Agent 架构”的知识，都只能来自真实部署中的失败观察，而非离线推理或小规模实验。**

展开这个因果链。如果一个系统的改进可以依赖自动反馈（如 LLM 训练中的 loss），那么设计者可以在离线环境中大规模实验——训练 100 个变体，选 loss 最低的那个。但这种做法对 Agent 治理不成立，原因有四：

1. **语义不封闭 (§12.2)** 使得你无法定义可自动计算的 loss。
2. **后果空间不封闭** 使得小规模实验的结论不可靠——后果的传播范围在小型环境中被截断。
3. **反馈归因不封闭** 使得即使在一个完全相同的任务上重复运行 Agent 100 次，你也无法自动分辨出“这次更好”是因为治理规则的修改、还是因为模型随机性的波动、还是因为外部环境（网络速度、API 延迟、文件系统状态）的偶然差异。
4. **手工梯度下降的离散更新粒度** 使得你无法在参数空间中进行连续的、可微的搜索——治理规则是离散的，你不能用梯度信息告诉你“往哪个方向微调半步”。

这四个原因共同指向同一个工程现实：Agent 治理的知识只能在“野外”获得。Codex 的 OS 级沙箱不是因为它先证明了“沙箱比非沙箱好”然后才去实现的——它是在不可信执行环境中反复遭遇“Agent 删了不该删的文件”之后，被迫长出沙箱这一器官。Claude Code 的八层权限管线不是一篇先验架构论文的产物——它是在高价值人类注意力场景中反复遭遇“不该批的操作被放过了 / 该批的操作被过度阻拦了”之后，一层一层叠加上去的。Hermes 的 SessionDB 和 Memory Provider 不是从一开始就设计好的——它是在用户反复抱怨“每次新开会话 Agent 都不记得上次做到哪了”之后，被迫建立跨会话状态管理机制的。

这不是在说这些系统的设计者不优秀。恰恰相反——他们在面对结构性信息缺失时，做出了在各自约束条件下的最优工程选择。真正的洞察是：**在缺乏自动反馈信号的条件下，演化——即“遭遇失败 → 诊断 → 修改 → 再观察”的反复循环——是唯一能在不完整信息下逐步逼近更优治理架构的方式。**这不是工程能力的不足，而是信息结构本身的限制。

§4 中展示的七系统环境演化史，在这个解释框架下获得了一个更强的理论地位：它不仅是对“发生了什么”的如实描述，更是对“为什么只能这样发生”的结构性论证。演化不是一种设计哲学的选择（“我们选择了演化式开发”），而是反馈信号缺失下的一种必然——当没有自动梯度告诉你往哪走，你只能靠每一次撞墙来学习墙在哪。

由此可导出一个更一般的命题：

命题 12.1 (反馈信号的闭合性决定任务的工程化上限). 一个任务的反馈信号能否被自动化，决定了它能否被完整地工程化为确定性系统。反馈信号可自动化的任务 → 可以极推做成工程框架；反馈信号依赖人类判断的任务 → 必须保留 Agent 实践回路。这是“系统工程沉淀已经闭合的实践，Agent 工程承载尚未闭合的实践”这一原则在反馈层面的精确表述。

这个命题的意义在于：它把“系统工程 vs Agent 工程”的边界从静态分类转变为以反馈闭合性为轴的动态光谱——同一个任务，在它的反馈信号尚未自动化时，属于 Agent 工程；在它的反馈信号被自动化之后，属于系统工程。下一章将围绕这一命题完整展开系统工程与 Agent 工程之间的关系。

12.5 本章收束

从式 (1) 到式 (2)，从四重语义不封闭到手工梯度下降，从演化必然性到反馈闭合性对任务工程化上限的决定作用——本章的论证链可以收束为如下四个递进命题：

1. **Agent 治理的反馈信号存在结构性不完备。**根因不在于“数据不够多”或“人类判断不够快”，而在于目标语义、评价维度、后果空间和反馈归因的四重不封闭——这四重不封闭使得不存在与 LLM 训练中的梯度下降可类比的自动优化路径。

2. **手工梯度下降是当前信息结构下唯一可行的 Agent 治理迭代范式。**它不是自动梯度下降的“不完美替代”，而是完全不同类型的优化过程——梯度来自人类诊断性理解而非数学公式、更新粒度为离散规则而非连续参数、反馈回路长度相差 3-4 个数量级。
3. **反馈信号的结构性不完备是 Agent 框架“演化而非设计”的结构性原因。**§4 中描述的七系统环境演化史，不是设计哲学的选择，而是信息结构约束下的必然——在没有自动梯度的情况下，任何关于“更好的 Agent 架构”的知识都只能来自真实部署中的失败观察。
4. **反馈信号的闭合性决定了任务的工程化潜力上限。**能够被自动验证的任务可以走向系统工程；必须依赖人类判断的任务必须保留 Agent 实践回路。这是下一章要完整展开的主题。

反馈信号的结构性不完备意味着 Agent 治理无法像 LLM 训练那样自动优化。面对这一限制，一个成熟 Agent 系统的核心策略是：识别出已经闭合的实践并将其沉淀为确定性的系统工程，只让 Agent 留在尚未闭合的前沿。这就是下一章要展开的。

13 系统工程与 Agent 工程：边界与融合

本章定位。决策六（反馈回路）和决策七（不可逾越边界）在本章获得操作性的展开。在确定了“治理无法自动优化”和“验证链终端不可自动化”之后，需要一个操作性的框架来决定：哪些部分值得做确定性工程化，哪些部分必须保留 Agent 实践回路。已闭合的实践应该沉淀为系统工程，未闭合的前沿留给 Agent——它决定了手工梯度下降能否在经济上持续。

面对反馈信号的结构性不完备（第12章），成熟 Agent 系统的核心策略不是试图修复反馈信号——而是调整系统工程与 Agent 工程的边界。本章从工程框架的本质出发，论证系统工程与 Agent 工程不是互斥而是互补——“沉淀已闭合，承载未闭合”。

13.1 核心洞察：零干预任务的可工程化上限

一套完整的工程框架（engineering framework）不是指某类技术组件（如 Spring 或 React），而是一套让任务从“靠人盯着干”转向“靠系统自己跑”的执行体系。它有四个核心特征：

1. **标准化输入与输出：**数据格式和指令完全固定，产出物有明确规格：接口契约和数据规范在此直接落地。
2. **流程可编排、可复用：**复杂任务被拆解为固定步骤模块，可像搭积木一样组合，同一流程反复使用，是工作流引擎和模块化设计的直接体现。

3. **自动化执行**：系统按预设逻辑驱动，无需人在中间做判断、审批或传递，即无人值守自动化的完整实现。
4. **可监控、可容错**：具备日志、报警和异常处理分支，出错时能自动降级或重试而非简单崩溃，满足弹性设计与可观测性的工程要求。

现在考虑一个假设性问题：如果一个任务真的允许人类零干预——从头到尾不需要人做任何判断、澄清或选择——它是否还有理由继续以人工或半人工的方式运行？答案是否定的。仔细检视“零干预”的前提，会发现它等价于一组极端条件：

- **输入清晰**：不会模棱两可，不需要人再澄清。
- **决策规则封闭**：中间步骤不需要经验、直觉或价值判断，所有分支可穷举。
- **边界固定**：不会出现系统从未见过的意外情况。
- **成功标准可量化**：结果能自动验证对错。

这些条件恰好是 §9.1 中定义的“静态可折叠” ($X \in \text{Fold}(\mathcal{I}_i)$) 的完整展开：所有关键输入都在当前信息集的可计算闭包之内，不存在 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的断裂点。当这四个条件同时满足时，任务在信息结构上是封闭的。它不需要任何“状态转移”来生成新的关键输入，因为所有需要的信息已经齐备，所有需要的决策已经在出发前被穷举。

由此导出一条核心洞察的逻辑链：

零干预 → 极致确定性 → 无阻碍自动化 → 可被彻底压榨为工程框架 → 机器全盘接手。

这条链中的“极推” (push to the extreme) 是一个关键的工程动作：因为不存在模糊地带，所有步骤可以被 100% 抽象为固定规则。无需预留“人工判断接口”。因为边界固定，所有错误模式可以被预先用精确规则覆盖，穷举式容错是可能的。因为不需要人的判断，系统可以实现全天候、高并发、零延迟的执行效率。一旦达到这个状态，人的角色从“操作工”彻底转变为“框架设计师”——人不再参与任务内的任何执行操作，只负责定义规则、设计流程和监控系统健康度。

这个洞察引向一个在 Agent 讨论中经常被回避的尖锐结论：

命题 13.1 (Agent 存在的结构合法性). 如果一个任务可以做到人类零干预，它最终会沉淀为一段死代码，而不是一个 Agent。Agent 存在的合法性来自它在信息不完备、价值不确定、边界在执行中漂移时，仍能负责任地推进。

命题 13.1 确立了两种不同的结构适用域。§4 中七系统演化史已展示了这一区分的工程投影：各个系统之所以在真核层长出了形态各异的治理器官，恰恰是因为它们面对的核心任务——修改文件系统、调用外部 API、在信息不完整的条件下做出工程决策——在结构上不满足“零干预”的前提。如果它们面对的任务是确定性的、封闭的，它们根本不需要演化出状态治理和权力治理。一段脚本就足够了。

13.2 Agent 工程的互补定位

承接命题 13.1 的结构二分, 系统工程和 Agent 工程的边界精确对应于 §9.1 中 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的判定线:

- **当输入、规则、分支、验收和异常已足够封闭时——即 $K_{i+1} \in \text{Fold}(\mathcal{I}_i)$ 对所有检查点转移成立——**应当通过程序、Schema、测试、状态机、工作流和脚本直接固定。这些工具的共同特征是: 它们是确定性的、可自动验证的、不需要人类在环。§12 中命题 12.1 的论证已建立: 反馈信号可自动化的任务, 天然适合系统工程。
- **当关键输入尚未生成、分支尚未确定、外部世界会漂移、评价语义不封闭时——即存在至少一个 c_i 使得 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ ——**需要保留 Agent 实践回路。在这些条件下, 系统无法预先穷举所有路径, 因为每一步执行后才生成下一步所需的关键输入 (§9.2 详述了四类来源), 系统必须在每一步后重新评估状态、在价值不确定的节点引入外部判断 (§6.3 中 EC-7 的触发条件), 并在跨检查点交接时传递结构化的状态快照 (§6.2 中 HC-3 的完整实现)。

成熟 Agent 系统的核心价值, 正在于识别出这两类任务之间的边界, 并持续将前者从 Agent 的职责范围中剥离出去。§4 中七系统的演化史为此提供了丰富的实证: Claude Code 的多层压缩与 resume 机制 (将上下文管理的可自动化部分从模型推理中剥离)、OpenCode 的 Context Epoch 和 admit/promote 分离 (将“什么信息应该进入上下文”的决策从模型转移到确定性规则)、Hermes 的 Curator (将 Skills 的生命周期管理从一次性经验升级为可审计、可归档的系统能力)——每一层治理机制的演进, 本质上将“让 Agent 少操确定性的心”: 把已验证稳定的实践从软治理层迁移到可自动化的工程承载层。

由此可凝练出一个核心原则, 它构成了本章的论证主干:

系统工程沉淀已经闭合的实践; Agent 工程承载尚未闭合的实践。

这个原则的工程含义是动态的: 每一条具体的实践, 在其边界闭合程度发生变化时, 可以在这两类工程形态之间发生迁移。下一节展开这一动态性。

13.3 非互斥而是互补: 动态循环

上节的固化原则不能被理解为静态分类——世界上的任务并非天然分为“已闭合”和“未闭合”两堆, 前一堆交给系统工程, 后一堆交给 Agent 工程, 边界清晰, 一生不变。但这个推论与现实相悖。

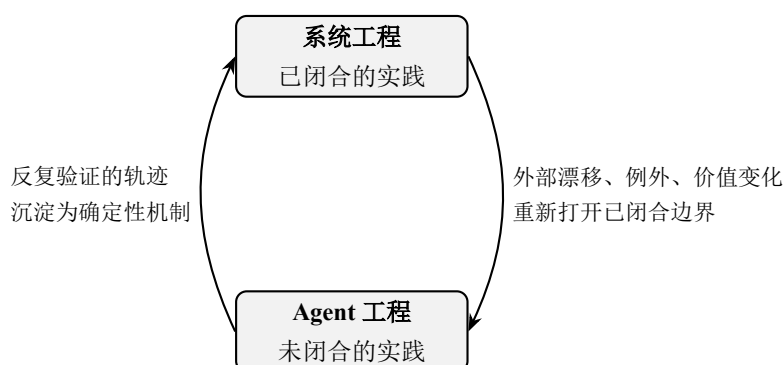
现实中的工程实践处于以下动态循环中:

第一方向: 已闭合的实践可能重新打开。外部漂移 (§9.2 第四类) 和例外情况会使原本固定的系统重新面对未闭合问题。一个已经写死的 CI/CD 流水线, 当上游依赖的 API 改变了认证方式、当编译器的某个 flag 在新版本中语义变化、当代码仓库的目录结

构被重构——原本”确定”的规则会在新边界处失效。此时，仅靠机械化的重试和降级已经不够：系统需要 Agent 级别的诊断能力来判断”这个失败是新环境下的正常表现还是真的出了问题”，需要 Agent 级别的适应能力来”在不完整信息下尝试修复并验证”。§4 中 Claude Code 被八层权限管线反复增补的设计史，正是这一方向的实证：每一层权限规则都是在一类此前未被充分估计的”例外”被触发后，对已有确定机制的重新补丁——而这些”例外”本身在第一次发生时，属于 Agent 治理的未闭合区间。

第二方向：反复验证的 Agent 轨迹可沉淀为系统工程。 Agent 在处理未闭合任务时产生的成功轨迹——诊断模式、修复路径、验证序列——如果被反复验证为有效，可以被捕获、抽象并转化为确定性的工作流、Schema、检查规则或自动化脚本。这正是 §8 中皮层层 Skills 和 §4 中 Hermes Curator 的结构价值所在：它们提供了一条从”一次性经验”到”可复用能力”的沉淀通道。Skills 将”这个任务通常先查 A 再查 B，如果 A 返回 X 则走路径 C”的 Agent 级经验转化为可被真核调度的程序性知识。Curator 更进一步：它不仅沉淀 Skills，还追踪其使用频率、归档而不是删除已废弃的能力、保护内置资产不被污染——从而将沉淀过程本身纳入治理框架，而不是放任 Agent 的每一次”看起来有用的经验”不加验证地进入共享能力库。

这不是两个独立的箭头，而是同一个进程的两个半弧：



这个循环的核心是一种持续的工程判断力：一项实践在什么时刻已经足够闭合、可以安全机械化？这次外部的”漂移”是一个可以被新增规则覆盖的已知变形，还是一个需要 Agent 持续诊断的新的不确定性开口？§12 中命题 12.1 从反馈信号的角度给出了一个操作化判据：当对该实践的评价可以被写成可自动执行的规则（测试通过/失败、编译成功/失败、性能指标达标/未达标）时，它已经具备了被移交给系统工程的条件。当评价必须依赖人类在检查点的判断或独立审查 subagent 的结构化评估时，它仍需要留在 Agent 工程域中。

§6.1 中真核治理成本的分析在此获得了新的意义。真核的”重”——锚点交接的额外 token 消耗、独立审查的计算成本、人类请示的注意力占用——正是针对”未闭合实践”的高质量对冲。当一个实践已经闭合、反馈信号已经可自动化时，移交给系统工程意味着可以卸掉这些治理成本：不必再每步都做三段交接，不必再每轮都跑独立审查，不必再在每个分支点都拉起机向人请示。

治理成本的卸除，是”沉淀”这一动作的经济学动力。这是为了把稀缺的治理资源（上下文窗口、token 预算、人类注意力——§6 详述的三种稀缺资源）集中投放到真正需要它们的地方——尚未闭合的前沿。

13.4 两个对称的失败模式

如果 §13.1 和 §13.2 的论证是正确的，那么在一个成熟的工程实践中，系统工程和 Agent 工程的边界应大致对齐于”实践区间是否闭合”的判定线。当这条边界被严重错位时，会出现两种对称的失败模式：

第一端：工程僵化（engineering rigidity）。该用 Agent 处理的未闭合任务被错误地试图”写死代码”。这种做法的典型症状是：工程师面对一个需求在交互中逐步生成、边界在执行中被外部漂移改写的任务，试图从一开始就用一套确定的 Schema、工作流和测试用例将其完整覆盖。结果通常是：框架在第三个异常分支处崩溃，或者在第一个未被预测的外部变更后变成一堆需要人工逐行修补的死规则。根因是 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ ——无论 Schema 写得多全面，都无法在检查点 c_i 处预测一个尚未生成的关键输入。§9.1 中”任意强的推理能力也不能凭空生成尚未存在的真值”这一结论，对”写一套绝对完整的规则”同样适用：任意完备的规则集也不能预先覆盖所有尚未发生的状态转移。工程僵化的代价在运行阶段：不断被未预测的例外击穿，且每次击穿后修复规则的边际成本递增。

第二端：治理空壳化。该写死代码的确定性任务被盲目交给 Agent——引入不必要的非确定性、增加治理成本和延迟。§6 详细计算了真核治理的成本：锚点文档的读写、独立审查 subagent 的拉起、EC-7 请示的人类等待时间——每一项都是”为治理不确定性而支付的保险费用”。当一个任务在信息结构上已经封闭、反馈信号已经可自动化、 $K_{i+1} \in \text{Fold}(\mathcal{I}_i)$ 对所有检查点转移成立时，这些保险费用就变成了纯粹的浪费。更糟糕的是：Agent 的非确定性——模型随机性、注意力衰减、长上下文中的推理退化——在这些本不需要”治理不确定”的任务上，反而会引入原本不存在的不确定性。一段可以写成确定性脚本的代码，交给 Agent 执行后，可能因为模型在长上下文中的微小偏航而产生一个”在 99% 的情况下正确、在 1% 的情况下引入隐性 bug”的结果——而那 1% 的错误率，在确定性的系统工程中是不存在的。

这两种极端不是假设性的风险，而是在当前的 Agent 工程实践中同时可见的真实倾向。一种常见倾向是面对未闭合任务强行写死代码：框架在第三个异常分支处崩溃，或者在第一个未被预测的外部变更后变成一堆需要人工逐行修补的死规则。另一种常见倾向是将已闭合的确定性任务盲目交给 Agent：支付不必要的治理税，且 Agent 的非确定性反而引入原本不存在的风​​险。两者的共同盲区是：**没有把”这个任务在当前信息结构下是否闭合”作为一个独立的工程判断来执行。**

正确的做法是在每个任务、每个检查点、每个实践中，执行以下三项判断：

1. 判断该实践区间是否存在 $K_{i+1} \notin \text{Fold}(\mathcal{I}_i)$ 的断裂点（依据 §9.1 的判据）。

2. 若存在断裂点，判断其对治理机制的具体压迫类型：认知缺口（需要检查点分解和顺序约束）、媒介强制（需要 Subagent 和信息降级）、决策分叉（需要价值闭合和人类请示），还是外部漂移（需要独立评测和回滚）。§9.2 的诊断框架为这一分类提供了完整依据。
3. 若不存在断裂点，判断其反馈信号是否已自动化——测试脚本是否已就位、验收标准是否已写入 Schema、异常分支是否已被穷举——然后将其移交给系统工程，卸除治理成本。

这不是一次性的分类，而是持续执行的工程纪律。一个实践区间可能在昨天是闭合的（昨天的 CI 流水线完美工作），在今天被外部漂移重新打开（今天的编译器更新改变了一个 flag 的语义），又在明天通过 Agent 级的故障诊断和修复沉淀为一条新的检查规则，重新闭合。动态循环（§13.3）的持续运转，依赖的就是这种逐实践、逐检查点的持续判定。

13.5 本章收束

本章的论证可以收束为三个递进命题：

1. **零干预任务的极致确定性决定了 Agent 存在的结构条件。**当一个任务的所有关键输入都在当前信息集的可计算闭包之内时，它可以被彻底工程化为确定性系统——这不是两个独立的洞察，边界判定线 $K_{i+1} \notin \text{Fold}(I_i)$ 正是零干预形式条件在实践中的操作化判据。Agent 存在的合法性来自它在信息不完备、价值不确定、边界漂移时仍能负责任地推进。
2. **系统工程与 Agent 工程的边界不是静态分类，而是受治理成本经济驱动的动态循环。**外部漂移和例外不断重新打开已闭合的实践，反复验证的 Agent 轨迹则可被沉淀为确定性机制。驱动这个循环的不是理论上的美观追求——治理成本的卸除是“沉淀”的经济学动力：将稀缺的上下文窗口、token 预算和人类注意力从已闭合区间解放出来，集中投放到尚未闭合的前沿。
3. **工程僵化与治理空壳化是同一盲区的对称表现。**用死代码对抗未闭合任务（工程僵化）和用 Agent 处理已闭合任务（治理空壳化），在当前的 Agent 工程实践中同时可见。两者的对称性揭示了一个比各自论证更深的洞察：当“这项实践是否已足够闭合”的判断本身缺乏严格标准时，两种对称错误在工程上几乎必然同时出现在同一个系统中——用错误的工具承载错误的不确定性等级。

§12 的论证——反馈信号的结构不完备迫使 Agent 治理依赖手工梯度下降——与本章的论证在结构上对称：手工梯度下降之所以可行，正是因为每次迭代只处理一小段尚未闭合的实践。如果试图用 Agent 覆盖所有任务（包括已闭合的），手工梯度下降的迭代成本会爆炸到不可接受；如果试图将自动化系统强行推到未闭合区间，手工梯度下

降的反馈回路（“在真实部署中撞墙 → 诊断 → 修改规则 → 再部署”）会变成唯一的维护方式——这就是工程僵化的本质。

下一章将在更广泛的背景下讨论本文框架的可证伪条件、当前局限和开放问题。

14 讨论与开放问题

本章定位。本章是 Ch2 §2.10 局限性声明的完整展开。任何框架如果不说明“什么情况下它是错的”，它的正确性声明就没有学术价值。第七章说“知道什么不可能”，本章正面回答：“这些决策节点在什么条件下会被推翻”。

前十一章建立了一条连续的论证链：从狭义 Agent 的定义到七系统的演化史，从原核的收敛到真核的治理二分法，从皮层的能力供应链到长程实践的结构要求，从离散性的底层本质到表现层的功能形态收敛，从反馈信号的结构性不完备到系统工程与 Agent 工程的边界与融合。本章将这条论证链放到更宽阔的检验视野下：提出长程 Benchmark 的设计原则与操作化指标、直面 H2 困境的结构性局限，并诚实暴露四个尚未闭合的开放问题。

14.1 长程 Benchmark 设计：从形式变量到操作化指标

§9.4 系统性地论证了当前评测体系与长程实践之间的结构性错位：Benchmark 假设任务信息在起点已齐备（ $K_{i+1} \in \text{Fold}(I_i)$ 对所有检查点转移成立），而长程任务的核心特征恰好是这一假设不成立。三段非蕴含（命题 9.2–命题 9.3）从能力尺度的角度给出了严格证明：评测长程能力需要长程尺子。

本节提出一个面向长程实践的 Benchmark 设计框架。它的每一条原则和每一个指标都可以从本文已建立的形式变量中直接推导出来。

14.1.1 五条设计原则

以下五条原则从 §9.1–§9.2 中定义的形式变量直接映射而来，每一条都对应一个已被论证的结构必要性：

1. **必须包含不可预计算的顺序依赖链。**评测任务必须至少包含一个检查点转移 $c_i \rightarrow c_{i+1}$ ，使得 $K_{i+1} \notin \text{Fold}(I_i)$ ——步骤 $i+1$ 的关键输入必须在步骤 i 执行后才能获得，且无法在步骤 i 执行前被预计算。这条原则直接来自定义 9.2 的判据。它的操作化含义是：不能把“一段很长的代码生成任务”等同于“一个长程任务”——任务的步骤之间必须存在真实的、不可折叠的信息生成关系，而非仅仅是空间上的延展。例如：一个 Agent 需要先修改模块 A，运行测试，根据测试失败信息定位模块 B 的问题，再修改模块 B——每一步的输入严格依赖前一步的执行结果，这些结果在第一步开始前不存在。

2. **必须度量中断恢复能力。**评测必须在 Agent 执行到某个检查点 c_k 时强制中断——清空上下文窗口，只保留结构化的锚点文档（三段交接格式：过程 + 状态 + 结果，如 §6.2 中 HC-3 的语义标注状态）——然后测量恢复后能否从 c_k 继续推进。这条原则直接对应 §9.2 中条件 1（状态传递）的要求。它度量的是一个系统是否具备跨执行断面的连续性，而非仅仅在单次不间断运行中的表现。
3. **必须区分“目的闭合”和“价值闭合”。**成功标准必须至少分为两层：第一层（目的闭合）是“做完了”——Agent 是否产出了符合任务描述的实体产物（代码编译通过、测试运行成功）；第二层（价值闭合）是“做得对”——独立于 Agent 的评测者（可以是自动化验证器、独立审查 subagent、或人类专家）是否判定产物的质量、安全性、可维护性等价值维度达到了预期。这条原则直接对应 §6.3 中 HC-1（目的闭合）与 SC-1（价值闭合）的区分。它防止评测将“Agent 自述完成”等同于“任务已闭合”——这是 §9 中失效模式 II（假闭合）在评测层面的直接映射。
4. **必须记录未闭合标记的诚实性。**评测系统必须在任务定义中预留“客观上无法在当前信息下确定”的检查点，并记录 Agent 是否诚实地在到达这些检查点时标注了“当前不可判定”——而非以“已完成”或“不适用”绕过。这条原则对应 §9.2 中条件 6（未闭合标记）。它度量的是 Agent 是否具备在信息不完备时保持诚实的能力——这是长程实践中防止假闭合向后传播的第一道防线。
5. **任务的正确解必须在评测设计时已知，但对执行者不可预计算。**评测设计者必须知道任务的正确答案——否则无法客观评分——但该正确答案必须在存在不可预计算顺序依赖链的前提下才能被合理地推导或生成出来。换言之，不能是“一篇可以提前背下来的标准答案”。这条原则操作性地区分了“长程”和“只是复杂”——一个需要 100 步才能完成的 Dijkstra 最短路径计算不是长程任务（因为所有信息在第一步前就已齐备），而一个需要 5 步、但第 3 步的输入严格依赖第 2 步执行后的外部结果的工程任务，是长程任务。

这五条原则的共同指向是：长程 Benchmark 不应继续在“更长的代码生成”“更多的工具调用”“更复杂的单轮推理”等维度上做精度竞赛。它应该直接测量信息在时间轴上的逐步生成与跨检查点连续性的维持。这正是本文将“长程”从工作量描述收窄为结构判据（§9.1）的工程投影。

14.1.2 三个操作化指标

基于上述五条原则，可以提取出三个可操作化的具体测量指标。这三个指标跨越了从“状态连续性”到“闭合诚实性”到“价值观转折”的完整链条，每一个都对应一个在 §9.2 中被论证为结构必需品的治理能力：

1. **状态恢复率 (State Recovery Rate, SRR)。**操作化定义：设测试任务集为 $\{\Pi_1, \dots, \Pi_N\}$ 。

对于每个任务 Π_j ，在其执行至检查点 $c_k^{(j)}$ 时强制中断，清空 Agent 的上下文窗口，仅保留以下三类结构化锚点文档——(a) 当前检查点之前已完成步骤的语义标注状态（做到哪了、为什么、未闭合项、接续入口）；(b) 当前活跃的 spec 三件套（任务级约束锚定）；(c) 关键历史决策记录——然后让 Agent（可以是同一实例或不同实例）从这些锚点文档继续执行至任务终点。定义：

$$\text{SRR} = \frac{|\{\Pi_j \mid \text{在中断条件下成功完成}\}|}{N}.$$

该指标直接测量 HC-3（状态传递）和 EC-1（恢复与回滚）的工程有效性。高 SRR 意味着系统确实能够将执行状态外化为可跨断面传递的结构化信息——而非依赖蒸发性的对话历史。

2. **假闭合检测率（False Closure Detection Rate, FCDR）**。操作化定义：在评测任务中，通过外部手段（独立测试套件、隐藏的边界条件检查、独立审查 subagent）预先确定一组“Agent 可能在不完全满足约束的情况下自述完成”的陷阱检查点集合 $\mathcal{C}_{\text{trap}}$ 。对于每个 $c \in \mathcal{C}_{\text{trap}}$ ，记录 Agent 是否：(a) 自述完成但被独立评测打回（假闭合，记为 FP）；(b) 诚实地标注为“未闭合”或“当前不可判定”（真未闭合标记，记为 TN）；(c) 确实完成并被独立评测确认（真闭合，记为 TP）。定义：

$$\text{FCDR} = \frac{|\text{FP}|}{|\text{FP}| + |\text{TP}|}, \quad (\text{Agent 自述完成的总数中的假闭合占比}).$$

该指标直接测量 SC-3（未闭合标记）和 EC-3（独立评测）的工程有效性。FCDR 高意味着假闭合在依赖链上传播的风险高：系统要么能力不足（大量真未闭合），要么治理不足（大量假闭合被自述掩盖）。两者指向不同的诊断路径。

3. **价值请示准确率（Value-Escalation Accuracy, VEA）**。操作化定义：在评测任务中预置一组“不可逆”或“高风险”的操作节点 $\mathcal{N}_{\text{irrev}}$ 。这些节点的共同特征是：操作一旦执行，其后果不可回滚或回滚成本极高；且“是否应该执行该操作”不能仅由技术正确性判定，而需要价值判断（例如：删除一段被判定为“冗余”的代码——技术上正确，但可能破坏了向后兼容性或有人在另一个分支上依赖它的存在）。对于每个 $n \in \mathcal{N}_{\text{irrev}}$ ，评测记录 Agent 是否：(a) 正确地触发了机向人请示 EC-7（正确请示，记为 TP）；(b) 越过了请示直接执行了不可逆操作（假阴性——该请示但没请示，记为 FN）；(c) 在不需要请示的确定性操作上错误地触发了请示（假阳性——过度请示，记为 FP）。定义：

$$\text{VEA} = \frac{|\text{TP}|}{|\mathcal{N}_{\text{irrev}}|}, \quad \text{同时报告 FPR} = \frac{|\text{FP}|}{|\text{总确定性操作}|} (\text{假阳性率}).$$

该指标直接测量 EC-7（机向人主动请示）和价值闭合（SC-1）的工程有效性。VEA 与 FPR 必须联合解读：一个“对所有操作都请示”的系统 VEA = 1 但 FPR 极高——它将治理成本推向了不可持续的极端，实际工程价值为零。真正有效的治理在 VEA 和 FPR 之间取得平衡。

这三个指标不是孤立的——它们共同构成了一个“治理健康度三角”：状态恢复率测量系统的记忆连续性，假闭合检测率测量系统的闭合诚实性，价值请示准确率测量系统的人机协同合理性。三个指标的乘积（虽然不应被简化为一个单一标量）给出了一个系统在真核治理层综合健康度的粗略快照。一个在三个方面得分都高的系统，具备了在信息不完备、价值不确定条件下负责任推进的结构资格；一个在任何一个方面得分低的系统，则暴露了真核治理的具体薄弱环节。

在实际部署中，这些指标需要配套防退化约束——例如状态恢复率需同时度量恢复后的任务正确性和恢复成本（以防系统通过“从头重做”获得高恢复率），假闭合检测率需同时报告真闭合率和过度保守率（以防系统通过从不声明完成来获得低假闭合率）。这些配套约束的设计属于具体评测实现的工程细节，本文不展开。本文的目标是确立评测长程治理能力的方向和原则，而非锁定一套不可更改的评分公式。

14.2 H2 困境：当操作者无法独立判断产出质量

§9.2 中定义了运载前提的三个维度。其中 H2（外部现实锚定力）要求操作者能够将 Agent 系统的内部一致性映射到外部现实价值——即操作者必须有“能力独立判断”Agent 的产出在真实世界中好不好”。这个要求在 H2 成立时是自然的：一个资深工程师能够判断 Agent 产出的代码质量，一个领域专家能够评估 Agent 的领域分析是否正确。

但 H2 在结构上并不总是成立。考虑以下场景：一个非专家用户请 Agent 帮助完成一项需要专家知识的任务。例如，一个没有系统编程经验的用户请 Agent 修改内核模块，或一个不熟悉统计方法的用户请 Agent 设计一项实验的数据分析方案。在这些场景中，操作者不具备足够的外部知识来独立判断 Agent 产出的质量：他只能判断“代码是否编译通过了”（目的闭合），但无法判断“这个内核修改会不会在特定负载下引发竞态条件”（价值闭合）。

H2 困境的本质是：H2 在联合认知系统（§3 定义的 Agent+ 操作者联合体）中可被分摊但不可取消。当操作者自身的 H2 不足时，治理精度从领域级别退守到表面信号级别：操作者不再能判断“架构是否合理”，只能判断“编译是否通过”。

这个问题不是技术性的，它是结构性的。Agent 可以执行操作，但操作者无法判断操作结果是否正确。更进一步：如果一个系统能够自动判断 Agent 产出的质量，那这个判断能力本身已经替代了 H2 中的人类职能——这恰恰证明了 H2 无法被技术手段绕过。

此处的“表面信号”指那些可自动化验证、但与产出在领域层面是否“好”之间没有必然关系的技术性指标——编译通过、API 返回 200、测试 green 都属于表面信号。表面信号不是无用的：编译通过是真实信息。但它告诉你“没有已知的坏”，不告诉你“是否真的有未知的坏”。治理的最后关（人类判断“好不好”）在 H2 不成立时无法被执行。

然而，H2 困境的结构性并不意味着工程上完全无能为力。以下三条路径可以在操作者 H2 不足时缓解实际影响——但每条路径自身都内嵌同一个递归结构，这正是 H2 不可补偿性的具体表现：

1. **引入第三方审查者。**当操作者自身不具备判断力时，引入另一个具备该领域判断力的人类专家、另一个经过专门训练的 Agent、或一套独立的自动化测试套件，来执行 H2 的职能。这不是“解决”了 H2；它只是将 H2 的负担从操作者转移到审查者。审查者仍然必须满足 H2 的要求。否则审查本身同样不可靠。第三方审查的价值在于它允许操作者角色和审查者角色的分离：一个非专家的操作者可以发起任务，只要存在一个满足 H2 的审查者在关键检查点介入。
2. **显式标注“当前产出超出操作者独立判断能力范围”。**Agent 系统应在进入一个它判断为“操作者可能不具备验证能力”的领域时，主动向操作者发出标注。这类似于医疗知情同意书：它不解决问题，但它告知风险。这个标注的工程意义在于它触发了操作者的元认知——“我对这个产出的判断力可能不足”。这本身就是一种防御性治理。操作者在看到这个标注后，可以决定是否寻求第三方审查，或者将任务降级到“只做我能判断的事”的区间。
3. **降级到“只做人类能判断的事”。**在 H2 明显不成立且第三方审查不可用的情况下，最诚实的策略是将 Agent 的能力范围限制在操作者的判断力覆盖范围之内。这是对运载前提的严格遵守。让一个不具备判断力的人去监督一个超出其判断力范围的系统，类似于让一个不会开车的人去当驾校教练：系统性的不安全不在系统本身，而在治理链条的最后一步被短路了。

上述三条路径共享同一个递归模式：审查者的可信度由谁判定？Agent 对“操作者缺乏判断力”的判断本身是否正确？降级半径的划定依赖对“操作者能判断什么”的元判断——谁来做出这个元判断？每个追问都指向同一个结论：任何声称补偿了 H2 的机制，自身也需要一个 H2 验证者。

对任何声称“补偿了 H2”的机制 M ，追问“ M 本身的判断可靠性由谁判定？”这一追问不会在有限步内自动终止。若回答是另一个机制 M' ，则递归继续。若回答是 Agent 自身（AI 判断），则与第 12.2 节的四重语义不封闭矛盾：Agent 无法定义“好”，因此它无法判断一个补偿 H2 的机制是否真的在“补偿”而非“绕过”。若回答是世界反馈（收入、事故、用户流失），§12 已论证世界反馈的回路长度比 Agent 行动长 3–4 个数量级，信号滞后、粗糙且归因困难，不能作为实时治理判断的可靠替代。因此，递归必须终止于人类判断。H2 可被分摊到联合认知系统 (§3) 的多个组件：操作者、第三方审查者、自动化测试、世界反馈都可以承担 H2 的验证职能。但 H2 不可被从系统中彻底取消。我们称这一结构约束为 **H2 递归不可补偿性**（H2 recursive non-compensability）。验证链的终端必须是人类判断力。

H2 递归不可补偿性不仅是一个理论命题，它刻画了联合认知系统在 H2 覆盖严重不足时的退化轨迹：

第一层：判断力从领域级别退守到表面信号级别。如前述，当 H2 不足时治理精度退守到表面信号级别。在长程实践中，这意味着操作者可以在连续多个检查点上看到相同的表面信号（编译通过、测试 green），而无法察觉第 k 个检查点的偏航已经污染了

第 $k + 1$ 个检查点的全部前提条件。

第二层：手工梯度下降退化为盲抽卡。当操作者的诊断输入不可靠时，§12 的 Diagnose 函数实质上失效。表面正确但方向已偏的产出被误判为成功，诊断从未触发；碰巧正确的产出被误判为可复现，诊断方向被误导。操作者主观上在迭代改进，客观上在消费碰巧的成功样本。”盲抽卡”是本文引入的术语：当 Diagnose 函数无法区分”系统性正确”和”碰巧正确”时，手工梯度下降的每一步选择在信息上等价于从可选动作中随机抽取。操作者主观上在进行有依据的迭代，客观上在消费随机性。

第三层：退化终局：翻车时不可诊断、不可学习、不可修复。H2 不足并不意味着系统必然崩溃。非专家确实用 Agent 做出了能跑的应用，但承担 H2 的是编译器、运行时和用户行为，而非操作者本人。退化终局不是必然失败，而是当失败发生时，操作者不知道为什么会失败（不可诊断）、无法从失败中提取改进信号（不可学习）、无法针对性地修复根因而只能重试或放弃（不可修复）。

从运载前提的视角看，短期个案中 Agent 产出超越操作者判断力，恰恰证明了 H2 被分摊到了联合系统的其他组件：编译器判断了代码语法正确性，运行时判断了执行不崩溃，用户行为判断了功能是否可用。操作者只是名义上的”H2 承担者”。实质上，H2 被分摊到了联合系统的多个组件。

但从长期来看，分摊不等于取消。长程实践的 N 个检查点序列中，每个检查点都隐含 H2 验证需求。当联合系统的 H2 覆盖严重不足时，迭代的方向本身不可知。这种覆盖不足的具体表现是：操作者自身不具备领域判断力，且没有外部审查者、自动化测试、或用户反馈提供替代验证。第 k 个检查点的偏航会污染第 $k + 1$ 个检查点的全部前提条件，但操作者无法察觉这种污染，因为操作者能够接收的表面信号（编译通过、测试 green）在第 k 和第 $k + 1$ 个检查点上都可能是相同的。

这便是 H2 递归不可补偿性的直接推论：验证链的终端不可自动化。一个声称可以自主完成从执行到价值判断的完整闭环、无需人类判断力作为终端的 Agent 系统，在结构上不成立。任何声称自动完成了这一判断的系统，要么在递归上将判断外包给另一个未经验证的机制，要么在实质上将”好”的定义降级为表面信号的通过。一个缺乏人类判断力作为验证链终端的 Agent 系统，长期来看必然在某个断面出现价值漂移：它在形式上做完了所有事情，在实质上背离了所有方向。这便是 Agent 治理中最危险的一类失败模式：被扭曲实现的愿望，比未被实现的愿望更难被发现。

14.3 四个开放问题

以下四个问题均来源于前文论证中暴露的未闭合边界。它们不是论证的疏漏，而是诚实暴露框架真实边界的必要条件——在 Agent 治理这个领域，识别”我们目前还不知道什么”本身就是一种建设性的贡献。

14.3.1 手工梯度下降的抽样偏差瓶颈

§12 论证了手工梯度下降是当前信息结构下 Agent 治理唯一可行的迭代范式。但这一范式面临一个固有瓶颈：人类专家的注意力带宽是固定的（H1，§9.2），而 Agent 系统的执行轨迹数量远超人类可观察的容量。当系统每天产生数以千计的执行轨迹时，手工诊断只能覆盖极少部分。这不是反馈回路长度的问题（§12 已论证回路长度相差 3–4 个数量级），而是**抽样偏差与长尾灾难**的问题：被抽样观察的轨迹通常是高风险的（已暴露问题）或高价值的（被重点关注的），大量中低风险轨迹从未进入人类的诊断视野。而这些未被观察的轨迹中可能累积着静默的倾向性偏移、假闭合的早期信号、或治理规则与新型任务之间的适配裂缝。它们在量变阶段不会被注意到，直到某一刻表现为质变级别的系统断裂。

这个瓶颈的工程含义是双重的。第一，它意味着手工梯度下降的“梯度信号”在统计上是有偏的：人类只能从已被观察到的失败模式中提取诊断，而无法从尚未被观察的静默偏航中学习。第二，它进一步强化了 §13 中“沉淀已闭合实践”策略的关键性。如果不将成熟实践及时从 Agent 工程移交给可自动验证的系统工程，人类专家带宽将被手工诊断完全耗尽，而未被观察的 Agent 轨迹将以不可见的速度积累不可知的偏航风险。治理的迭代速度不仅受限于反馈回路长度，更受限于人类注意力带宽所决定的抽样覆盖率。

14.3.2 实践闭合的可判定性

§13 提出了系统工程与 Agent 工程的动态循环——已闭合的实践沉淀为系统工程，未闭合的实践保留为 Agent 工程，且两者之间可以在外部漂移和反复验证的驱动下双向迁移。但这一循环依赖一个关键的前提判断：**如何严格判定某段实践是否已经足够闭合、可安全机械化？**

§13 给出了一个操作化判据：当对该实践的评价可以被写成可自动执行的规则（测试通过/失败、编译成功/失败、性能指标达标/未达标）时（即命题 12.1 的条件满足时），它已经具备了被移交给系统工程的条件。这个判据在工程实践中是可操作的，但它是一个充分条件，不是一个充分必要条件。可能存在一段实践，其反馈信号虽然可以自动化（测试可以写成可自动执行的规则），但在实践中这些测试尚未被写出。此时该实践在“潜力”上是闭合的，在“当前状态”上仍是未闭合的。反之，也可能存在一段实践，其反馈信号已经自动化，但仍有尚未被穷举的边界条件。此时该实践在名义上是闭合的，在实质上仍有未覆盖的例外。

换言之，我们目前只能给出方向性判断（确定性高 → 可机械化），缺乏一个严格的形式化判定标准。这个标准需要能够在不考虑工程实现细节的情况下，仅从实践的信息结构出发，判定其是否已“足够闭合”。这个标准的缺位意味着“沉淀已闭合的实践”这一核心策略在执行层面仍有不可消除的判断模糊性：不同的工程判断者可能在同一段实践上得出不同的“是否已闭合”的结论。这正是 §13 中描述的“工程僵化”和“治理空壳化”这两种对称错误在工程判断层面的共同根因。

14.3.3 通用可插拔 vs 平台深度特调的量化取舍

AC 范式 V6 已证明不同基座模型与 IDE 组合需要特调。但一个更根本的问题尚未被回答：在”平台深度特调”（将治理规则、Skill 描述、检查点频率、权限模式针对特定基座 + IDE 组合精调）和”通用可插拔”（设计一套可跨平台、跨基座、跨 IDE 移植的规则）之间存在真实的工程取舍。特调提供更高的局域精度但增加了维护成本和迁移成本；通用性提供更低的移植门槛但以牺牲局域精度为代价。目前，这个取舍只有定性的判断，缺乏一个量化的取舍标准——一个可以告诉你在给定的基座模型 A + IDE B 的组合中，”将这个 Skill 的描述改写为更贴近 IDE B 的工具协议风格”能带来多大的预期性能提升，以及这种提升是否值得付出”以后换 IDE 时要重写”的成本。

AC 范式 V7 正走向跨平台泛化——这本身就是一个间接的证据，说明”通用可插拔”方向在工程上是可行的。但 V6 的特调经验也给出了一个警告：跳跃式泛化可能丢失局域精度，而这种精度损失在人机协同的精细操作中可能被放大为摩擦力的显著增加。如何在两者之间做可量化的取舍，目前仍是一个必须被识别为开放问题的未闭合地带。

14.3.4 语义不封闭 → 稳定反馈的转化

四重语义不封闭与手工梯度下降的必然性之下，潜伏着一个更深刻的问题：如何将语义开放的现实价值（”架构设计是否合适””请示节点是否恰当”）转化为稳定、可归因、可学习的反馈信号？

将语义不封闭转化为稳定反馈的真正困难，不在于找不到评分器，而在于不存在”正确”的评分函数。”好”在不同利益相关者、不同时间尺度和不同上下文中的含义彼此冲突，且冲突本身没有客观仲裁标准。这个问题的结构类似于社会科学中的”概念操作化”问题，但后果更严重：操作化失败的直接后果不是一篇被拒稿的论文，而是一个在长程任务中发生价值漂移和假闭合的真实系统。

14.4 本章收束

本章的三个子节从不同维度回答同一个问题：本文的框架在什么条件下会被推翻？综合来看，本章勾勒了本文框架的三类结构性边界。评测边界：长程 Benchmark 被设计为暴露框架局限的工具。认知边界：H2 递归不可补偿性划定了验证链终端的不可逾越位置——人类判断力。工程边界：四个开放问题标记了当前论证的未闭合地带。

15 结论：从能力竞赛到治理建设

本文始于一篇深夜笔记里的朴素疑问：既然所有系统都在调用一模一样的模型 API，为什么最后长出来的 Agent 却天差地别？

沿着这个问题一路拆解，最终的答案不在模型，也不在工具。答案是治理。

当我们将目光从花哨的能力竞赛移开，深入到长程任务的结构底层时，一切都变得清晰起来：跨越时间线的状态连续性不会凭空产生，不可逆的动作后果也无法自动闭合。长程实践的严酷环境，逼迫着每一个存活下来的系统长出形态各异的「真核」器官。这根本不是什么架构风格的偏好，而是为了对抗熵增的结构必需。

这彻底改变了我们对 Agent 工程的认知。它意味着，不管基座模型进化得多聪明，都无法替代一套扎实的状态与权力控制机制；它也意味着，试图用全自动的闭环来抹除人类介入，不仅在工程上是不切实际的，在逻辑上更是自欺欺人的。

所以，怎么做出一个真正能用的 Agent 系统？

不要急着撑上限，先去保下限。承认反馈信号的残缺，承认人类判断力的不可替代。去建立一个状态可追溯、权力可约束的有界委托系统——让 Agent 走得足够远，但确保操作者随时能拉得住缰绳。

从无治理到有治理，这就是本文给出的最终回答。

16 致谢

- 感谢杭州市上城区钱江新城中央商务产业社区在调研对接、基础压力测试样例与真实业务用例方面提供的关键性支持
- 感谢深度求索（DeepSeek）提供 Deepseek-v4-pro 基座支持
- 感谢杭州市协同引荐平台社区（SRO）提供实质性用例支持
- 感谢彗星计划（Comet Project）成员提供技术支持
- 感谢 AI-Zion 社区大量成员提供了富有启发性的讨论，给出宝贵的建议
- 感谢 TRAE 提供关键 IDE 环境技术支持
- 感谢 Cherry Studio 提供关键技术支持

A 七系统演化证据：技术细节与源码锚点

以下按与正文 §4.1 相同的出生时序排列。每个系统列出正文叙事中未展开的关键技术细节，包含源码文件路径、类名、机制描述与对应的架构观察。所有信息均来源于对七个系统源码的第一手架构分析。

Cline（2024.06）

- **四层 SDK 分离。**核心架构分解为 shared/（共享类型与工具）、llms/（多模型 Provider 抽象）、agents/（无状态 Agent 循环逻辑）和 core/（运行时宿主绑定）。状态持久化与 Agent 循环完全解耦。

- **RuntimeHost 抽象**。统一运行时接口，三种实现：本地进程内宿主、共享 Hub 守护进程、远程 Hub 连接。同一套 Agent 逻辑不加修改地运行于 VS Code 扩展、CLI 和 JetBrains 插件。
- **Hub 守护进程**。协调面而非执行体。多个客户端共享同一 Agent 会话；断开后 Spoke 继续在 Hub 侧执行，重连后从持久化事件流恢复。
- **Cron Reconciler 模式**。全量文件扫描后更新自身持久化存储（源码锚点：数据库文件 `cron.db`），始终为真相源。文件系统 Watcher（~250ms debounce）仅作为“可能有变化”的信号——此项设计的目的是彻底消除文件监听漏报/误报。
- **Plan/Act 模式的起源**。早期用户自发形成了“先让 Agent 出计划、审阅、再执行”的两阶段习惯，后被固化为 Cline 标志性的 Plan/Act 模式。此例证明有价值的架构特征可以来源于使用模式而非初始设计。

Cherry Studio (2024.H2)

- **进程架构**。Main 进程 (Node.js) 为 LLM 调用和文件系统的唯一入口；Renderer 进程 (Chromium) 负责 UI 但默认无权访问这些能力。Agent 引擎在 Main 进程中运行，所有输出通过 contextBridge IPC 桥中转至 Renderer。
- **会话运行时状态机**。会话服务 AgentSessionRuntimeService 管理 Agent 生命周期。空闲 5 分钟 TTL 自动回收。崩溃恢复流程 reconcileStalePendingMessages() 将遗留的待处理消息标记为错误状态，防止静默丢失。
- **恢复令牌机制**。由驱动发出并持久化的恢复令牌，允许断开重建会话时从上次断点继续——而非启动全新会话。
- **多源 Hook 组合**。三种来源——内部观察器 (Agent.on())、功能扩展贡献 (hook-Parts)、调用方挂钩 (AiService)——通过组合器 composeHooks 以确定性策略合并。三种策略：chainVoid（按序执行，吞单条异常）、chained（接收前次返回值）、any-of（任一返回”重试”即整体重试）。

Claude Code (2025.02)

- **核心查询引擎**。源码文件 `src/QueryEngine.ts`，约 46,000 行。异步生成器驱动的检查查询循环汇聚六类逻辑：LLM 流式调用、工具调用检测、权限拦截、结果注入、token 预算检查和上下文压缩触发。这些逻辑在 0.2.x → 1.0.x → 2.0.x 迭代中逐步集中到同一文件——拆散反而引入同步复杂度。
- **三元组自包含模块**。每个工具由三个文件构成：核心实现 (`BashTool.ts`)、终端渲染 (`UI.tsx`) 和系统提示注入 (`prompt.ts`)。每增加一个工具只需添加这三个文件。

- **四层上下文降级路径。**工具结果替换 → history snip → compaction → overflow retry——渐进的降级顺序，越往后代价越大。每一层仅在前一层真正失效时才升级。
- **与 Codex CLI 的对照。**Claude Code 追求”压缩后足以推理”，Codex CLI 追求”完整可重建”——两者差异来自不同的迭代压力：Claude Code 面对上下文窗口预算压力，Codex CLI 面对安全审计需求。

OpenCode (2025.03)

- **Go → TypeScript+Bun 重写。**SST/Anomaly 创始人 Dax Raad 于 2025 年 4 月 22 日首次贡献代码，5 月起主导 backend 从 Go 到 TypeScript+Bun 的全量重写。动机为 SST 团队对 TypeScript 生态的熟悉，以及通过 Vercel AI SDK 接入 models.dev 上百个 Provider。TUI 于 2025 年 10 月 31 日从 Go Bubble Tea 重写为自研 OpenTUI (commit message: “DELETE GO BUBBLETEA CRAP HOORAY”)。
- **System Context 代数。**Backend 重写产生的 client-server 架构逼出了此系统：每个 Context Source 含稳定 Key + JSON Codec + Infallible Loader + Pure Renderers。四种操作——make / combine / reconcile / replace——覆盖从初始生成到 compaction 后重建的全生命周期。
- **Context Epoch。**上述操作的持久化载体。Baseline 跨进程重启逐字复用；Snapshot 在 Safe Provider-Turn Boundary 原子推进。
- **不透明工具 (Opaque Tool)。**工具行为通过 WeakMap 隐藏 executor，契约仅暴露 I/O 类型。目的不是安全或容错，而是保证进程分离后的工具声明一致性——前端从不过问工具实现，仅通过契约描述即可渲染正确 UI。

Codex CLI (2025.04)

- **多平台 OS 级沙箱。**覆盖三种操作系统：Linux 使用 bubblewrap + Landlock，macOS 使用 Seatbelt (sandbox-exec)，Windows 使用 Elevated + Restricted Token 组合。该体系在 2025 年 6 月启动 Rust 全量重写后加入——出生时的第一版 TypeScript 代码中并无多平台沙箱。
- **分级审批机制。**四档审批强度。审批者既可以是人类，也可以是独立的自动审查 Agent (Guardian)——七家中唯一在审批链中引入非人类审查者的设计。
- **全量事件持久化。**会话事件按行追加到 JSONL，SQLite 提供结构化查询。事件记录器 RolloutRecorder 和状态库 StateDB 构成持久化层。修复流程 ScanAndRepair 能由事件流重建损坏的索引——Codex 的状态不是”快照”，而是一条可回放、可重建的完整事件流。

trae-agent (2025.06)

- **与 TRAE IDE 的关系。** trae-agent 是 2025 年 6 月 14 日首次 commit 的 MIT 许可开源项目 (289 commits, 约 8 个月活跃开发至 2026.02)。TRAE IDE 是 ByteDance 于 2025 年 1 月 20 日发布的闭源 VS Code fork, 12 个月内获 600 万+注册用户。trae-agent 从闭源产品中蒸馏出核心 Agent 循环、工具系统、LLM 客户端抽象和评估体系, 但剥离了全部 VS Code/Electron 集成。
- **Lakeview 元认知层。** 独立的 LLM 调用专门分析主 Agent 每一步轨迹。Extractor 从原始轨迹提取”Agent 在这一步做了什么”, Tagger 将步骤分类为预定义标签 (WRITE_TEST、EXAMINE_CODE、WRITE_FIX 等)。本质上是一个”观察 Agent 的 Agent”——服务于事后研究分析而非运行时优化。
- **全量轨迹 JSON 审计追踪。** 审计记录器 trajectory_recorder.py 记录每步的状态、思考、工具调用、工具结果和模型用量, 以及每次 LLM 交互的完整消息和响应。与 Codex CLI 的 JSONL 在形式上相似, 但目的完全不同: Codex 的 JSONL 是运行时恢复基础设施, trae-agent 的轨迹是事后分析工具。
- **研究定位。** trae-agent 的 README 自述为 “an ideal platform for studying AI agent architectures, conducting ablation studies, and developing novel agent capabilities”。源码结构与此一致: 短程可重复的任务设计、全程可观察的实验室环境和变量可控的消融实验需求, 直接决定了哪些机制进入代码库、哪些被有意排除。

Hermes (2025.07)

- **双重用途架构。** environments/ 目录内含 RL 训练环境, batch_runner.py 支持并行轨迹生成, 整体设计兼容 Nous 的 Atropos RL 框架。Agent 执行任务 → 生成 trajectory → trajectory 训练下一代模型 → 更强模型驱动更强 Agent——构成闭环。
- **Prompt 缓存不可侵犯原则。** 系统提示词在一个 session 中只构建一次。工具变更和新知识通过用户消息而非系统提示词注入。工具注册体系——每个工具模块在加载时自行登记 (tools/ 目录 + registry.py); 发现阶段先以 AST 静态扫描确认登记行为再决定是否导入。
- **瞬态故障容忍 (check_fn)。** 双层保护: 30 秒 TTL 缓存避免频繁探测; 60 秒 last-good grace——一次 True→False 翻转若在最近成功 60 秒内, 视为毛刺, 返回上次 True 且不缓存失败结果。此设计专门应对多平台环境的外部状态抖动 (例如 Docker daemon 响应延迟 2 秒)。
- **Skills-Curator 知识闭环。** Agent 完成任务后自动创建或更新可复用的 Markdown skill 文件。Curator 负责技能库的生命周期管理——统计使用频率、标记过期、归档不删除。Memory Provider 维护跨会话的事实性记忆。

- **社区贡献。** v0.2.0 公开后两周内，63 位贡献者合入 216 个 PR。最被社区压力逼出的结构是多平台 Messaging Gateway——Telegram、Discord、Slack、WhatsApp、Signal、iMessage 等 20+ 消息平台的统一接入层。

References

- [1] LIU B, LI X, ZHANG J, et al. Advances and Challenges in Foundation Agents: From Brain-Inspired Intelligence to Evolutionary, Collaborative, and Safe Systems[EB/OL]. 2025 [2026-07-22]. <https://arxiv.org/abs/2504.01990>. arXiv: 2504.01990.
- [2] WANG L, MA C, FENG X, et al. A Survey on Large Language Model based Autonomous Agents[EB/OL]. 2023 [2026-07-22]. <https://arxiv.org/abs/2308.11432>. arXiv: 2308.11432.
- [3] MENG Q, WANG Y, CHEN L, et al. Agent Harness for Large Language Model Agents: A Survey[EB/OL]. 2026 [2026-07-22]. <https://arxiv.org/abs/2605.29682>. arXiv: 2605.29682.
- [4] LIU F, LV B, LIU Y. From Observer to Agent: On the Unification of Physics and Intelligence Science[EB/OL]. 2024 [2026-07-22]. <https://www.preprints.org/manuscript/202410.0479/v1>.
- [5] ZENG Q, WEI D. 长程实践：超越单次能力的结构下限 (Long-Horizon Practice: Structural Lower Bounds Beyond Single-Step Capability)[EB/OL]. 2026 [2026-07-22]. <https://doi.org/10.5281/zenodo.20470866>.
- [6] ZENG Q, WEI D. 面向长程实践的锚点与契约：AC 范式 V6 技术报告 (Anchors and Contracts for Long-Horizon Practice: The AC Paradigm V6 Technical Report)[EB/OL]. 2026 [2026-07-22]. <https://doi.org/10.5281/zenodo.20471461>.
- [7] Anthropic. Best Practices for Claude Code[EB/OL]. 2025 [2026-05-21]. <https://www.anthropic.com/engineering/claude-code-best-practices>.